Contents

1 字符串 2
1.1 最小表示法 2

2 数据结构 2
2.1 LCT 2
2.2 广义串并联图 3
2.3 四边形不等式 3
2.3.1 决策单调性分治 3
2.3.2 SMAWK 4
2.3.3 Wilber 5
2.3.4 二分栈 5
2.4 wqs 二分构造方案 5
2.5 O(1) RMQ 6

3 图论 7
3.1 费用流 7
3.2 支配树 8
3.3 一般图最大匹配 8

4 数学 9
4.1 Barret 约简 9
4.2 拉格朗日插值 9
4.3 类欧 9
4.4 反射容斥 10

5 数论 10
5.1 分数还原 10
5.2 O(V)-O(1) GCD 10
5.3 离散对数 11
5.4 中国剩余定理 11
5.5 Miller-Rabin 素性检验 11
5.6 Pollard-Rho 分解质因数 12
5.7 最小模线性值 12
5.8 二次剩余 12
5.9 原根与阶 13
5.10 积性函数 13
5.10.1 杜教筛 13
5.10.2 min_25 14
5.11 下取整和 15
5.12 高斯整数 15

6 多项式 16
6.1 任意模数 NTT 16
6.2 多项式复合 16
6.3 多项式复合逆 18
6.4 最短线性递推 20
6.5 线性递推第 n 项 21
6.6 复数 FFT 卷积 21
6.7 多点求值 21
6.8 多项式指数 22
6.9 多项式对数 22
6.10 多项式开根 22
6.11 多项式求逆 22
6.12 带余除法 23
6.13 快速插值 23

7 计算几何 23
7.1 基础几何库 23
7.2 Delaunay 三角剖分 26
7.3 Voronoi 图 28
7.4 三维凸包 28
7.5 三维向量 29
7.6 三维平面 30
7.7 三维旋转 31
7.8 三维直线与线段 32
7.9 上凸壳 33
7.10 凸包内点判定 34
7.11 半平面交 34
7.12 圆相关求交与面积交 35
7.13 多边形包含 36
7.14 最小圆覆盖 37
7.15 最近点对 37
7.16 简单多边形三角剖分 38
7.17 多边形重心 39

8 通用 39
8.1 大质数表 39
8.2 常数速查表 39

1 字符串

1.1 最小表示法

```cpp
// a 是支持 size() 与下标的只读序列;返回最小循环表示的起点,空序列返回 0。
template <class T> int minrep(const T &a) {
    int n = a.size(), i = 0, j = 1, k = 0;
    while(i < n && j < n && k < n) {
        if(a[(i + k) % n] == a[(j + k) % n]) {
            ++k;
        } else {
            if(a[(i + k) % n] > a[(j + k) % n]) swap(i, j);
            j += k + 1;
            if(j == i) ++j;
            k = 0;
        }
    }
    return i;
}
// rotate(a.begin(), a.begin() + minrep(a), a.end());
```

2 数据结构

2.1 LCT

$O(\log n)$ 摊还的动态 lct:支持 link/cut/路径聚合(默认异或,改 pu 可换求和或最值)。

- `init()`: 先 `n` 与 `a[N]` 就位,然后对 $1..n$ 初始化
- `link(u, v)`: 连边(自动判环);`cut(u, v)`: 断边;`que(u, v)`: 路径聚合值;`upd(u, x)`: 单点改值并维护
- `mkr/access/splay/fd` 为内部机制,一般不用管
- 依赖: 全局 `N` 与点权数组 `a[N]`

```cpp
namespace LCT {
#define ls(k) ch[k][0]
#define rs(k) ch[k][1]
int ch[N][2], fa[N];
bool lz[N];
int tr[N];
// 向上维护辅助树异或和；要求左右儿子与 a[k] 已是当前值；修改 tr[k]。
inline void pu(int k) {
    tr[k] = tr[ls(k)] ^ tr[rs(k)] ^ a[k];
}
// 翻转一棵辅助树根；k=0 时无操作；修改懒标记与左右儿子。
inline void pl(int k) {
    if(k) lz[k] ^= 1, swap(ls(k), rs(k));
}
// 下传翻转标记；要求 k 是待访问辅助树结点；清空 lz[k]。
inline void pd(int k) {
    if(lz[k]) pl(ls(k)), pl(rs(k)), lz[k] = 0;
}
// 判断 k 是否为父亲的右儿子；要求 k 非当前辅助树根。
inline bool gc(int k) { return ch[fa[k]][1] == k; }
// 判断 k 是否为辅助树根；虚父也视为根；返回当前 splay 根状态。
inline bool ir(int k) { return ch[fa[k]][gc(k)] != k; }
// 从辅助树根到 k 下传懒标记；要求数组容量足够一条实链长度。
void pda(int k) {
    static int st[N], t;
    st[++t] = k;
    while(!ir(k)) st[++t] = k = fa[k];
    while(t) pd(st[t--]);
}
// 单次旋转；要求 u 不是辅助树根；修改父子指针并维护原父亲。
void rot(int u) {
    int f = fa[u], g = fa[f], t = gc(u), x = ch[u][!t];
    if(!ir(f)) ch[g][gc(f)] = u;
    fa[u] = g, fa[f] = u;
    ch[u][!t] = f, ch[f][t] = x;
    if(x) fa[x] = f;
    pu(f);
}
// 将 u 旋到当前辅助树根；会先下传路径标记并维护 tr[u]。
void splay(int u) {
    for(pda(u); !ir(u); rot(u))
        if(!ir(fa[u])) {
            rot(gc(u) == gc(fa[u]) ? fa[u] : u);
        }
    pu(u);
}
// 打通根到 u0 的首选路径；结束后 u0 为辅助树根且其右儿子为空。
void access(int u0) {
    for(int u = u0, p = 0; u; p = u, u = fa[u]) splay(u), rs(u) = p, pu(u);
    splay(u0);
}
// 查询 u 所在原树的根；会改变辅助树形态并把根 splay 到顶。
int fd(int u) {
    access(u);
    while(ls(u)) u = ls(u), pd(u);
    return splay(u), u;
}
// 将 u 设为所在原树的根；要求 u 是有效点号；修改翻转标记。
void mkr(int u) {
    access(u), pl(u);
}
// 初始化点权聚合；要求 n 与 a[1..n] 已设置，且全局 LCT 数组为干净状态。
void init() {
    For(i, 1, n) pu(i);
}
// 连接两棵树；若 u、v 已连通则不修改结构。
void link(int u, int v) {
    mkr(u);
    if(fd(v) != u) mkr(v), fa[v] = u;
}
// 删除边 u-v；要求该边存在时才保证语义正确；不存在时只按当前实现断开直接父关系。
void cut(int u, int v) {
    mkr(u), access(v), access(u);
```


2

```
    if(fa[v] == u) fa[v] = 0;
}
// 修改单点权值；要求 u 已在当前森林中；更新 a[u] 与其辅助树聚合。
void upd(int u, int x) {
    a[u] = x, splay(u);
}
// 查询 u 到 v 路径点权异或；要求 u、v 连通；返回路径上 a 的异或和。
int que(int u, int v) {
    mkr(u), access(v);
    return tr[v];
}
}  // namespace LCT
```

2.2 广义串并联图

```
// 新图使用值初始化实例,先设 m = 原边数;add 的原边编号为 1..m,无自环。
// 顶点数与最终结点数须 < N。typ 为真表示串联,假表示并联;fa 是约简森林的父结点。
// build 保留不可约核,不构造任意图的完整 SPQR 分解;删点的邻接表可有残留。
struct SPQR_tree {
    int m = 1, fa[N];
    bool typ[N];
    unordered_map<int, int> t[N];
    bitset<N> vis;
    // 新建串/并联内部结点；要求 u、v 是已有边/内部结点编号；返回新结点编号并设父指针。
    int node(int u, int v, bool kind) {
        typ[++m] = kind;
        fa[u] = fa[v] = m;
        return m;
    }
    // 加入无自环边 u-v，边号为 w；平行边会合并成新的并联结点并覆盖邻接表边号。
    void add(int u, int v, int w) {
        if(t[u].count(v)) {
            t[u][v] = t[v][u] = node(t[u][v], w, 0);
        } else {
            t[u][v] = t[v][u] = w;
        }
    }
    // 约简 n 点图的度 1/2 顶点；要求已先 add 原边；修改 fa/typ/t 并保留不可约核。
    void build(int n) {
        queue<int> q;
        auto ins = [&](int u) { if (!vis[u] && t[u].size() <= 2) vis[u] = 1,
q.push(u); };
        For(i, 1, n) ins(i);
        while(q.size()) {
            int u = q.front();
            q.pop();
            if(t[u].empty()) continue;
            if(t[u].size() == 1) {
                int v = t[u].begin()->first;
                t[v].erase(u), ins(v);
            } else {
                int x = t[u].begin()->first, y = next(t[u].begin())->first;
                int w = node(t[u][x], t[u][y], 1);
                t[x].erase(u), t[y].erase(u);
                add(x, y, w);
                ins(x), ins(y);
            }
        }
    }
} d1;
int *fa = d1.fa, &m = d1.m;
bool *typ = d1.typ;
```

2.3 四边形不等式

区间包含单调性：区间越大权值越大，即 $a \le b \le c \le d$ 时 $w(b,c) \le w(a,d)$。

四边形不等式：两段交叉着取的代价，不会比一段包住另一段更大，即 $a \le b \le c \le d$ 时

$$w(a,d) + w(b,c) \ge w(a,c) + w(b,d)$$

令 $a = l-1$、$b = l$、$c = r$、$d = r+1$，它就等价于

$$w(l-1, r+1) + w(l,r) \ge w(l, r+1) + w(l-1, r)$$

，验性质时一般就随机代这个相邻的式子。求 max 时把 w 取负，下面都只讲 min。

1D1D DP：形如 $f_i = \min_{j<i}\{f_j + w(j,i)\}$，记 p_i 为让 f_i 取到最小的 j。w 满足四边形不等式时 p_i 单调不降。理由：对 $i<j<k<l$，四边形不等式等价于

$$w(j,l) - w(j,k) \le w(i,l) - w(i,k)$$

，也就是说 j 在 k 处比 i 优，那它在 l 处也一定还比 i 优 —— 决策点只会往右挪。

- w 能 $O(1)$ 算：从前往后用单调栈维护每段的决策点、二分分界，就是二分栈，$O(n \log n)$ 次 w；
- p 与 f 无关（离线）时：分治，同样 $O(n \log n)$；SMAWK 做到 $O(n)$，配合 Wilber 还能在线 $O(n)$。

区间 DP：形如 $f_{l,r} = \min_{l \le i < r}\{f_{l,i} + f_{i+1,r}\} + w(l,r)$。$w$ 同时满足区间包含单调性与四边形不等式时，f 也满足四边形不等式，最优决策点满足

$$p_{l,r-1} \le p_{l,r} \le p_{l+1,r}$$

，每个区间只在这对上下界之间枚举，总共 $O(n^2)$。

2D1D DP：形如 $f_{i,j} = \min_{k<j}\{f_{i-1,k} + w(k,j)\}$。$w$ 满足四边形不等式时 $f_{i,j}$ 关于 i 下凸，可以 WQS 二分掉一维，化成一串 1D1D（见 wqs 一页）。

好用的判据：只跟区间长度有关、且关于长度下凸的代价（如 $w(l,r) = (r-l+1)^2$）一定满足四边形不等式；邮局问题里“一段村庄到一个中位数的距离和”也是。拿不准就按上面那个相邻式子随机对拍。

2.3.1 决策单调性分治

求 $f_i = \min_{j<i}\{f_j + w(j,i)\}$（$f_0 = 0$），要求 w 满足四边形不等式。代价不直接求值，而是用两个游标各自的滑动窗口维护：每次只把某个端点挪一格，靠 add/del 增量更新，适合「代价只能边走边维护」的题。

- `work(n)`：算出 f_0 到 f_n，结果在 `f[]`，最优决策点记在 `p[]`
- 使用者先提供 `struct W`：`ll w` 存当前窗口代价，`add(x)/del(x)` 把下标 x 加入/移出窗口，`clear()` 清空；模板开两个独立实例 `W Wa, Wb`
- 窗口是 $(l,r]$，即下标 $l+1$ 到 r 这些元素；整轮共 $O(n\log n)$ 次移动
- 与「二分栈」同样是在线 $O(n\log n)$，都要求四边形不等式

```
// 代价 w(l, r) 用两个滑动窗口各自维护,使用者先提供
//   struct W {
//       ll w = 0;                  // 当前窗口 (l, r](下标 l+1..r)的代价
//       void add(int x);           // 把下标 x 加进窗口并更新 w
//       void del(int x);           // 把下标 x 移出窗口并更新 w
//       void clear();              // 清空窗口,w 归零
//   };
// 模板会开两个独立实例 Wa, Wb(两个游标各一个窗口),所以 W 的状态要能存两份。

ll f[N];
int p[N];
struct wcur {
    W &w;
    int l, r;
    // 移动左端点到 x，维护窗口 (l,r]；要求移动过程中下标在调用方有效范围内。
    void ml(int x) { while(l > x) w.add(l--); while(l < x) w.del(++l); }
    // 移动右端点到 x，维护窗口 (l,r]；要求移动过程中下标在调用方有效范围内。
    void mr(int x) { while(r < x) w.add(++r); while(r > x) w.del(r--); }
    // 清空游标窗口到 (0,0]；可重复调用；同时清空底层 W 状态。
    void rs() { w.clear(), l = r = 0; }
};
W Wa, Wb;
wcur s{Wa, 0, 0}, t{Wb, 0, 0};
// 用候选前驱 k 更新 f[i]；要求 w 是区间 (k,i] 的代价。
void relax(ll w, int i, int k) {
    ll v = f[k] + w;
    if(v < f[i]) f[i] = v, p[i] = k;
}
// 分治填充开区间 (l,r) 的 DP；要求 p[l]..p[r] 决策单调且 f[l],f[r] 已可用。
void dnc(int l, int r) {
    if(r - l == 1) return;
    int m = (l + r) >> 1;
    s.mr(m), s.ml(p[l]);
    For(k, p[l], p[r]) {
        relax(Wa.w, m, k);
        if(k < p[r]) s.ml(k + 1);
    }
    s.ml(p[l]), s.mr(l);
    dnc(l, m);
    t.mr(r), t.ml(l + 1);
    For(k, l + 1, m) {
        relax(Wb.w, r, k);
        if(k < m) t.ml(k + 1);
    }
    t.ml(m), t.mr(m);
    dnc(m, r);
}
// 计算 f[0..n] 与决策 p[0..n]；可重复调用，会先清空两个滑动窗口。
void work(int n) {
    s.rs(), t.rs();
    f[0] = 0, p[0] = 0;
    if(!n) return;
    For(i, 1, n) f[i] = (ll) 4e18, p[i] = 0;
    s.mr(n), s.ml(0), relax(Wa.w, n, 0), s.ml(0), s.mr(0);
    dnc(0, n);
}
```

2.3.2 SMAWK

离线求全单调矩阵每行的最小值：n 行 m 列，$O(n+m)$ 次询问，比逐行扫快。

- `SMAWK<T>(n, m, que)`：`que(i, j)` 给第 i 行第 j 列的值（$1\le i\le n$，$1\le j\le m$）
- 返回 `{f, fp}`：`f[i]` 是第 i 行的最小值，`fp[i]` 是取到最小值的列
- 前提：矩阵全单调（每行的最左最小值位置随行号单调不降）；由四边形不等式导出的矩阵都满足
- `T` 要显式写，例如 `auto [f, fp] = SMAWK<ll>(n, m, que);`

```
// 求全单调矩阵每行最小值；行列均 1-based，要求 que(i,j) 可比较；返回 f 与最小列 fp。
template <typename T, typename Q> pair<vect<T>, vect<int>> SMAWK(int n0, int m, Q &&que) {
    vect<T> f(n0 + 1, T());
    vect<int> fp(n0 + 1, 0), b(m + 1, 0);
    For(i, 1, m) b[i] = i;
    auto F = [&](auto &&F, int d, int m) -> void {
        int n = (n0 >> d), k = 1, j = 1;
        if (!n) return;
        if (n < m) { // 行数少于列数:先删掉不可能成为行最小值位置的列
            while (j < m) {
                if (que(k << d, b[j]) > que(k << d, b[j + 1])) {
                    if(k == 1) ++j;
                    else b[j] = b[--k];
                } else {
                    if(k < n) b[k++] = b[j];
                    else b[j + 1] = b[j];
                    ++j;
                }
            }
            b[k] = b[m];
            m = k;
            j = 1;
        }
        // 同名新变量:每层留一份截断后的列序供本层取用,不要改写成对外层 b 的赋值
        vect<int> b2 = b.substr(0, m + 1), b = move(b2);
        F(F, d + 1, m);
        For(i, 1, n) if (i % 2 == 1) {
            T &s = f[i << d];
            int &p = fp[i << d];
                p = b[j];
```


4

```
31                 s = que(i << d, p);
32             while (j < m && (i == n || b[j + 1] <= fp[(i + 1) << d])) {
33                 T s2 = que(i << d, b[++j]);
34                     if(s2 < s) s = s2, p = b[j];
35             }
36         }
37     };
38     F(F, 0, m);
39     return {f, fp};
40 }
```

2.3.3 Wilber

在线求 $f_i = \min_{j<i}\{f_j + w(j,i)\}$（$f_0 = 0$），只要 $O(n)$ 次 que，比二分栈少一个 log。

- Wilber<T>(n, que, Z)：que(j, i) 给 $w(j,i)$（$j < i$）；Z 是极大值，ll 用 LLONG_MAX / 4
- 返回 {f, fp}：f[i] 是最小值，fp[i] 是最优决策点
- 前提：w 满足四边形不等式；依赖先提供 SMAWK
- 模板参数 B（默认 16）：小区间直接扫的阈值，可调

```
1  // 求在线 1D1D DP；要求 que(k,i) 定义于 0 <= k < i <= n 且决策单调；返回 f 与前驱 fp。
2  template <typename T, int B = 16, typename Q>
3  pair<vect<T>, vect<int>> Wilber(int n, Q &&que, const T Z =
   numeric_limits<T>::max()) {
4      vect<T> f(n + 1, Z);
5      f[0] = 0;
6      vect<int> fp(n + 1, 0);
7      int c = 0;
8      while (c < n) {
9          int r = fp[c], p = min(n, c + (c - r + 1));
10         if (c - r + 1 <= B) {
11             For(j, c + 1, p) For(i, fp[j - 1], j - 1) {
12                 T v = f[i] + que(i, j);
13                     if(v < f[j]) f[j] = v, fp[j] = i;
14             }
15             c = p;
16             continue;
17         }
18         auto q1 = [&](int x, int y) { return f[y + r - 1] + que(y + r - 1, x +
   c); };
19         auto [g, gp] = SMAWK<T>(p - c, c - r + 1, q1);
20         For(i, c + 1, p) f[i] = g[i - c], fp[i] = gp[i - c] + r - 1;
21         auto q2 = [&](int x, int y) { return x > y ? f[y + c] + que(y + c, x + c) :
   Z; };
22         auto [h, hp] = SMAWK<T>(p - c, p - c, q2);
23         int nc = p; // 默认整段归纳;一旦某行能由第二段取得更优值,就停在该行重算
24         For(i, c + 1, p) {
25             if (f[i] > h[i - c]) {
26                 f[i] = h[i - c];
27                 fp[i] = hp[i - c] + c;
28                 nc = i;
29                 break;
30             }
31         }
32         c = nc;
33     }
34     return {f, fp};
35 }
```

2.3.4 二分栈

依赖: 全局 N(数组)与 calc(l, r) 函数。要求 calc 满足四边形不等式(决策单调)。

```
1  struct nd {
2      ll x;
3      int c;
4      // 按（总代价,段数）字典序比较；返回当前状态是否更优。
5      bool operator<(const nd b) const {
6          return x < b.x || (x == b.x && c < b.c);
7      }
8      // 合并两个 DP 状态；要求代价相加不溢出 ll。
9      nd operator+(const nd b) const { return {x + b.x, c + b.c}; }
10 };
11 // 计算带罚分 dt 的最优划分；要求全局 n 与 calc(l,r) 已设置；返回 f[n] 的值和段数。
12 nd work(ll dt) {
13     static nd f[N];
14     deque<pii> q;
15     f[0] = {0, 0}, q.push_back({n, 0});
16     auto get = [&](int j, int i) -> nd {
17         return f[j] + nd{calc(j + 1, i) - dt, 1};
18     };
19     For(i, 1, n) {
20         f[i] = get(q.front()[1], i);
21         if(q.front()[0] <= i) q.pop_front();
22         while(q.size()) {
23             auto [R, p] = q.back(); q.pop_back();
24             int L = q.size() ? q.back()[0] + 1 : i + 1;
25             if(L > R) continue;
26             if(!(get(p, L) < get(i, L))) continue;
27             int l = L, r = R + 1;
28             while(l + 1 < r) {
29                 int m = (l + r) >> 1;
30                 if(get(p, m) < get(i, m)) l = m;
31                 else r = m;
32             }
33             q.push_back({l, p});
34             break;
35         }
36         q.push_back({n, i});
37     }
38     return f[n];
39 }
```

2.4 wqs 二分构造方案

外层 WQS 二分惩罚值,内层调用 Wilber 做区间 DP,把段数打包在值的低位,最终构造分割点数组。

- 接口: `wqs(n, k, que, up)`;`que(l, r)` 返回 i128,下标约定须与内层一致。本书 `Wilber` 使用 0-based 边界,计算区间 `l+1..r` 的代价
- 前提: $n \geq 1$、$1 \leq k \leq n$,恰好 k 段的最优值关于 k 凸;`up >= 0`,二分范围 -up..1 必须覆盖所需惩罚值
- 打包值与 DP 中间和须在 i128 和内层无穷大范围内。零代价允许;负值用乘 2^B 打包,不能左移负数
- 返回: 0-based 分割点数组(以 0 起、n 终),恰含 k 个区间段
- 依赖: 事先提供 `Wilber<T>(n, que, INF)`,返回 `pair<vect<T>, vect<int>>`(DP 值、前驱边界);本书实现还需 `SMAWK`
- 复杂度: `O(log C)` 次内层 Wilber

```cpp
// n >= 1,1 <= k <= n;up >= 0,二分范围须覆盖所需惩罚值,OPT(k) 关于 k 凸。
// 打包值与 DP 中间和须在 i128 及内层 Wilber 的无穷大范围内。
// 构造恰好 k 段的最优划分；que(l,r) 为 (l,r] 的代价，0 <= l < r <= n。
// 返回以 0 开始、n 结束的划分边界序列。
template <typename Q> vect<int> wqs(int n, int k, Q &&que, i128 up) {
    int B = __lg(n) + 1;
    ll T = (1ll << B) - 1;
    auto chk = [&](i128 dt) {
        return Wilber<i128>(
            n, [&](int l, int r) { return (que(l, r) - dt) * ((i128)1 << B) + 1; }, 1e36);
    };
    auto get = [&](i128 dt) {
        auto [f, fp] = chk(dt);
        vect<int> pt;
        for (int i = n; i; i = fp[i]) pt += i;
        pt += 0;
        reverse(all(pt));
        return pt;
    };
    i128 l = -up - 1, r = 1;
    while (l + 1 < r) {
        i128 m = l + r >> 1;
        if((chk(m).first[n] & T) < k) l = m;
        else r = m;
    }
    auto fp = get(l), gp = get(l + 1);
    if (gp.size() - 1 == k) return gp;
    int j = 1;
    For(i, 1, (int)gp.size() - 2) {
        while (fp[j] < gp[i]) ++j;
        if (fp[j - 1] <= gp[i - 1] && fp.size() - j + i - 1 == k) {
            return gp.substr(0, i) + fp.substr(j);
        }
    }
    assert(0);
}
```

2.5 O(1) RMQ

```cpp
template <typename T, typename cmp = less<T>, int dt = 1>
struct rmq_t {
    static constexpr int B = 20;
    vector<T> a, pre, suf;
    vector<vector<T>> st;
    vector<int> f;
    // 查询整块稀疏表区间 [L,R] 的最优块值；要求 0 <= L <= R 且块编号有效。
    T _que(int L, int R) {
        int k = __lg(R - L + 1);
        return max(st[k][L], st[k][R - (1 << k) + 1], cmp());
    }
    // 查询第 k 个块内原数组闭区间 [L,R]；要求 L,R 属于同一块且 f 已由 bd 构造。
    T _que_bitc(int k, int L, int R) {
        int l = k * B, x = f[R] & ~((1 << (L - l)) - 1);
        return a[l + __lg(x & -x)];
    }
    // 构建 RMQ；要求 n >= 1，_a[dt..dt+n-1] 可读；覆盖内部数组与稀疏表。
    void bd(int n, T *_a) {
        a.resize(n), pre.resize(n), suf.resize(n), f.resize(n);
        For(i, 0, n - 1) pre[i] = suf[i] = a[i] = _a[i + dt];
        int k = (n - 1) / B;
        st.resize(__lg(k + 1) + 1), st[0].resize(k + 1);
        For(i, 0, k) {
            int l = i * B, r = min(i * B + B - 1, n - 1);
            rFor(i, r - 1, l) pre[i] = max(pre[i], pre[i + 1], cmp());
            For(i, l + 1, r) suf[i] = max(suf[i], suf[i - 1], cmp());
            st[0][i] = suf[r];
            static int b[B + 1];
            int t = 0, s = 0;
            For(i, l, r) {
                while(t && cmp()(a[b[t]], a[i])) s ^= 1 << (b[t--] - l);
                f[i] = s ^= 1 << (i - l);
                b[++t] = i;
            }
        }
        For(i, 1, __lg(k + 1)) {
            int m = k - (1 << i) + 1;
            st[i].resize(m + 1);
            For(j, 0, m) st[i][j] = max(st[i - 1][j], st[i - 1][j + (1 << (i - 1))], cmp());
        }
    }
    // 从连续容器构建；要求 v 非空，查询下标仍按 dt 偏移约定。
    void bd(vector<T> &v) { bd(v.size(), v.data() - dt); }
    // 用生成函数构建；func 收到下标 dt..dt+n-1，结果覆盖当前结构。
    template <class F> void bd(int n, F func) {
        vector<T> v(n);
        For(i, 0, n - 1) v[i] = func(i + dt);
        bd(v);
    }
    // 查询闭区间 [l,r] 最优值；要求已 bd，dt <= l <= r < dt+n。
    T que(int l, int r) {
        l -= dt, r -= dt;
        int il = l / B, ir = r / B;
        if(il == ir) {
```


6

```
            return _que_bitc(il, l, r);
        } else {
            T s = max(pre[l], suf[r], cmp());
            if(il + 1 < ir) s = max(s, _que(il + 1, ir - 1), cmp());
            return s;
        }
    }
};
```

3 图论

3.1 费用流

```
// 点编号 1..n；容量为非负 int、费用为 ll；要求无可增广负环且总流量/费用在返回类型内。
struct MCMF_t {
    static constexpr int N = 1e6 + 10, FL = 1e9;
    static constexpr ll Z = 1e18;
    struct edge {
        int v, n, w;
        ll c;
    } e[N];
    int n, mx, hd[N], tot = 1, s, t, p[N];
    // 加一条残量边；要求容量非负；只修改邻接表和边池。
    void _add(int u, int v, int w, ll c) {
        e[++tot] = {v, hd[u], w, c}, hd[u] = tot;
    }
    // 加一条有向费用容量边；要求端点为正、容量非负；平行边保留为多条边。
    void add(int u, int v, int w, ll c) {
        // mx 是 clear() 的清零点,add() 就得抬它(只靠 mcmf() 抬的话,建完图没跑过就 clear() 会残留)
        cmax(mx, u), cmax(mx, v);
        _add(u, v, w, c), _add(v, u, 0, -c);
    }
    // 清空用过点的邻接表；要求边已不再需要；重置边池和源汇但保留 mx 作为清零上界。
    void clear() {
        // 清到"用过的最大点数" mx:mcmf2() 的内层会用 _n + 2 个点跑,
        // 只清到当前 n 的话,超级源/汇那几条残留 hd[] 会让下一次调用拿陈旧前驱、
        // dijkstra 回退时在 e[] 里乱跳 → 死循环。
        For(i, 1, mx) hd[i] = 0;
        tot = 1, s = t = 0;
    }
    ll h[N], d[N];
    // 求初始势能；要求 s,n 已设置且不存在可达负环；覆盖 h[1..n]。
    void spfa() {
        static bool vis[N];
        For(i, 1, n) h[i] = Z;
        queue<int> q;
        q.push(s), h[s] = 0, vis[s] = 1;
        while(q.size()) {
            int u = q.front();
            q.pop();
            vis[u] = 0;
            for(int i = hd[u]; i; i = e[i].n) {
                int v = e[i].v;
                if(e[i].w && h[u] + e[i].c < h[v]) {
                    h[v] = h[u] + e[i].c;
                    if(!vis[v]) q.push(v), vis[v] = 1;
                }
            }
        }
    }
    // 用非负约化费用找最短增广路；要求势能 h 有效；修改 d,p,h，返回 t 是否可达。
    bool dij() {
        For(i, 1, n) d[i] = Z;
        priority_queue<pair<ll, int>> q;
        d[s] = 0;
        q.emplace(0, s);
        while(q.size()) {
            auto [tmp, u] = q.top();
            q.pop();
            if(-tmp != d[u]) continue;
            for(int i = hd[u]; i; i = e[i].n)
                if(e[i].w) {
                    int v = e[i].v;
                    auto nd = d[u] + h[u] - h[v] + e[i].c;
                    if(nd < d[v]) d[v] = nd, p[v] = i, q.emplace(-nd, v);
                }
        }
        For(i, 1, n) if(d[i] != Z) d[i] += h[i], h[i] = d[i];
        return d[t] != Z;
    }
    // 求最小费用最大流；要求图无负环、点编号 1.._n；mcf=1 时只推负费用增广，返回(流量,费用)。
    pair<int, ll> mcmf(int _s, int _t, bool mcf = 0, int _n = 0) {
        s = _s, t = _t, n = _n, cmax(mx, _n);
        spfa();
        int a1 = 0;
        ll a2 = 0;
        while(dij() && (!mcf || d[t] < 0)) {
            int m = FL;
            for(int u = t; u != s; u = e[p[u] ^ 1].v) m = min(m, e[p[u]].w);
            a1 += m, a2 += m * d[t];
            for(int u = t; u != s; u = e[p[u] ^ 1].v) e[p[u]].w -= m, e[p[u] ^ 1].w += m;
        }
        return {a1, a2};
    }
    // 先饱和负费用边再求费用流；要求无负环且 _n+2 不越界；会临时加辅助点并返回(流量,费用)。
    pair<int, ll> mcmf2(int _s, int _t, bool mcf = 0, int _n = 0) {
        s = _s, t = _t, n = _n, cmax(mx, _n + 2);
        static int ex[N];
        For(i, 1, _n) ex[i] = 0;
        int a1 = 0; ll a2 = 0;
        For(i, 2, tot) if(i % 2 == 0 && e[i].c < 0) {
            int u = e[i ^ 1].v, v = e[i].v;
            swap(e[i].w, e[i ^ 1].w);
```

```
            ex[u] -= e[i ^ 1].w, ex[v] += e[i ^ 1].w;
            a2 += e[i ^ 1].w * e[i].c;
        }
        For(i, 1, _n) { ex[i] > 0 ? add(_n + 1, i, ex[i], 0) : add(i, _n + 2, -
   ex[i], 0);
        }
        add(_t, _s, FL, 0);
        int aux = tot, n0 = n;
        a2 += mcmf(_n + 1, _n + 2, 0, _n + 2).second;
        a1 += e[aux].w;
        tot = aux - 2 * _n - 2, n = n0;
        For(i, 1, n0) while(hd[i] > tot) hd[i] = e[hd[i]].n;
        auto [a3, a4] = mcmf(_s, _t, mcf, _n);
        return {a1 + a3, a2 + a4};
    }
};
```

3.2 支配树

```
// 有向图以 1 为根；调用方维护 t/t2 输入边，getdom 覆盖可达点 dfn/f1/dm，需递归栈足够。
namespace DOM {

const int N = 1e6 + 10;
int n;
vect<int> t[N], t2[N], q1[N];
int fa[N], f1[N], dfn[N], pos[N], dt, sd[N], dm[N];
pii f[N];
// 加一条有向边；要求点编号为正且不超过后续 _n；同时写入正反邻接表。
void add(int u, int v) {
    t[u] += v, t2[v] += u;
}
// 从根 1 的可达部分做 DFS 编号；要求 dfn 已清零；修改 dfn,pos,f1。
void dfs(int u) {
    dfn[u] = ++dt, pos[dt] = u;
    for(auto v : t[u]) if(!dfn[v]) dfs(v), f1[v] = u;
}
// 并查集路径压缩查询半支配最小值；要求 fa/f 已初始化；返回当前链上的最小 (dfn,点)。
pii que(int u) {
    if(fa[u] == u) return f[u];
    cmin(f[u], que(fa[u]));
    fa[u] = fa[fa[u]];
    return f[u];
}
// 求以 1 为根的支配树；要求 t/t2 已是当前图且点编号 1.._n；dm[v] 为可达点 v 的直接支配点。
void getdom(int _n) {
    n = _n;
    For(i, 1, n) q1[i].clear(), fa[i] = f1[i] = dfn[i] = pos[i] = sd[i] = dm[i] = 0;
    dt = 0;
    dfs(1);
    For(i, 1, n) fa[i] = i, f[i] = {dfn[i], 0};
    rFor(i, dt, 1) {
        int u = pos[i], s = i;
        for(auto x : q1[u]) dm[x] = que(x)[1];
        if(i == 1) break;
        for(auto x : t2[u]) if(dfn[x]) cmin(s, que(x)[0]);
        sd[u] = pos[s], q1[sd[u]] += u, f[u] = {dfn[sd[u]], u};
        for(auto v : t[u]) if(f1[v] == u) fa[v] = u;
    }
    For(i, 2, dt) {
        int u = pos[i];
        dm[u] = sd[dm[u]] == sd[u] ? sd[u] : dm[dm[u]];
    }
}

}  // namespace DOM
```

3.3 一般图最大匹配

```
// n: 点数，t: 无向边；点编号 1..n，匹配状态 mat 为 getmat 内部静态数组。

int n, fa[N];
vect<int> t[N];
// 查询并压缩并查集代表；要求 fa 已在本次搜索初始化；返回 u 所在花的代表。
int fd(int u) { return fa[u] == u ? u : fa[u] = fd(fa[u]); }
// 从未匹配根 rt 尝试一次增广；要求外层按未匹配点反复调用；修改内部静态 mat 和搜索状态。
void getmat(int rt) {
    static int vis[N], mat[N], pr[N];
    For(i, 1, n) vis[i] = pr[i] = 0, fa[i] = i;
    queue<int> q;
    auto lca = [](int x, int y) {
        static int mk[N], tm; ++tm;
        while(1) {
            x = fd(x);
            if(x && exchange(mk[x], tm) == tm) return x;
            x = pr[mat[x]], swap(x, y);
        }
        return -1;
    };
    auto blossom = [&](int x, int y, int z) {
        for(; fd(x) != z; x = pr[y]) {
            pr[x] = y, y = mat[x], fa[x] = fa[y] = z;
            if(vis[y] == 1) vis[y] = 2, q.push(y);
        }
    };
    vis[rt] = 2, q.push(rt);
    while(q.size()) {
        int u = q.front(); q.pop();
        for(auto v : t[u]) {
            if(fd(u) == fd(v)) continue;
            if(!vis[v]) pr[v] = u, vis[v] = 1, vis[mat[v]] = 2, q.push(mat[v]);
            if(!mat[v]) {
                while(v) mat[v] = pr[v], swap(mat[pr[v]], v);
                return;
            }
```

```
            if(vis[v] == 2) {
                int l = lca(u, v);
                blossom(u, v, l), blossom(v, u, l);
            }
        }
    }
}
```

4 数学

4.1 Barret 约简

```
struct barret {
    u128 w; u64 M;
    // 预处理固定模数；要求 2 <= M <= 2^64-1；保存 floor(2^64 / M)。
    barret(const u64 M) : M(M), w(u64((u128(1) << 64) / M)) {}
    // 计算未归一余数；要求构造时 M >= 2；返回 x % M 或 x % M + M。
    inline u64 mod_without_chk(u64 x) const { return x - ((w * x) >> 64) * M; }
    // 计算 x mod M；要求构造时 M >= 2；返回 [0,M) 内余数。
    inline u64 mod(u64 x) const {
        u64 r = mod_without_chk(x);
        return r >= M ? r - M : r;
    }
};
```

4.2 拉格朗日插值

已知 $n+1$ 个点 $(x_0,y_0),(x_1,y_1),...,(x_n,y_n)$，其中 x_i 互不相同，则存在唯一的次数不超过 n 的多项式 $L(x)$ 满足 $L(x_i)=y_i$。

Lagrange 插值多项式为：

$$L(x)=\sum_{i=0}^{n} y_i \prod_{j=0,j\neq i}^{n} \frac{x-x_j}{x_i-x_j}$$

当 x_i 是连续整数（如 $0,1,...,n$）时，可 $O(n)$ 计算 $L(x)$（需预处理阶乘逆元）。此时：

$$L(x)=\sum_{i=0}^{n} y_i * \frac{\prod_{j=0,j\neq i}^{n}(x-j)}{(-1)^{n-i}i!(n-i)!}$$

其中分子可通过前后缀积 $O(1)$ 得到。

```
// 依赖:For/rFor/ll/ksm、常量 M = 998244353;所有点值与系数在 [0, M)。
// lag 的横坐标须两两不同,返回点数+1 项(最高项为 0)。
// lag_i/lag_p 要求 1 <= n < M,并提供返回 int 的 ifac(n)。
// 由任意互异横坐标点值还原多项式；要求点值次数小于点数；返回低到高系数且末项为 0。
vector<int> lag(vector<pair<int, int>> a) {
    int n = a.size();
    vector<int> f(n + 1), ans(n + 1, 0);
    f[0] = 1;
    For(i, 1, n) {
        auto [x, y] = a[i - 1];
        rFor(j, i - 1, 0) f[j + 1] = (f[j + 1] + f[j]) % M, f[j] = 1ll * f[j] * (M - x) % M;
    }
    For(i, 1, n) {
        auto [x, y] = a[i - 1];
        int w = 1, z = 0;
        For(j, 1, n) if(i != j) w = 1ll * w * (x - a[j - 1].first + M) % M;
        w = ksm(w, M - 2, M) * y % M;
        rFor(j, n - 1, 0) z = (f[j + 1] + 1ll * x * z) % M, ans[j] = (ans[j] + 1ll * z * w) % M;
    }
    return ans;
}

// a[1..n] 是 1..n 处的值;原地写回 a[0..n-1] 的系数,a[n] 清零。
void lag_i(int n, int *a) {
    vector<int> f(n + 1, 0), g(n, 0);
    f[0] = 1;
    For(i, 1, n) rFor(j, i - 1, 0) f[j + 1] = (f[j + 1] + f[j]) % M, f[j] = 1ll * f[j] * (M - i) % M;
    For(i, 1, n) {
        int w = 1ll * a[i] * ifac(n - i) % M * ifac(i - 1) % M, s = 0;
        if((i ^ n) & 1) w = (M - w) % M;
        rFor(j, n - 1, 0) s = (f[j + 1] + 1ll * i * s) % M, g[j] = (g[j] + 1ll * s * w) % M;
    }
    For(i, 0, n - 1) a[i] = g[i];
    a[n] = 0;
}
// a[0..n-1] 是 0..n-1 处的值;求 x 处的值,x 在 [0, M)。
int lag_p(int n, int *a, int x) {
    if(x < n) return a[x];
    vector<int> f(n + 1), g(n + 1);
    int ans = 0;
    f[0] = g[n] = 1;
    For(i, 0, n - 1) f[i + 1] = 1ll * f[i] * (x - i + M) % M;
    rFor(i, n - 1, 0) g[i] = 1ll * g[i + 1] * (x - i + M) % M;
    For(i, 0, n - 1) {
        int w = 1ll * f[i] * g[i + 1] % M * a[i] % M * ifac(n - 1 - i) % M * ifac(i) % M;
        if(!((i ^ n) & 1)) w = (M - w) % M;
        ans = (ans + w) % M;
    }
    return ans;
}
```

4.3 类欧

$O(\log n)$ 求 $f=\sum_{i=0}^{n}\lfloor\frac{ai+b}{c}\rfloor$ 及其平方、带权。

- `nd f(int n, int a, int b, int c)`: 参数顺序 *(n, a, b, c)*,一次递归同时得到 `{f, g, h}`:
  - $f=\sum_i\lfloor\frac{ai+b}{c}\rfloor$;$g=\sum_i\lfloor\frac{ai+b}{c}\rfloor^2$;$h=\sum_i i\lfloor\frac{ai+b}{c}\rfloor$
- 参数用普通整数；结果用 `int`，按常量 `M = 998244353` 取模；依赖 `ll/ksm`

- 只要裸下取整和(不取模、无平方带权)，用数论章的 下取整和

```cpp
// 契约:n/a/b >= 0,c > 0;参数保留普通整数,结果按常量 M = 998244353 取模。
// 依赖:ll/ksm;f/g/h 分别为下取整和、平方和、乘 i 的带权和(i = 0..n)。
const int i2 = (M + 1) / 2, i6 = ksm(6, M - 2, M);
struct nd {
    int f, g, h;
};
// 计算 Σ floor((a*i+b)/c) 及平方、乘 i 和；要求 n/a/b >= 0,c > 0；返回三量模 M。
nd f(int n_, int a, int b, int c) {
    if(n_ < 0) return {0, 0, 0};
    ll n = n_ % M, x = (a / c) % M, y = (b / c) % M;
    ll s0 = (n + 1) % M, s1 = n * s0 % M * i2 % M;
    ll s2 = n * s0 % M * ((n + n + 1) % M) % M * i6 % M;
    if(!a) return {int(s0 * y % M), int(s0 * y % M * y % M), int(s1 * y % M)};
    if(a >= c || b >= c) {
        nd rs = f(n_, a % c, b % c, c);
        ll F = rs.f, G = rs.g, H = rs.h;
        ll cross = (x * H + y * F + s1 * x % M * y % M) % M;
        ll g = (G + s2 * x % M * x % M + s0 * y % M * y % M + 2 * cross) % M;
        return {int((F + s1 * x + s0 * y) % M), int(g), int((H + s2 * x + s1 * y) % M)};
    }
    int m = (1ll * a * n_ + b) / c;
    nd rs = f(m - 1, c, c - b - 1, a);
    ll F = rs.f, G = rs.g, H = rs.h, z = m % M, nz = n * z % M;
    ll g = (nz * z - H * 2 - F + 3ll * M) % M;
    ll h = i2 * ((n * nz % M + nz - G - F + 2ll * M) % M) % M;
    return {int((nz - F + M) % M), int(g), int(h)};
}
```

4.4 反射容斥

从 $(0,0)$ 走到 (x,y)，只能往右往上，不触碰 $y-x=k$（$k>0$），往右的连续段数量为 b 的方案数

1. 开头 R，结尾 R

$$N_{RR}=\binom{x-1}{b-1}\binom{y-1}{b-2}-\binom{x+k-2}{b-2}\binom{y-k}{b-1}$$

2. 开头 R，结尾 U

$$N_{RU}=\binom{x-1}{b-1}\binom{y-1}{b-1}-\binom{x+k-2}{b-2}\binom{y-k}{b}$$

3. 开头 U，结尾 R

$$N_{UR}=\binom{x-1}{b-1}\binom{y-1}{b-1}-\binom{x+k-2}{b-1}\binom{y-k}{b-1}$$

4. 开头 U，结尾 U

$$N_{UU}=\binom{x-1}{b-1}\binom{y-1}{b}-\binom{x+k-2}{b-1}\binom{y-k}{b}$$

推导方法

记不限制开头结尾的总方案数为

$$F(x,y,k,b)=\binom{x-1}{b-1}\binom{y+1}{b}-\binom{x+k-1}{b-1}\binom{y-k+1}{b}$$

利用删除首尾的 U 步，可以得到：

$$N_{UU}=F(x,y-2,k-1,b)$$

$$N_{UR}=F(x,y-1,k-1,b)-N_{UU}$$

$$N_{RU}=F(x,y-1,k,b)-N_{UU}$$

$$N_{RR}=F(x,y,k,b)-N_{UU}-N_{UR}-N_{RU}$$

5 数论

5.1 分数还原

`approx(p, q, A)` 返回 (x,a)，满足 $x\equiv qa(\bmod p)$、$|x|\le A$、$a\neq 0$，并使 $|a|$ 最小。

要求 $p>0$、$A\ge 0$；q 会先按 p 取到非负余数。 若要解释成模意义下的分数 $\frac{x}{a}$，还需 a 与 p 互质；通常用素数模数和正的分子界。

```cpp
// approx(p, q, A):返回 (x, a),满足 x ≡ q * a (mod p),|x| <= A,a != 0,且 |a| 最小。
// 要解释成模意义的 x / a,还要求 a 与 p 互质;通常用素数模数和正的分子界。
// 参数顺序与 skip2004 一致,第一个是模数;按给定的分子界 A 寻找最小分母。
// 要求 p > 0,A >= 0;q 先规范到 [0,p)。实现是连分数的辗转相除,O(log p)。
pii approx(int p, int q, int A) {
    q %= p;
    if(q < 0) q += p;
    int x = q, y = p, a = 1, b = 0;
  while(x > A) {
        swap(x, y), swap(a, b);
        a -= x / y * b, x %= y;
    }
  return {x, a};
}
```

5.2 O(V)-O(1) GCD

```cpp
struct fgcd_t {
    static constexpr int n = 1e7, B = 3163;
    array<int, 3> a[n + 1];
    int s[B + 1][B + 1];
    bitset<n + 1> b;
    // 构造 O(n) 预处理表；对象含大数组不宜放栈上；单个对象可重复查询。
    fgcd_t() {
        vect<int> p;
        a[1] = {1, 1, 1};
        For(i, 2, n) {
            if(!b[i]) {
```

```
                if(i <= B) p += i;
                a[i] = {i, 1, 1};
            }
            for(auto z : p) {
                int x = i * z; if(x > n) break;
                b[x] = 1, a[x] = a[i];
                *min_element(all(a[x])) *= z;
                if(i % z == 0) break;
            }
        }
        For(i, 0, B) For(j, 0, i) s[i][j] = !j ? i : s[j][i % j];
    }
    // 查询 gcd；要求 0 <= x,y <= n；返回非负最大公约数,支持 0。
    int operator()(int x, int y) const {
        if(!x || !y) return x | y;
        if(x == 1 || y == 1) return 1;
        auto g1 = [&](int u, int v) {
            if(v == 1) return 1;
            int t = u % v;
            return t == 0 ? v : (v > B ? 1 : s[v][t]);
        };
        int a0 = g1(x, a[y][0]);
        int a1 = g1(x /= a0, a[y][1]);
        int a2 = g1(x /= a1, a[y][2]);
        return a0 * a1 * a2;
    }
};
```

5.3 离散对数

```
// 求最小的 x >= 0 使 a^x ≡ b (mod p),无解返回 -1。
// 要求:gcd(a, p) = 1 且 p 是素数(定阶用 p - 1 的因子分解,求逆用 Fermat)。
// 不满足这两个前提时结果无意义,请先把方程约简(用 gcd 消去公因子)。
// 复杂度 O(sqrt(p))。
ll BSGS(ll a, ll b, ll p) {
    a %= p, b %= p;
    if(p == 1) return 0;
    if(b == 1) return 0;
    if(!a) return -1;
    // l = a 的阶:l | p - 1,从 p - 1 出发按素因子反复试除
    ll l = p - 1;
    for(ll q = 2; q * q <= l; ++q)
        for(; l % q == 0 && ksm(a, l / q, p) == 1; l /= q);
    if(l > 1 && ksm(a, l - 1, p) == 1) l = 1; // 此时阶只能是 1(a ≡ 1)
    // m = ceil(sqrt(l)),保证 m * m >= l;大解落在 (m, m * m) 里
    ll m = (ll) sqrt((db) l);
    while (m * m < l) ++m;
    for(ll y = 0, w = 1 % p; y < std::min(m, l); ++y) { // y < m 的小解
        if(w == b) return y;
        w = (i128) w * a % p;
    }
    unordered_map<ll, ll> pos;
    pos.reserve(m + 2);
    for(ll j = 0, w = 1 % p; j < m; ++j) {
        pos.emplace(w, j);
        w = (i128) w * a % p;
    }
    ll step = ksm(a, m, p), iv = ksm(step, p - 2, p), w = (i128) b * iv % p, ans =
-1;
    for(ll i = 1; i <= m; ++i, w = (i128) w * iv % p)
        if(auto it = pos.find(w); it != pos.end()) {
            ll y = (ll) i * m + it->second;
            if(ksm(a, y, p) == b && (ans < 0 || y < ans)) ans = y; // 显式验证,稳一手
        }
    return ans < 0 ? -1 : ans;
}
```

5.4 中国剩余定理

```
// 合并两个同余方程；返回 [0,lcm(p1,p2)) 内的解，无解返回 -1。
// 要求 0<=a1<p1、0<=a2<p2，且模乘及加法的所有中间值均在 ll 内。
// 统一安全条件为 1<=p1,p2<=2^31；更大模数需另行核验溢出。
ll CRT(ll a1, ll p1, ll a2, ll p2) {
    ll x, y, g = exgcd(p1, p2, x, y);
    if((a2 - a1) % g) return -1;
    ll m1 = p1 / g, m2 = p2 / g, L = m1 * p2; // L = lcm(p1, p2)
    ll k = (a2 - a1) / g % m2 * (x % m2 + m2) % m2;
    ll ans = ((i128) k * p1 + a1) % L; // mod lcm 得到 [0, lcm) 内的解
    return (ans + L) % L;
}
// 多方程:ans = 0, mod = 1;
//   For(i, 1, n) {
//       ans = CRT(ans, mod, a[i], p[i]);
//       if(ans < 0) 无解;
//       mod = mod / gcd(mod, p[i]) * p[i];
//   }
```

5.5 Miller-Rabin 素性检验

```
// 模乘用 unsigned __int128 覆盖完整 u64 乘积；不要换成普通 ksm。
// 模乘与快速幂沿用 lambda，保持 C++17 constexpr 调用兼容。
// 判定 p 是否为素数；u64 全范围确定性判定，支持编译期调用。
constexpr bool chkp(u64 p) {
    if(p < 2) return 0;
    // 返回 (a*b) mod p，覆盖完整 u64；要求 p>0。
    auto mul = [](u64 a, u64 b, u64 p) { return (unsigned __int128) a * b % p; };
    // 求 r*a^x mod p；要求 p>0；x=0 时原样返回 r，不额外取模。
    auto pw = [&](u64 a, u64 x, u64 p, u64 r = 1) {
        for(; x; x >>= 1, a = mul(a, a, p))
            if(x & 1) r = mul(r, a, p);
        return r;
    };
    constexpr u64 small[] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37};
    for(u64 d : small)
```

```
        if(p % d == 0) return p == d;
    constexpr u64 B[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022};
    u64 s = __builtin_ctzll(p - 1), d = (p - 1) >> s;
    for(u64 a : B) {
        // 合数底数的素因子会整除底数，须跳过以免误判 73 等素数。
        if(a % p == 0) continue;
        u64 x = pw(a % p, d, p);
        if(x == 1 || x == p - 1) continue;
        bool ok = 0;
        For(i, 1, s - 1) {
            x = mul(x, x, p);
            if(x == p - 1) { ok = 1; break; }
        }
        if(!ok) return 0;
    }
    return 1;
}
```

5.6 Pollard-Rho 分解质因数

```
// 求整数平方根；要求 x 为 u64；返回最大 r 使 r^2 <= x,乘法用 u128 防溢出。
u64 isqrt(u64 x) {
    if(x < 2) return x;
    u64 r = sqrtl((long double) x);
    for(; (unsigned __int128) r * r > x; --r);
    for(; (unsigned __int128) (r + 1) * (r + 1) <= x; ++r);
    return r;
}
// 返回 x 的一个非平凡因子；要求 x>=2 且 chkp 可用，x 为素数时返回 x。
// 先检查 2、3、5 的因子与完全平方，其余用 Floyd；每 127 步对差积求一次 gcd。
// 差积为 0 等退化批次会重启；随机过程不保证固定运行时间。
u64 rho(u64 x) {
    if(x % 2 == 0) return 2;
    if(x % 3 == 0) return 3;
    if(x % 5 == 0) return 5;
    if(u64 s = isqrt(x); s * s == x) return s;
    if(chkp(x)) return x;
    static mt19937_64 gen(114514);
    for(;;) {
        u64 c = 1 + gen() % (x - 1), t = 0, r = 0, w = 1;
        auto f = [&](u64 v) { return ((unsigned __int128) v * v + c) % x; };
        for(int i = 1;; ++i) {
            t = f(t), r = f(f(r));
            if(t == r) break; // 撞环:换个 c 重来(w 一并重置)
            u64 d = t > r ? t - r : r - t;
            w = (unsigned __int128) w * d % x;
            if(!w) {
                if(u64 g = gcd(d, x); g > 1 && g < x) return g;
                break;
            }
            if(i % 127 == 0) {
                u64 g = gcd(w, x);
                if(g > 1 && g < x) return g;
                if(g == x) break;
            }
        }
    }
}
// 递归分解质因数；要求 x >= 1 且 chkp 可用；把未排序质因子带重数追加到 res。
void fact_rec(u64 x, vect<u64> &res) {
    if(x == 1) return;
    if(chkp(x)) return res += x, void();
    u64 d = rho(x);
    fact_rec(d, res), fact_rec(x / d, res);
}
```

5.7 最小模线性值

```
// 返回 min_{0<=i<n} (a*i+b) mod m；要求 n>=0、m>0；n=0 时返回 m。
// O(log m)；n,m,a 按 int 范围，b 按 ll 范围，a、b 均可负。
int minmod(int n, int m, int a, ll b) {
    int ans = m;
    while(n) {
        a %= m;
        b %= m;
        if(b < 0) b += m;
        cmin(ans, (int) b);
        n = ((ll) (n - 1) * a + b) / m;
        b -= (ll) m * n;
        swap(a, m);
    }
    return ans;
}
```

5.8 二次剩余

```
// 不复用 base header 的 ksm:它按 ll 相乘,不能保证大模数下不溢出。
// 这里自带 u128 模乘与快速幂，与 Miller-Rabin 的内部模乘同理。
namespace QR {
    // 计算 a*b mod p；要求 p > 0；用 u128 支持完整 u64 乘法。
    auto mul = [](u64 a, u64 b, u64 p) { return (u64) ((unsigned __int128) a * b %
p); };
    // 求 r*a^x mod p；要求 p>0；x=0 时原样返回 r，不额外取模。
    auto pw = [](u64 a, u64 x, u64 p, u64 r = 1) {
        for(a %= p; x; x >>= 1, a = mul(a, a, p))
            if(x & 1) r = mul(r, a, p);
        return r;
    };
}
// quadres(a, p): 判断 a 是否为奇素数 p 的二次剩余(要求 0 <= a < p)。
bool quadres(u64 a, u64 p) {
    return !a || QR::pw(a, (p - 1) >> 1, p) == 1;
}
// sqrtp(x, p): 求 x 在模奇素数 p 下的平方根(要求 0 <= x < p)。
```

```cpp
// p 模 4 余 3 时直接算,否则用 Cipolla;x 非二次剩余时返回 -1。
ll sqrtp(ll x, ll p) {
    if(x <= 1) return x;
    if(QR::pw(x, (p - 1) >> 1, p) != 1) return -1;
    if(p % 4 == 3) return QR::pw(x, (p + 1) >> 2, p);
    static mt19937_64 gen;
    ll w, a;
    do a = gen() % p; while (!a || QR::pw(w = (QR::mul(a, a, p) + p - x) % p, (p - 1) >> 1, p) != p - 1);
    auto prod = [&](pll u, pll v) { // F_p[√w] 里的乘法:(u0 + u1√w)(v0 + v1√w)
        ll t0 = (QR::mul(u.first, v.first, p) + QR::mul(QR::mul(u.second, v.second, p), w, p)) % p;
        ll t1 = (QR::mul(u.first, v.second, p) + QR::mul(u.second, v.first, p)) % p;
        return make_pair(t0, t1);
    };
    pll r{1, 0}, b{a, 1};
    for(ll k = (p + 1) >> 1; k; k >>= 1, b = prod(b, b))
        if(k & 1) r = prod(r, b);
    return r.first;
}
```

5.9 原根与阶

```cpp
// 求模素数 p 的最小原根;p = 2 时返回 1。
// g 是原根 <=> 对 p - 1 的每个素因子 q 都有 g^((p - 1) / q) != 1。
// fac_of(x): x 的所有互异素因子,root 与 ord 共用。
vect<int> fac_of(int x) {
    vect<int> fac;
    for(int i = 2; 1ll * i * i <= x; ++i)
        if(x % i == 0) {
            fac += i;
            while (x % i == 0) x /= i;
        }
    if(x > 1) fac += x;
    return fac;
}
// 求模素数 p 的最小原根;要求 p 为素数;无原根时返回 -1。
int root(int p) {
    if(p == 2) return 1;
    auto fac = fac_of(p - 1);
    For(g, 2, p - 1) {
        bool ok = 1;
        for(int q : fac) ok &= ksm(g, (p - 1) / q, p) != 1;
        if(ok) return g;
    }
    return -1;
}
// ord(a, p): a 模素数 p 的阶,即最小的 d > 0 使 a^d ≡ 1 (mod p)。要求 gcd(a, p) = 1。
// 从 p - 1 出发,对 p - 1 的每个素因子 q 反复试除(能除动说明阶里不含 q),
// 除到除不动为止,剩下的就是阶。
int ord(int a, int p) {
    if(p == 2) return 1;
    int x = p - 1;
    auto fac = fac_of(x);
    for(int q : fac)
        while (x % q == 0 && ksm(a, x / q, p) == 1) x /= q;
    return x;
}
```

5.10 积性函数

狄利克雷卷积:

$$(f*g)(n)=\sum_{d|n}f(d)g\left(\frac{n}{d}\right)$$

$$g(1)S_{f(n)}=S_{f*g}(n)-\sum_{i=2}^{n}g(i)S_{f(\lfloor\frac{n}{i}\rfloor)}$$

其中 $S_{h(n)}=\sum_{i=1}^{n}h(i)$ 是前缀和。

5.10.1 杜教筛

```cpp
// 求 S(n) = sum_{i=1}^{n} f(i)。利用 g(1) * S(n) = H(n) - sum_{i=2}^{n} g(i) * S(n / i),
// 其中 H 是 f * g 的前缀和;对 n / i 记忆化后复杂度 O(n^{2/3})。
// 前置:调用方提供阈值 N,并预处理 mu/phi 的前缀和 smu[0..N]、sphi[0..N](下标 0 为 0)。
unordered_map<ll, ll> smu_big, sphi_big;
// 求莫比乌斯函数前缀和;要求已预处理 smu[0..N];返回 sum_{i<=n} mu(i),并记忆化大 n。
ll DJS_mu(ll n) { // f = mu,取 g = 1,则 H(n) = 1
    if(n <= N) return smu[n];
    if(smu_big.count(n)) return smu_big[n];
    ll ans = 1;
    for(ll l = 2, r; l <= n; l = r + 1)
        r = n / (n / l), ans -= (r - l + 1) * DJS_mu(n / l);
    return smu_big[n] = ans;
}
// 求欧拉函数前缀和;要求已预处理 sphi[0..N] 且 n*(n+1)/2 不溢出 ll;返回 sum_{i<=n} phi(i)。
ll DJS_phi(ll n) { // f = phi,取 g = 1,则 H(n) = n * (n + 1) / 2
    if(n <= N) return sphi[n];
    if(sphi_big.count(n)) return sphi_big[n];
    ll ans = n * (n + 1) / 2;
    for(ll l = 2, r; l <= n; l = r + 1)
        r = n / (n / l), ans -= (r - l + 1) * DJS_phi(n / l);
    return sphi_big[n] = ans;
}
// 通用形式:只需实现 f 与 g,其余照抄上面两个函数的结构
//   ll S(ll n) {
//       if(n <= N) return pre[n];
//       if(mp.count(n)) return mp[n];
//       ll ans = H(n); // f * g 的前缀和
//       for(ll l = 2, r; l <= n; l = r + 1)
//           r = n / (n / l), ans -= (g_pre(r) - g_pre(l - 1)) * S(n / l);
//       return mp[n] = ans;
//   }
```

5.10.2 min_25

$O\left(\frac{n^{\frac{3}{4}}}{\log}n\right)$ 求积性函数 f 的前缀和,要求 f 在素数点 $f(p)$ 是关于 p 的低次多项式(最后拆成 p 的幂的线性组合)。

两个递推

设 p_j 是第 j 个素数($p_1=2$),$g_{k(n,j)}$ 是把 $i\le n$ 里所有"i 是素数,或 i 的最小素因子 $>p_j$"的 i^k 累加起来的结果($j=0$ 时就是 $\sum_{i=2}^{n}i^k$)。j 每加一,就把最小素因子恰好是 p_j 的合数减掉(只在 $p_j^2\le n$ 时用,再往后这些合数都超过 n):

$$g_{k(n,j)}=g_{k(n,j-1)}-p_j^k\left(g_{k\left(\left\lfloor\frac{n}{p_j}\right\rfloor,j-1\right)}-\sum_{i=1}^{j-1}p_i^k\right)$$

筛到 $p_j^2>n$ 时 $g_{k(n,j)}$ 就只剩素数:$g_{k(n)}=\sum_{p\le n}p^k$。把 $f(p)=\sum_k c_kp^k$ 的每一段 p^k 各筛一遍再按系数加起来,就得到素数处的 $\sum_{p\le n}f(p)$。真正的 f 前缀和再按最小素因子递归:

$$S(n,m)=\left(\sum_{p\le n}f(p)-\sum_{i=1}^{m}f(p_i)\right)+\sum_{j>m,p_j^2\le n}\sum_{e\ge1,p_j^{e+1}\le n}\left(f\left(p_j^e\right)S\left(\left\lfloor\frac{n}{p_j^e}\right\rfloor,j\right)+f\left(p_j^{e+1}\right)\right)$$

答案 $=S(n,0)+f(1)$;模板里的 `f(n, m)` 就是这个 $S(n,m)$,m 表示"前 m 个素数已经排除"。只有 $O(\sqrt{n})$ 个不同的 $\lfloor\frac{n}{i}\rfloor$,所以状态数是 $O(\sqrt{n})$。

- 接口: `solve(n)`,内部依次 `init()`(线性筛出 10^5 内质数与 `s1/s2/s`: $\sum p_i$、$\sum p_i^2$、$\sum(p_i^2-p_i)$)+ `solveg(n)`(求 g 数组)+ `f(n, m)` 递归求和
- `func(x)`: **素数幂 $x=p^k$ 处 f 的取值**,题变则改(示例 $x(x-1)$ 即 $f(p)=p(p-1)$,欧拉函数类)
- 上限: $\sqrt{n}\le10^5$,即 $n\le10^{10}$;模数就用公共常量 M
- g 数组大小 `2e5`(双值映射 `id(n)`)

```
// 1 <= n <= 1e10;solve 可重复调用,init 只筛一次;func 是素数幂处的函数值。
constexpr int i6 = (M + 1) / 6;
// 模 M 减法;要求 0 <= a,b < M;原地把 a 减去 b。
void sub(int &a, int b) {
    a -= b;
    a < 0 && (a += M);
}
const int N = 1e5 + 10;
int p[N], pc, p2[N];
int s1[N], s2[N], s[N];
// 初始化 1e5 内素数及前缀表;可重复调用,已初始化时不再累加。
void init() {
    if(pc) return;
    static const int n = 1e5;
    static int b[N];
    For(i, 2, n) {
        if(!b[i]) p[++pc] = i;
        For(j, 1, pc) {
            int x = i * p[j];
            if(x > n) break;
            b[x] = 1;
            if(i % p[j] == 0) break;
        }
    }
    p[pc + 1] = n + 1;
    For(i, 1, pc) {
        p2[i] = 1ll * p[i] * p[i] % M;
        s1[i] = (s1[i - 1] + p[i]) % M;
        s2[i] = (s2[i - 1] + p2[i]) % M;
        s[i] = (s2[i] - s1[i] + M) % M;
    }
}
ll lim, n0;
int g1[N * 2], g2[N * 2];
// 映射 n0/i 的取值到数组下标;要求已调用 solveg 设置 n0 与 lim。
int id(ll n) {
    return n <= lim ? n : n0 / n + lim;
}
// 预处理 g 表;要求 1 <= _n0 <= 1e10 且 init 已完成;覆盖当前 n0 的全部除法块。
void solveg(ll _n0) {
    n0 = _n0;
    lim = sqrt(n0);
    bool fl = n0 / lim == lim;
    auto put = [&](ll k, int slot) {
        k %= M;
        sub(g1[slot] = k * (k + 1) / 2 % M, 1);
        sub(g2[slot] = k * (k + 1) % M * (k + k + 1) % M * i6 % M, 1);
    };
    auto doit = [&](int m, int i1, int i2) {
        sub(g1[i1], 1ll * (g1[i2] - s1[m - 1] + M) * p[m] % M);
        sub(g2[i1], 1ll * (g2[i2] - s2[m - 1] + M) * p2[m] % M);
    };
    For(i, 1, lim) put(i, i);
    For(i, 1, lim - fl) put(n0 / i, i + lim);
    For(i, 1, pc) {
        ll x = p[i], y = x * x, z = n0 / x;
        For(j, 1, min(lim - fl, n0 / y)) doit(i, j + lim, id(z / j));
        rFor(j, lim, min(lim + 1, y)) doit(i, j, j / x);
    }
}
// 查询筛后素数贡献前缀;要求 solveg(n0) 后且 n 是 n0 的除法块取值;返回模 M 结果。
int g(ll n) { return (g2[id(n)] - g1[id(n)] + M) % M; }
// 计算素数幂处函数值;要求 n >= 1;返回 n*(n-1) mod M。
ll func(ll n) {
    n %= M;
    return n * (n - 1) % M;
}
// 递归计算 Min_25 剩余贡献;要求 solveg(n) 已建表且 p[m] 以下素数已剔除;返回模 M 结果。
int f(ll n, int m) {
    if(p[m] > n) return 0;
    ll ans = g(n) - s[m] + M;
    For(j, m + 1, pc) {
        int q = p[j];
        if(1ll * q * q > n) break;
```


14

```
        for(ll w = q; w * q <= n; w *= q) ans += func(w) * f(n / w, j) % M + func(w * q);
    }
    return ans % M;
}
// 求 1 + sum_{2<=i<=n} F(i)；要求 1 <= n <= 1e10；可重复调用并返回模 M 结果。
ll solve(ll n) {
    init();
    solveg(n);
    return (f(n, 0) + 1) % M;
}
```

5.11 下取整和

`floor_sum(n, m, a, b)` ：$\sum_{i=0}^{n-1} \left\lfloor \frac{ai+b}{m} \right\rfloor$。

```
// 求 sum_{0<=i<n} floor((a*i+b)/m)；要求 m > 0 且 a*n+b 能放入 u64；返回值须不超过 u64 上界。
u64 floor_sum(u64 n, u64 m, u64 a, u64 b) {
    unsigned __int128 ans = 0;
    for(;;) {
        if(a >= m) {
            ans += (unsigned __int128)n * (n - 1) / 2 * (a / m);
            a %= m;
        }
        if(b >= m) {
            ans += (unsigned __int128)n * (b / m);
            b %= m;
        }
        u64 y = (i128) a * n + b;
        if(y < m) break;
        n = y / m;
        b = y % m;
        swap(m, a);
    }
    return (u64)ans;
}
```

5.12 高斯整数

$N(a+bi) = a^2 + b^2$ 叫 $a+bi$ 的模长平方,它可乘:$N(xy) = N(x)N(y)$。下面说的“拆”,都是要让 N 落回目标 n。

素数分类

素数分为三种：

- 2 是特例：$2 = (1+i)(1-i)$，拆出来的两个只差 ± 1、$\pm i$；
- $4k+1$ 型的 p 能拆：$p = (x+yi)(x-yi)$，也就是 $p = x^2 + y^2$；
- $4k+3$ 型的 p 拆不动，自己就是高斯素数。

这三类就是全部的高斯素数。每个非零高斯整数都能写成它们的乘积，而且写法唯一，只差整体的 ± 1、$\pm i$。

判定 $x^2 + y^2 = n$

n 的每个 $4k+3$ 型素因子指数都是偶数时才能写成 $x^2 + y^2$，否则不能。直接分解即可。

计数 $x^2 + y^2 = n$

记 $\chi_4(d)$ 在 d 模 4 余 1、余 3、为偶数时分别是 1、-1、0，则 $\chi_4(d)$ 是完全积性函数 $\Rightarrow$ $\sum_{d|n} \chi_4(d)$ 是积性函数。

记 d_1 是 n 的模 4 余 1 的因子个数，d_3 是模 4 余 3 的因子个数，则 $d_1 - d_3 = \sum_{d|n} \chi_4(d)$，且也是解个数。

- x、y 取任意整数（包括负数和交换顺序）：解数是 $4(d_1 - d_3)$。
- 只要 $0 \le x \le y$ 的那些：解数是 $\frac{d_1 - d_3 + s}{2}$，其中 s 数的是 n、$\frac{n}{2}$ 里完全平方的个数（0、1 或 2）。

如 3 无解，$2 = 1^2 + 1^2$，$65 = 8^2 + 1^2 = 7^2 + 4^2$。

Cornacchia

把 $4k+1$ 型的 p 拆成 $p = x^2 + y^2$：

1. 从 $g = 2, 3, 5, \ldots$ 试出模 p 的非二次剩余，算 $r = g^{\frac{p-1}{4}} \bmod p$，此时 $r^2 \equiv -1 \bmod p$。
2. 对 (p, r) 做辗转相除，除到余数 $\le \sqrt{p}$ 为止。
3. 停下的那个余数就是 x；拿 $p - x^2$ 开方得 y，验一下 $x^2 + y^2 = p$。

它跟求 $\gcd(p, r+i)$ 是一回事，但全程只用整数，不会溢出。

高斯整数 gcd

跟整数一样辗转相除，只是“除”要除到最近的整数：$\frac{x}{y}$ 先算成 $x \cdot \frac{\bar{y}}{|y|^2}$，再把实部虚部各自四舍五入。 可以证明每一步余数都满足 $N(r) \le \frac{N(y)}{2}$，所以只要 $O(\log N)$ 步；结果唯一到只差 ± 1、$\pm i$。中间乘积会到 10^{36} 量级，代码里要用 128 位整数。

全部解

分解质因数，然后：

- 2^k 换成 $(1+i)^k$（它的模长平方就是 2^k）；
- $4k+1$ 型的 p^e：先用 Cornacchia 拆出 p 的那对 $(x+yi)$、$(x-yi)$，再换成 $(x+yi)^e$ 或 $(x-yi)^e$，两种都试；
- $4k+3$ 型的 p^e（e 必是偶数）换成实数 $p^{\frac{e}{2}}$。

把这些乘起来得到高斯整数 $A+Bi$，它的模长平方正好是 n，所以 $(|A|, |B|)$ 就是一组解。 每个 $4k+1$ 型的因子有两种选法，把所有选法 DFS 组合一遍（$4k+1$ 型的因子没几个）， 每组的 $(|A|, |B|)$ 统一成 $0 \le x \le y$，排序去重就是全部解。

```
// 高斯整数上的辗转相除：每步余数的模长平方 <= |y|^2 / 2，所以 O(log N) 步。
// 商取最近的整数点；返回归一后的 gcd(乘 ±1、±i 使实部 > 0、虚部 >= 0)，这样写法唯一；gcd(0,0) = 0。
// 坐标绝对值 <= 1e18(中间量 <= 2e36，离 i128 上限 1.7e38 还有两个数量级)。
// 拆 4k+1 型素数 p：取 r = g^((p-1)/4) 是模 p 的 -1 的平方根，gcd(p, r + i) 的模长平方就是 p。

struct gi { ll a, b; };
// 对 p/d 四舍五入到最近整数；要求 d > 0；负数按远离 0 的半数处理。
```

```cpp
i128 rd(i128 p, i128 d) { return (p >= 0 ? p + d / 2 : p - d / 2) / d; }
// 求高斯整数 gcd；要求坐标绝对值 <= 1e18；返回按单位元归一后的 gcd。
gi gi_gcd(gi x, gi y) {
    while(y.a || y.b) {
        i128 d = (i128) y.a * y.a + (i128) y.b * y.b;                  // |y|^2
        i128 qa = rd((i128) x.a * y.a + (i128) x.b * y.b, d);          // q = round(x / y)
        i128 qb = rd((i128) x.b * y.a - (i128) x.a * y.b, d);
        gi r = {(ll) (x.a - qa * y.a + qb * y.b), (ll) (x.b - qa * y.b - qb * y.a)};
        x = y, y = r;
    }
    if(x.a < 0 || (x.a == 0 && x.b < 0)) x = {-x.a, -x.b};             // 归一：实部 > 0、虚部 >= 0
    if(x.b < 0) x = {-x.b, x.a}; else if(x.a == 0) x = {x.b, 0};
    return x;
}
```

6 多项式

6.1 任意模数 NTT

```cpp
// 契约:① 变换长度 l = 2^ceil(lg(n + m - 1)) 必须 <= 2^20(三模里 M1 = 1004535809 = 479·2^21+1 最小);
//       超限结果错,但 P 恰为 998244353 / 469762049 时 CRT 退化成单模、反而看着对,别拿它们试上限;
//       ② 系数请先归一到 [0, P):传负数(`{-1}`)结果不保证(非负但 >= P 的未归一系数是安全的)。

namespace Mul {
using poly = vector<int>;
// 原地做模 M 的 NTT；要求 a 长度为 2 的幂且符合长度上限，k=1 正变换、k=-1 逆变换；结果覆盖 a。
template <int M, int G> static void NTT(poly &a, const int k) {
    auto ksm = [&](ll a, int b) {ll ans=1;for(;b;b>>=1,a=a*a%M)if(b&1)ans=ans*a%M;return ans; };
    static poly r, w;
    int n = a.size();
    if(n != r.size()) {
        int l = __lg(n);
        r.resize(n);
        For(i, 0, n - 1) r[i] = (r[i >> 1] >> 1) | ((i & 1) << (l - 1));
    }
    For(i, 0, n - 1) if(i < r[i]) swap(a[r[i]], a[i]);
    ll wi = ksm(G, ((M - 1) >> 1) / n * k + M - 1);
    w.resize(n), w[0] = 1;
    For(i, 1, n - 1) w[i] = w[i - 1] * wi % M;
    for(int m = 1, l = __lg(n); m < n; m <<= 1, --l)
        for(int i = 0; i < n; i += m + m)
            For(j, 0, m - 1) {
                ll x = a[i + j], y = 1ll * w[j << l] * a[i + j + m] % M;
                a[i + j] = x + y >= M ? x + y - M : x + y;
                a[i + j + m] = x - y < 0 ? x - y + M : x - y;
            }
    if(k == -1) {
        ll inv = ksm(n, M - 2);
        For(i, 0, n - 1) a[i] = a[i] * inv % M;
    }
}
// 单一模数下做卷积；要求 a、b 已补零到变换长度 l（2 的幂）；返回长度 l 的卷积结果。
template <int M, int G> static poly conv(poly a, poly b, int l) {
    NTT<M, G>(a, 1), NTT<M, G>(b, 1);
    For(i, 0, l - 1) a[i] = 1ll * a[i] * b[i] % M;
    NTT<M, G>(a, -1);
    return a;
}
// 任意模数卷积；要求系数非负 int、P ∈ [2, 2^31-1] 且满足 CRT 精度界；返回长度 n+m-1，空输入返回空。
poly mul(poly a, poly b, int P) {
    int n = a.size(), m = b.size();
    if(!n || !m) return poly();
    if(n == 1 && m == 1) return poly{int(1ll * a[0] * b[0] % P)};
    int l = 1 << (__lg(n + m - 2) + 1);
    a.resize(l), b.resize(l);
    static constexpr int M[3]{998244353, 1004535809, 469762049}, G = 3;
    poly c[3]{conv<M[0], G>(a, b, l), conv<M[1], G>(a, b, l), conv<M[2], G>(a, b, l)}, d;
    d.reserve(n + m - 1);
    static constexpr ll m01 = 1ll * M[0] * M[1];
    static constexpr int i0 = 669690699, i1 = 354521948;
    For(id, 0, n + m - 2) {
        auto A = c[0][id], B = c[1][id], C = c[2][id];
        ll x = 1ll * (B - A + M[1]) % M[1] * i0 % M[1] * M[0] + A;
        d.push_back((1ll * (C - x % M[2] + M[2]) % M[2] * i1 % M[2] * (m01 % P) % P + x) % P);
    }
    return d;
}
}  // namespace Mul
// 普通卷积使用 base header 的常量 M;任意模数用 Mul::mul(a, b, P)。
using poly = vector<int>;
// 模 M=998244353 的普通卷积；要求系数非负（负数请先归一）；返回长度 n+m-1，空输入返回空。
poly mul(poly a, poly b) { return Mul::mul(move(a), move(b), M); }
```

6.2 多项式复合

模 998244353,求 $F(G(x)) \bmod x^n$,一次调用解出。

- 接口与 `prep(22)` 前置见代码头注释;要求 $[x^0]G = 0$,`F`、`G` 不足 n 项补 0
- 内部 `solve(F, G, n, m)`: 分治卷积,先乘 $A \cdot B$ 再做奇偶抽取
- 复杂度: 多项式级(小常数大),适合 $n \le 10^5$ 量级

```cpp
// 系数低次在前且在 [0, mod)；使用前须 prep(22)，覆盖内部 NTT 长度。
// 两份复合模板独立使用；静态缓冲约 168 MiB，不含临时向量。

typedef vector<int> vi;
typedef vector<vi> vv;
typedef unsigned long long ull;
```

```cpp
const int N = 4.4e6 + 15, mod = 998244353;
int X[N], Y[N];
int rev[N * 2], omg[N * 2];
// 预处理 NTT 的 rev/omg 表；要求 n >= 0（按 2^n 长度上界）；用 NTT 前必须先调用一次。
void prep(int n)
{
  int i, j, w;
  for(i = 0; i <= n; i++)
  {
    for(j = 1; j < (1 << i); j++)
      rev[(1 << i) + j] = rev[(1 << (i - 1)) + (j >> 1)] | ((j & 1) << (i - 1));
    w = ksm(3, (mod - 1) >> i, mod);
    omg[1 << i] = 1;
    for(j = 1; j < (1 << i); j++)
      omg[(1 << i) + j] = (ll)omg[(1 << i) + j - 1] * w % mod;
  }
}
ull tmp[N * 2];
// 原地做长度 2^n 的 NTT；要求 A 有 >= 2^n 个元素、已调用 prep(>= n)；t=false 正变换、t=true 逆变换。
inline void ntt(int *A, int n, bool t)
{
  int i, j, k, q, x;
  for(i = 0; i < (1 << n); i++)
    tmp[i] = A[i];
  for(i = 0; i < (1 << n); i++)
    if(rev[(1 << n) + i] > i)
      swap(tmp[rev[(1 << n) + i]], tmp[i]);
  for(i = 0; i < n; i++)
  {
    for(j = 0; j < (1 << n); j += (2 << i))
      for(k = 0; k < (1 << i); k++)
      {
        x = tmp[j + k + (1 << i)] * omg[(2 << i) + k] % mod;
        tmp[j + k + (1 << i)] = tmp[j + k] + mod - x;
        tmp[j + k] = tmp[j + k] + x;
      }
    if(i == 17 || i == n - 1)
      for(j = 0; j < (1 << n); j++)
        tmp[j] %= mod;
  }
  for(i = 0; i < (1 << n); i++)
    A[i] = tmp[i];
  if(t)
  {
    for(i = 1; i < (1 << n); i++)
      if(i < (1 << n) - i)
        swap(A[i], A[(1 << n) - i]);
    q = ksm(1 << n, mod - 2, mod);
    for(i = 0; i < (1 << n); i++)
      A[i] = (ll)A[i] * q % mod;
  }
}
struct poly
{
  vector<int> c;
  // 默认构造：空多项式。
  poly() = default;
  // 由系数向量构造；d 为低次在前的系数。
  poly(vector<int> d) : c(move(d)) {}
  // 模 mod 卷积；任一为空返回空；返回长度 |f|+|g|-1 的乘积。
  friend poly operator * (poly f, poly g)
  {
    if(f.c.empty() || g.c.empty())
      return poly();
    poly h;
    int i, l = 0;
    while((f.c.size() + g.c.size()) >> l)
      l++;
    memset(X, 0, (1 << l) * sizeof(int));
    memset(Y, 0, (1 << l) * sizeof(int));
    for(i = 0; i < f.c.size(); i++)
      X[i] = f.c[i];
    for(i = 0; i < g.c.size(); i++)
      Y[i] = g.c[i];
    ntt(X, l, 0);
    ntt(Y, l, 0);
    for(i = 0; i < (1 << l); i++)
      X[i] = (ll)X[i] * Y[i] % mod;
    ntt(X, l, 1);
    h.c.resize(f.c.size() + g.c.size() - 1);
    for(i = 0; i < h.c.size(); i++)
      h.c[i] = X[i];
    return h;
  }
};
// 复合的递归分治核心，提取双变量级数的 [x^[0,n)) y^(-m,0] 系数；要求 G 的 x 常数项为 1；返回分治合并结果。
vv solve(vi F, vv G, int n, int m)
{
  int i, j, S, T, a, b, d = 2 * n - 1;
  vv ans;
  if(n == 1)
  {
    F.resize(m);
    ans.resize(m);
    for(i = 0; i < m; i++)
      ans[i] = {F[i]};
    return ans;
  }
  poly A, B, C;
  vv H;
  S = m * d + n;
  A.c.resize(S);
  B.c.resize(S);
  for(i = 0; i <= m; i++)
```

```
    for(j = 0; j < n; j++)
    {
      A.c[i * d + j] = G[i][j];
      B.c[i * d + j] = (j & 1 ? mod - G[i][j] : G[i][j]);
    }
  A = A * B;
  a = (n + 1) / 2;
  b = 2 * m;
  H.resize(b + 1);
  for(i = 0; i <= b; i++)
  {
    H[i].resize(a);
    for(j = 0; j < a; j++)
      H[i][j] = A.c[i * d + 2 * j];
  }
  auto P = solve(F, H, a, b);
  T = (b - 1) * d + n;
  C.c.resize(T);
  for(i = 0; i < b; i++)
    for(j = 0; j < a; j++)
      C.c[i * d + 2 * j] = P[b - 1 - i][j];
  C = C * B;
  ans.resize(m);
  for(i = 0; i < m; i++)
  {
    ans[i].resize(n);
    for(j = 0; j < n; j++)
      ans[i][j] = C.c[(b - 1 - i) * d + j];
  }
  return ans;
}
// 求 F(G(x)) mod x^n；要求 n>=1、G[0]=0（实现忽略 G[0]）。
// F、G 按值传入，低阶不足按零补齐；返回恰好 n 项，不改调用方输入。
// 须先 prep(k) 使预处理覆盖内部 NTT 长度。
poly comp(poly F, poly G, int n)
{
  int i;
  vv Gp(2, vi(n)); // 2×n:第一行 1,第二行 -G
  Gp[0][0] = 1;
  if((int) G.c.size() < n) G.c.resize(n);
  if((int) F.c.size() < n) F.c.resize(n);
  for(i = 1; i < n; i++)
    Gp[1][i] = mod - G.c[i];
  auto P = solve(F.c, Gp, n, 1);
  return poly(P[0]);
}
```

6.3 多项式复合逆

模 998244353,给定 F,求 G 使 $F(G(x)) = x \pmod{x^n}$。

- `comp_inv(F, n)`: 返回 G 的 n 个系数,要求 $[x^0]F = 0$ 且 $[x^1]F \neq 0$(首项可逆,代码里直接算 `qpow(F[1], mod-2)`)
- `F` 的系数个数**必须** >= **n**(内部会读 `F[1]`,且 `n = 1` 时也要求至少给 2 个),否则越界读
- `|F| > n` 时**高次项不会被截断**,会让结果静默算错 —— 调用方自己补/截到 n
- 调用前必须先执行一次 **`prep(22)`**(见代码头注释;除单位根外还预处理好阶乘/逆元表 `fac/inv/Inv`)
- 算法基于 Lagrange 反演 $[x^n]F^{-k}$,内部 `pw_pj` 做形式幂级数逆 + 幂迭代

```
// 接口:comp_inv(F, n) 求 G 使 F(G(x)) ≡ x (mod x^n),返回 n 个系数;要求 F[1] != 0。
// 使用前必须先调用一次 prep(22)(初始化 NTT 的 rev/omg;长度上界按 2^22 准备),
// 否则内部 NTT 用的是未初始化的单位根 —— 原来这一步写在模板自带的 main 里。

typedef vector<int> vi;
typedef vector<vi> vv;
typedef long long ll;
typedef unsigned long long ull;
const int N = 4.4e6 + 15, mod = 998244353;
int F[N], X[N], fac[N], inv[N], Inv[N];
int rev[N * 2], omg[N * 2];
// 快速幂；要求 y >= 0；返回 x^y mod mod 在 [0, mod) 的值。
int qpow(int x, int y)
{
  int ans = 1;
  while(y)
  {
    if(y & 1)
      ans = (ll)ans * x % mod;
    x = (ll)x * x % mod;
    y >>= 1;
  }
  return ans;
}
// 预处理 NTT 的 rev/omg 与阶乘/逆元表；要求 n >= 0（按 2^n 长度上界）；用 NTT 前必须先调用一次。
void prep(int n)
{
  int i, j, w, m = N - 5;
  for(i = 0; i <= n; i++)
  {
    for(j = 1; j < (1 << i); j++)
      rev[(1 << i) + j] = rev[(1 << (i - 1)) + (j >> 1)] | ((j & 1) << (i - 1));
    w = qpow(3, (mod - 1) >> i);
    omg[1 << i] = 1;
    for(j = 1; j < (1 << i); j++)
      omg[(1 << i) + j] = (ll)omg[(1 << i) + j - 1] * w % mod;
  }
  fac[0] = 1;
  for(i = 1; i <= m; i++)
    fac[i] = (ll)fac[i - 1] * i % mod;
  inv[m] = qpow(fac[m], mod - 2);
  for(i = m; i >= 1; i--)
    inv[i - 1] = (ll)inv[i] * i % mod;
  for(i = 1; i <= m; i++)
    Inv[i] = (ll)inv[i] * fac[i - 1] % mod;
}
```


18

```
ull tmp[N * 2];
// 原地做长度 2^n 的 NTT；要求 A 有 >= 2^n 个元素、已调用 prep(>= n)；t=false 正变换、t=true 逆变换。
inline void ntt(int *A, int n, bool t)
{
  int i, j, k, q, x;
  for(i = 0; i < (1 << n); i++)
    tmp[i] = A[i];
  for(i = 0; i < (1 << n); i++)
    if(rev[(1 << n) + i] > i)
      swap(tmp[rev[(1 << n) + i]], tmp[i]);
  for(i = 0; i < n; i++)
  {
    for(j = 0; j < (1 << n); j += (2 << i))
      for(k = 0; k < (1 << i); k++)
      {
        x = tmp[j + k + (1 << i)] * omg[(2 << i) + k] % mod;
        tmp[j + k + (1 << i)] = tmp[j + k] + mod - x;
        tmp[j + k] = tmp[j + k] + x;
      }
    if(i == 17 || i == n - 1)
      for(j = 0; j < (1 << n); j++)
        tmp[j] %= mod;
  }
  for(i = 0; i < (1 << n); i++)
    A[i] = tmp[i];
  if(t)
  {
    for(i = 1; i < (1 << n); i++)
      if(i < (1 << n) - i)
        swap(A[i], A[(1 << n) - i]);
    q = qpow(1 << n, mod - 2);
    for(i = 0; i < (1 << n); i++)
      A[i] = (ll)A[i] * q % mod;
  }
}
// 把 F 补到 2^n 长后做 NTT；要求 F.size() <= 2^n、已调用 prep(>= n)；t=false 正变换、t=true 逆变换；返回变换后的 F。
vi pv(vi F, int n, bool t)
{
  int i;
  F.resize(1 << n);
  for(i = 0; i < (1 << n); i++)
    X[i] = F[i];
  ntt(X, n, t);
  for(i = 0; i < (1 << n); i++)
    F[i] = X[i];
  return F;
}
struct poly
{
  vector<int> c;
  // 默认构造：空多项式。
  poly() = default;
  // 由系数向量构造；d 为低次在前的系数。
  poly(vector<int> d) : c(move(d)) {}
  // 模 mod 卷积；任一为空返回空；返回长度 |f|+|g|-1 的乘积。
  friend poly operator * (poly f, poly g)
  {
    if(f.c.empty() || g.c.empty())
      return poly();
    poly h;
    int i, l = 0, len = f.c.size() + g.c.size() - 1;
    while((f.c.size() + g.c.size()) >> l)
      l++;
    f.c = pv(f.c, l, 0);
    g.c = pv(g.c, l, 0);
    for(i = 0; i < (1 << l); i++)
      X[i] = (ll)f.c[i] * g.c[i] % mod;
    ntt(X, l, 1);
    h.c.resize(len);
    for(i = 0; i < h.c.size(); i++)
      h.c[i] = X[i];
    return h;
  }
};
// 复合逆的 Lagrange 反演辅助（提取 [x^{n-1}] 1/(1-yF) 的 y 系数）；要求 F.size() >= n；返回长度 n 的系数。
poly pw_pj(poly F, int n)
{
  int i, j, S, R, m = 1, d = n - 1, l, r;
  vv A, B;
  vi pA, pB, pC;
  A.resize(2);
  A[0].resize(n);
  A[0][0] = 1;
  A[1].resize(n);
  B = A;
  for(i = 0; i < n; i++)
    B[1][i] = mod - F.c[i];
  while(d)
  {
    R = 2 * d + 1;
    S = m * R + d + 1;
    l = 0;
    while((S - 1) >> l)
      l++;
    l++;
    pA = pB = pC = {};
    pA.resize(S);
    pB.resize(S);
    pC.resize(S);
    for(i = 0; i <= m; i++)
      for(j = 0; j <= d; j++)
      {
        pA[i * R + j] = A[i][j];
```

```
        pB[i * R + j] = B[i][j];
        pC[i * R + j] = (j & 1 ? mod - B[i][j] : B[i][j]);
      }
    pA = pv(pA, l, 0);
    pB = pv(pB, l, 0);
    pC = pv(pC, l, 0);
    for(i = 0; i < (1 << l); i++)
    {
      pA[i] = (ll)pA[i] * pC[i] % mod;
      pB[i] = (ll)pB[i] * pC[i] % mod;
    }
    pA = pv(pA, l, 1);
    pB = pv(pB, l, 1);
    r = d & 1;
    d /= 2;
    m *= 2;
    A.resize(m + 1);
    B.resize(m + 1);
    for(i = 0; i <= m; i++)
    {
      A[i].resize(d + 1);
      B[i].resize(d + 1);
      for(j = 0; j <= d; j++)
      {
        A[i][j] = pA[i * R + j * 2 + r];
        B[i][j] = pB[i * R + j * 2];
      }
    }
  }
  F.c.resize(n); // 必须截到 n:否则调用方传进来的更长多项式会把高次项留在后面,
                 // 接着 comp_inv 里的 reverse 会把它们倒到低次位置参与运算 → 静默算错
  for(i = 0; i < n; i++)
    F.c[i] = A[i][0];
  return F;
}
// 求复合逆 G，使 F(G(x)) ≡ x mod x^n；要求 n>=1、F[0]=0、F[1]!=0。
// 按值补齐不足系数，不改调用方输入；返回恰好 n 项，须先 prep 覆盖内部 NTT 长度。
poly comp_inv(poly F, int n)
{
  // 补齐系数:下面要读 F.c[1](n == 1 时也得有),pw_pj 也要求 size >= n
  if((int) F.c.size() < std::max(n, 2)) F.c.resize(std::max(n, 2));
  int i, r, p = qpow(F.c[1], mod - 2), c;
  for(i = 0, c = 1; i < n; i++, c = (ll)c * p % mod)
    F.c[i] = (ll)F.c[i] * c % mod;
  poly G;
  F = pw_pj(F, n);
  reverse(F.c.begin(), F.c.end());
  for(i = 0; i < n; i++)
    F.c[i] = (ll)F.c[i] * Inv[n - 1 - i] % mod * (n - 1) % mod;
  r = mod - Inv[n - 1];
  G.c = {1};
  while(r)
  {
    if(r & 1)
    {
      G = G * F;
      G.c.resize(n);
    }
    F = F * F;
    F.c.resize(n);
    r >>= 1;
  }
  G.c.resize(n);
  for(i = n - 1; i > 0; i--)
    G.c[i] = (ll)G.c[i - 1] * p % mod;
  G.c[0] = 0;
  return G;
}
```

6.4 最短线性递推

```
// 契约:a 是 M = 998244353 下的序列,各项在 [0, M),长度任意(含空)。返回最短递推系数 c:
//         a[i] = Σ_{j=1..|c|} c[j-1] * a[i-j]  对一切 i >= |c| 成立;
//       全零序列 / 空序列返回空向量(零阶递推)。
//       算法只保证"给定 a 的最短递推",用 c 外推更远的项需要 |a| >= 2|c|(标准结论)。
//       退化输入(如 {0,0,…,0,x},最短阶数恰好等于 |a|)下系数本来就不唯一(没有约束方程),
//       本实现按同一公式给出其中一组,不保证是"最自然"的那组。
// 复杂度:O(n²)。依赖:base header 的 For/ll/ksm/常量 M。

using poly = vector<int>;
// 求给定前缀的最短线性递推；要求系数在 [0,M)；返回按 a[i]=Σ c[j-1]a[i-j] 使用的系数。
poly BM(const poly &a) {
    int n = a.size();
    if(!n) return poly();
    int len = 0, off = 0;        // len: 当前递推阶数;off: 距上次更新阶数的项数
    int s = 1;                   // s: 上次更新时的偏差
    poly res(n + 1), lst(n + 1);  // 开 n+1:len 最多到 n,末尾要用 res[len]
    res[0] = 1;
    For(i, 0, n - 1) {
        int d = 0;                          // 偏差 d = Σ res[j]*a[i-j](j <= len <= i)
        For(j, 0, len) d = (d + 1ll * res[j] * a[i - j]) % M;
        ++off;
        if(!d) continue;
        poly tmp = res;
        int k = 1ll * d * ksm(s, M - 2, M) % M;                // res ← res - (d/s) * lst * x^off
        For(j, off, n) res[j] = (res[j] - 1ll * k * lst[j - off] % M + M) % M;
        if(len * 2 > i) continue;
        len = i - len + 1, lst = tmp, s = d, off = 0;
    }
    poly c(len);
    For(i, 0, len - 1) c[i] = (M - res[i + 1]) % M;   // res[0]=1 且 d=0 ⇒ a[i] = -Σ_{j>=1} res[j]a[i-j]
    return c;
```

```
}
```

6.5 线性递推第 n 项

```
// 求 [x^n] P(x) / Q(x)。要求 n >= 0,Q[0] != 0;P 可为空,也可比 Q 长。
// k 阶递推:Q = 1 - Σ c_i x^i,P = (Q * Σ_{i<k} a_i x^i) mod x^k。
// 复杂度 O(k log k log(n+2)),k 为输入长度上界。依赖:mul、ksm、M = 998244353;系数在 [0, M)。

using poly = vector<int>;
// 求有理生成函数第 n 项;要求 n >= 0 且 Q[0] != 0;返回 [x^n]P/Q。
int bostan_mori(poly P, poly Q, ll n) {
    if(P.empty()) return 0;
    if(Q.size() == 1) return n < (ll)P.size() ? 1ll * P[n] * ksm(Q[0], M - 2, M) % M : 0;
    while(n) {
        // 只求第 n 项,高于 n 次的分子/分母项可安全丢弃。
        if(n < (ll) P.size()) P.resize(n + 1);
        if(n < (ll) Q.size()) Q.resize(n + 1);
        poly Qm = Q;
        For(i, 1, (int) Qm.size() - 1) if(i & 1) Qm[i] = (M - Qm[i]) % M; // Q(-x)
        poly U = mul(P, Qm), V = mul(Q, Qm);
        P.clear();
        for(int i = n & 1; i < (int) U.size(); i += 2) P.push_back(U[i]); // 取偶数/奇数项
        Q.clear();
        for(int i = 0; i < (int) V.size(); i += 2) Q.push_back(V[i]);
        n >>= 1;
    }
    return 1ll * P[0] * ksm(Q[0], M - 2, M) % M;
}
```

6.6 复数 FFT 卷积

```
// 求两个实系数多项式的卷积。返回的向量长度是 n + m - 1。
// 前置:结果系数绝对值要明显小于 1e15(否则 double 精度不够,请用本目录 ntt.cpp 的模意义卷积)。
// 复杂度 O((n + m) log(n + m))。

// cp:复数类型,用于 FFT 的变换域。
using cp = complex<db>;
// 原地做长度为 n 的 FFT(k=1)/逆 FFT(k=-1,含除以 n);要求 n 为 2 的幂;空或长度 1 直接返回。
void fft(vect<cp> &a, int k) {
    int n = a.size();
    if(n <= 1) return;
    static vect<int> r;
    if((int) r.size() != n) {
        int l = __lg(n);
        r.resize(n);
        For(i, 0, n - 1) r[i] = (r[i >> 1] >> 1) | ((i & 1) << (l - 1));
    }
    For(i, 0, n - 1) if(i < r[i]) swap(a[i], a[r[i]]);
    for(int m = 1; m < n; m <<= 1) {
        cp wm = polar((db) 1, acos((db) -1) / m * k);
        for(int i = 0; i < n; i += m + m) {
            cp w = 1;
            For(j, 0, m - 1) {
                cp x = a[i + j], y = a[i + j + m] * w;
                a[i + j] = x + y, a[i + j + m] = x - y, w *= wm;
            }
        }
    }
    if(k == -1) For(i, 0, n - 1) a[i] /= n;
}
// 求两个实系数多项式的卷积;任一为空返回空;返回长度 n+m-1 的结果(精度约束见头注释)。
vect<db> fft_conv(const vect<db> &a, const vect<db> &b) {
    int n = a.size(), m = b.size();
    if(!n || !m) return {};
    int l = 1;
    while (l < n + m - 1) l <<= 1;
    vect<cp> fa(l), fb(l);
    For(i, 0, n - 1) fa[i] = a[i];
    For(i, 0, m - 1) fb[i] = b[i];
    fft(fa, 1), fft(fb, 1);
    For(i, 0, l - 1) fa[i] *= fb[i];
    fft(fa, -1);
    vect<db> res(n + m - 1);
    For(i, 0, n + m - 2) res[i] = fa[i].real();
    return res;
}
```

6.7 多点求值

```
// 契约:f 任意(可为空向量,视作零多项式),xs 是求值点(可为空、可重复)。
//       返回长度 = |xs| 的向量,第 i 项是 f(xs[i])。空 xs 返回空向量。
// 做法:分治建出每段的乘积多项式 Π(x - xs[i]);从根往下令 f ← f mod 该段乘积
//       (余式度数 < 段长),走到叶子时余式是常数,即该点的值。
// 复杂度:O((|f| + m) log²(|f| + m)),m = |xs|;额外空间 O(m log m)(线段树上的多项式)。
// 依赖:poly_mod、poly_inv、mul(ntt.cpp)、常量 M = 998244353;系数与点在 [0, M)。
// 注意:xs 可重复(叶子 (x - xs[l]) 重根也无所谓)。

using poly = vector<int>;
// 分治建线段树,节点存该段 Π(x - xs[i]);要求 t 已分配 4m、l..r 为当前段;返回时 t[k] 为该段乘积。
static void _eval_build(poly *t, int k, int l, int r, const poly &xs) {
    if(l == r) return void(t[k] = poly{(M - xs[l]) % M, 1});  // x - xs[l]
    int mid = (l + r) >> 1;
    _eval_build(t, k + k, l, mid, xs), _eval_build(t, k + k + 1, mid + 1, r, xs);
    t[k] = mul(t[k + k], t[k + k + 1]);
}
// 从根向下对 f 取模,叶子处余式常数即该点值;要求 t 已建好、f 为原多项式;结果写入 res。
static void _eval_solve(const poly *t, int k, int l, int r, const poly &f, poly &res) {
    poly g = poly_mod(f, t[k]);
    if(l == r) return void(res[l] = g.empty() ? 0 : g[0]);
    int mid = (l + r) >> 1;
```

```
    _eval_solve(t, k + k, l, mid, g, res), _eval_solve(t, k + k + 1, mid + 1, r, g, res);
}
// 求 f 在 xs 各点的值；xs 可为空或含重复点；返回长度 |xs| 的向量，空 xs 返回空。
poly poly_eval(const poly &f, const poly &xs) {
    int m = xs.size();
    if(!m) return poly();
    vector<poly> t(4 * m);
    _eval_build(t.data(), 1, 0, m - 1, xs);
    poly res(m);
    _eval_solve(t.data(), 1, 0, m - 1, f, res);
    return res;
}
```

6.8 多项式指数

```
// 契约:f 非空且 f[0] == 0(否则 exp f 不是形式幂级数:先提出 e^{f[0]} 这个常数因子);n >= 1。
//       返回 exp f (mod x^n),长度恰为 n,常数项恒为 1。
// 复杂度:O(n log n),常数较大 —— 每轮牛顿都重算一次 ln g(内部含一次完整求逆)。
// 依赖:poly_ln、poly_inv、mul(ntt.cpp)、常量 M = 998244353;系数在 [0, M),n < M。
// 每轮:g ← g*(1 - ln g + f) mod x^m,精度翻倍。

using poly = vector<int>;
// 求 exp f (mod x^n)；要求 f 非空且 f[0]==0、n >= 1；返回长度 n、常数项为 1 的向量。
poly poly_exp(const poly &f, int n) {
    poly g{1};
    for(int m = 1; m < n; m <<= 1) {
        int t = min(m << 1, n);
        poly c = poly_ln(g, t);  // ln g(mod x^t),g 只有 m 项、高次按 0 参与运算
        For(i, 0, t - 1) c[i] = ((i < (int) f.size() ? f[i] : 0) - c[i] + M) % M;
        c[0] = (c[0] + 1) % M;               // 1 - ln g + f
        g = mul(g, c);
        g.resize(t);
    }
    g.resize(n);
    return g;
}
```

6.9 多项式对数

```
// 契约:f 非空且 f[0] == 1(ln 的定义需要:常数项恒为 1 的形式幂级数才有可定义的对数;
//       f[0] != 1 时先提出常数因子,ln(c*h) = ln c + ln h);n >= 1。
//       返回 ln f = ∫ f'/f (mod x^n),长度恰为 n,常数项恒为 0。
// 复杂度:O(n log n)。
// 依赖:poly_inv、mul(ntt.cpp)、常量 M = 998244353;系数在 [0, M),n < M。

using poly = vector<int>;
// 返回 1..n 模逆元表（静态缓存）；要求 n >= 0；索引 i 处为 i 在模 M 下的逆元。
static const poly &_ln_inv_num(int n) { // 1..n 的模逆元表
    static poly iv{0, 1};
    if((int) iv.size() <= n) {
        int m = iv.size();
        iv.resize(n + 1);
        For(i, m, n) iv[i] = M - 1ll * (M / i) * iv[M % i] % M;
    }
    return iv;
}
// 求 ln f = ∫f'/f (mod x^n)；要求 f 非空且 f[0]==1；n<=0 返回空，否则返回长度 n、常数项为 0 的向量。
poly poly_ln(const poly &f, int n) {
    if(n <= 0) return poly();
    poly d(n - 1);
    For(i, 1, min((int) f.size(), n) - 1) d[i - 1] = 1ll * f[i] * i % M;  // f'
    d = mul(d, poly_inv(f, n));                                           // f'/f
    d.resize(n - 1);
    const poly &iv = _ln_inv_num(n);
    poly g(n);
    For(i, 1, n - 1) g[i] = 1ll * d[i - 1] * iv[i] % M;                   // 积分:系数除以 i
    return g;
}
```

6.10 多项式开根

```
// 契约:f 非空且 f[0] == 1(最常用情形;一般 f[0] 是二次剩余时先提出 sqrt(f[0]),
//       常量开方用「二次剩余」模板);n >= 1。
//       返回 g 满足 g² ≡ f (mod x^n),长度恰为 n 且 g[0] = 1(这个解唯一)。
// 复杂度:O(n log n)。
// 依赖:poly_inv、mul(ntt.cpp)、常量 M = 998244353;系数在 [0, M)。
// 每轮:g ← (g + f/g)/2 mod x^m,精度翻倍。

using poly = vector<int>;
// 求 g 使 g² ≡ f (mod x^n)；要求 f 非空且 f[0]==1、n >= 1；返回长度 n、g[0]=1 的唯一解。
poly poly_sqrt(const poly &f, int n) {
    const int ih = (M + 1) / 2;
    poly g{1};
    for(int m = 1; m < n; m <<= 1) {
        int t = min(m << 1, n);
        poly c(f.begin(), f.begin() + min((int) f.size(), t));  // f mod x^t
        poly d = mul(c, poly_inv(g, t));                        // f/g
        d.resize(t);
        g.resize(t);
        For(i, 0, t - 1) g[i] = (g[i] + 1ll * d[i]) * ih % M;             // (g + f/g)/2
    }
    g.resize(n);
    return g;
}
```

6.11 多项式求逆

```
// 契约:f 非空且 f[0] != 0(否则 mod x 下不可逆);n >= 1。
//       返回长度恰为 n 的 g,满足 f*g ≡ 1 (mod x^n);f 短于 n 时高次按 0 补齐。
// 复杂度:O(n log n)。每轮牛顿把精度翻倍:g ← g*(2 - f*g) mod x^m。
```

```
// 依赖:base header 的 For/ll/ksm/常量 M = 998244353,以及 ntt.cpp 的 mul。
//       系数均须在 [0, M),所有运算按 M 取模。

using poly = vector<int>;
// 求 g 使 f*g ≡ 1 (mod x^n);要求 f 非空且 f[0]!=0、n >= 1;返回长度 n 的逆。
poly poly_inv(const poly &f, int n) {
    poly g{int(ksm(f[0], M - 2, M))};
    for(int m = 1; m < n; m <<= 1) {
        int t = min(m << 1, n);
        poly c(f.begin(), f.begin() + min((int) f.size(), t));  // f mod x^t
        poly d = mul(g, c);
        d.resize(t);
        For(i, 0, t - 1) d[i] = ((i == 0 ? 2 : 0) - d[i] + M) % M;  // 2 - f*g
        g = mul(g, d);
        g.resize(t);
    }
    g.resize(n);
    return g;
}
```

6.12 带余除法

```
// 契约:f 任意(可为空向量),g 非空且首项系数 g.back() != 0(反转后它是常数项,要可逆;
//       g 尾部有 0 请调用方先自己 trim)。返回 {q, r} 满足 f = q*g + r 且 deg r < deg g:
//         |f| >= |g| 时 q 长度恰为 |f|-|g|+1,r 长度恰为 |g|-1
//           (高次可能是 0,不自动 trim;|g| = 1 时 r 为空),
//         |f| <  |g| 时 q 为空,r 就是传入的 f 原样(长度 |f|,不补到 |g|-1)。
// 做法:反转两者,商 rev(q) = rev(f) * inv(rev(g), |f|-|g|+1),再整回来算余式。
// 复杂度:O(n log n)。
// 依赖:poly_inv、mul(ntt.cpp)、常量 M = 998244353;系数在 [0, M)。

using poly = vector<int>;
// 带余除法;要求 g 非空且 g.back()!=0 (f 可为空);返回 {q, r} 满足 f=q*g+r 且 deg r < deg g。
pair<poly, poly> poly_divmod(const poly &f, const poly &g) {
    int n = f.size(), m = g.size();
    if(n < m) return {poly(), f};
    int k = n - m + 1;
    poly rf(f.rbegin(), f.rend()), rg(g.rbegin(), g.rend());
    poly q = mul(rf, poly_inv(rg, k));
    q.resize(k);
    reverse(q.begin(), q.end());
    poly r = mul(q, g);
    r.resize(n);
    For(i, 0, n - 1) r[i] = (f[i] - r[i] + M) % M;
    r.resize(m - 1);
    return {q, r};
}
// 求 f mod g;要求同 poly_divmod;返回带余除法的余式。
poly poly_mod(const poly &f, const poly &g) { return poly_divmod(f, g).second; }
```

6.13 快速插值

```
// 契约:xs 与 ys 等长(可空),要求 xs 两两不同(否则 M'(xs[i]) = 0 无法除)。
//       返回唯一的 deg < m 的多项式 f 满足 f(xs[i]) = ys[i],长度恰为 m(m = |xs|)。
// 做法:记 M = Π(x - xs[i]),则 f = Σ_i ys[i]/M'(xs[i]) · M(x)/(x - xs[i]);
//       M'(xs[i]) 用一点多点求值,后面用分治卷积合并。
// 复杂度:O(m log² m)。依赖:poly_eval、mul、常量 M = 998244353;所有输入在 [0, M)。
// 注意:点数重复时 M'(x_i) = 0 会除零,调用前请保证互异。

using poly = vector<int>;
// 分治建线段树,节点存该段 Π(x - xs[i]);要求 t 已分配 4m;返回时 t[k] 为该段乘积。
static void _interp_build(poly *t, int k, int l, int r, const poly &xs) {
    if(l == r) return void(t[k] = poly{(M - xs[l]) % M, 1});  // x - xs[l]
    int mid = (l + r) >> 1;
    _interp_build(t, k + k, l, mid, xs), _interp_build(t, k + k + 1, mid + 1, r, xs);
    t[k] = mul(t[k + k], t[k + k + 1]);
}
// 分治合并插值:P_node = P_left·t[right] + P_right·t[left];要求 t 已建好;返回该段的插值多项式。
static poly _interp_rec(const poly *t, int k, int l, int r, const poly &w) {
    if(l == r) return poly{w[l]};
    int mid = (l + r) >> 1;
    poly a = mul(_interp_rec(t, k + k, l, mid, w), t[k + k + 1]);
    poly b = mul(_interp_rec(t, k + k + 1, mid + 1, r, w), t[k + k]);
    a.resize(max(a.size(), b.size()), 0);
    For(i, 0, (int) b.size() - 1) a[i] = (a[i] + b[i]) % M;
    return a;
}
// 求过 (xs[i],ys[i]) 的唯一 deg < m 多项式;要求 xs、ys 等长且 xs 互异;返回长度 m 的系数,空输入返回空。
poly poly_interp(const poly &xs, const poly &ys) {
    int m = xs.size();
    if(!m) return poly();
    vector<poly> t(4 * m);
    _interp_build(t.data(), 1, 0, m - 1, xs);
    poly dM(m);
    For(i, 1, m) dM[i - 1] = 1ll * t[1][i] * i % M;      // M' 的系数
    poly ev = poly_eval(dM, xs), w(m);
    For(i, 0, m - 1) w[i] = 1ll * ys[i] * ksm(ev[i], M - 2, M) % M;    // M'(xs[i]) 互异时非 0
    poly res = _interp_rec(t.data(), 1, 0, m - 1, w);
    res.resize(m);
    return res;
}
```

7 计算几何

7.1 基础几何库

namespace Geo,eps=1e-10,双精度;p2 为点,seg 为线段。

- 基本: sign/cmp,det(叉积)、点乘 *、r90/rot/unit、dis/dis2

- 线段: `chkss`(相交含端点)/`chkss_s`(严格)、`isll`(直线交)、`ons`/`ons_s`(点在线上)、`proj`/`reflect`/`nearest`/`disss`(投影/翻转/点线距/线线距)
- 多边形: `area`(面积,有符号)、`contain`(点形关系: 0 外/1 线上/2 内)、`convex_hull`(凸包,默认去共线点,`nos=1` 保留)、`convex_diameter`(凸包直径)、`convex_cut`(半平面裁剪)
- 圆: `circumcenter`(外心)、`circumcircle_diameter`(外接圆直径,三点共线返 -1)

调用 convex_hull 的前置条件(容易踩)

- 输出缓冲 `b` **至少要能放 `2n + 2` 个点**,不是 `n` 个 —— 扫下凸壳时会先写满 n 个位置,之后上凸壳还会再写。
- 它**会原地排序输入数组 `a`**(契约如此),`b` 不能与 `a` 是同一块内存;需要保留原顺序就自己先拷一份。
- `n <= 1` 时直接返回 1 但不往 `b` 里写点,调用方得自己处理。

数值与退化

- eps 是**绝对**容差:几何尺度小于 10^{-5} 时叉积一律归零,小尺度问题请自行放大坐标或改 eps。
- `ons`、`nearest`、`disss` 支持零长线段(按端点处理);`proj`/`reflect` 的直线仍须非零长,否则会除以零。
- `area` 带符号(按顶点序,逆时针为正);要面积取 `abs`。

```cpp
// eps = 1e-10 是**绝对**容差:sign/cmp 与所有位置/相交判定都走它;坐标量级很大(>1e6)时请自行放宽。
// p2 兼作点与向量;seg{x, y} 是有向线段(方向 x -> y),也可当直线用。
namespace Geo {

const db eps = 1e-10;

struct p2;
struct seg;
// 符号(带 eps 容差):x < -eps 为 -1,x > eps 为 1,否则 0。
int sign(db x) { return x < -eps ? -1 : x > eps; }
// x 与 y 的容差比较:返回 x-y 的符号。
int cmp(db x, db y) { return sign(x - y); }
const db pi = acos(db(-1));
struct p2 {
    db x, y;
    // 叉积 x×b(带号面积 ×2)。
    db det(const p2 &b) const { return x * b.y - y * b.x; }
    // 极角,取值 (-pi, pi]。
    db alpha() const { return atan2(y, x); }
    // 按 eps 容差判等。
    bool operator==(const p2 &b) const {
        return !cmp(x, b.x) && !cmp(y, b.y);
    }
    // 字典序(x 优先),用于排序/二分。
    bool operator<(const p2 &b) const {
        int c = cmp(x, b.x);
        if(c) return c < 0;
        return cmp(y, b.y) < 0;
    }
};
// 逆时针旋转 90 度。
p2 r90(p2 x) { return {-x.y, x.x}; }
// 绕原点逆时针旋转 a 弧度。
p2 rot(p2 x, db a) { return {x.x * cos(a) - x.y * sin(a), x.x * sin(a) + x.y * cos(a)}; }
// 向量相加。
p2 operator+(p2 x, p2 y) { return {x.x + y.x, x.y + y.y}; }
// 向量相减。
p2 operator-(p2 x, p2 y) { return {x.x - y.x, x.y - y.y}; }
// 向量除以标量;要求 y 非零。
p2 operator/(p2 x, db y) { return {x.x / y, x.y / y}; }
// 数乘(标量在右)。
p2 operator*(p2 x, db y) { return {x.x * y, x.y * y}; }
// 数乘(标量在左)。
p2 operator*(db y, p2 x) { return {x.x * y, x.y * y}; }
// 点积。
db operator*(p2 x, p2 y) { return x.x * y.x + x.y * y.y; }
// 向量自加,返回新值。
p2 operator+=(p2 &x, p2 y) { return x = x + y; }
// 向量自减,返回新值。
p2 operator-=(p2 &x, p2 y) { return x = x - y; }
// 向量自除,返回新值;要求 y 非零。
p2 operator/=(p2 &x, db y) { return x = x / y; }
// 模长平方。
db dis2(p2 x) { return x.x * x.x + x.y * x.y; }
// 模长。
db dis(p2 x) { return hypot(x.x, x.y); }
// 单位化;要求 x 非零(零向量得 inf/nan)。
p2 unit(p2 x) { return x / dis(x); }
struct seg {
    p2 x, y;
    // 方向向量 y - x。
    p2 dir() const { return y - x; }
};
// 有向直线 p 与点 q 的叉积(q 在左 > 0)。
db cross(const seg &p, const p2 &q) { return p.dir().det(q - p.x); }
// 上述叉积的符号。
int crossop(const seg &p, const p2 &q) { return sign(cross(p, q)); }
// 点 b 是否在有向直线 l 上。
bool inc(seg l, const p2 &b) { return !crossop(l, b); }
// 两直线是否重合(方向可反)。
bool eqll(const seg &a, const seg &b) {
    return inc(a, b.x) && inc(a, b.y);
}
// 两直线是否不平行(有唯一交点)。
bool chkll(const seg &a, const seg &b) {
    db a1 = cross(b, a.x), a2 = -cross(b, a.y);
    return !!sign(a1 + a2);
}
// 两直线交点;要求不平行(见 chkll)。
p2 isll(const seg &a, const seg &b) {
    db a1 = cross(b, a.x), a2 = -cross(b, a.y);
    return (a.x * a2 + a.y * a1) / (a1 + a2);
}
// 两线段是否相交(含端点相接、共线重叠)。
```

```
bool chkss(const seg &p, const seg &q) {
    auto is = [](db l1, db r1, db l2, db r2) {
        if(l1 > r1) swap(l1, r1);
        if(l2 > r2) swap(l2, r2);
        return !(cmp(r1, l2) == -1 || cmp(r2, l1) == -1);
    };
    p2 p1 = p.x, p2 = p.y, q1 = q.x, q2 = q.y;
    return is(p1.x, p2.x, q1.x, q2.x) && is(p1.y, p2.y, q1.y, q2.y) &&
           crossop(p, q1) * crossop(p, q2) <= 0 && crossop(q, p1) * crossop(q, p2) <= 0;
}
// 两线段是否严格相交(交点严格在两段内部,端点/共线不算)。
bool chkss_s(const seg &p, const seg &q) {
    p2 p1 = p.x, p2 = p.y, q1 = q.x, q2 = q.y;
    return crossop(p, q1) * crossop(p, q2) < 0 && crossop(q, p1) * crossop(q, p2) < 0;
}
// m 是否落在 a、b 闭区间内(eps 容差)。
bool isMid(const db &a, const db &m, const db &b) { return cmp(a, m) * cmp(b, m) <= 0; }
// 逐分量判定 m 是否落在 a、b 之间。
bool isMid(const p2 &a, const p2 &m, const p2 &b) { return isMid(a.x, m.x, b.x) && isMid(a.y, m.y, b.y); }
// 点 q 是否在线段 p 上(含端点)。
bool ons(const seg &p, const p2 &q) { return !crossop(p, q) && isMid(p.x, q, p.y); }
// 点 q 是否严格在线段 p 内部(不含端点)。
bool ons_s(const seg &p, const p2 &q) {
    return !crossop(p, q) && sign((q - p.x) * (p.x - p.y)) * sign((q - p.y) * (p.x - p.y)) < 0;
}
// 点 q 在直线 p 上的投影;要求直线非零长。
p2 proj(const seg &p, const p2 &q) {
    p2 dir = p.y - p.x;
    return p.x + dir * ((dir * (q - p.x)) / dis2(dir));
}
// 点 q 关于直线 p 的对称点;要求直线非零长。
p2 reflect(const seg &p, const p2 &q) {
    return proj(p, q) * 2 - q;
}
// 点 q 到线段 p 的最近距离平方;线段可退化为点。
db nearest(const seg &p, const p2 &q) {
    if(dis2(p.dir()) == 0) return dis2(q - p.x);
    p2 h = proj(p, q);
    if(isMid(p.x, h, p.y)) return dis2(q - h);
    return min(dis2(p.x - q), dis2(p.y - q));
}
// 两线段间最近距离(平方);相交(含端点/共线)返回 0,支持退化线段。
db disss(const seg &p, const seg &q) {
    if(chkss(p, q)) return 0;
    return min({nearest(p, q.x), nearest(p, q.y), nearest(q, p.x), nearest(q, p.y)});
}
// 多边形带号面积(顶点顺序任意,顺/逆时针符号相反)。
db area(int n, p2 *a) {
    db ret = 0;
    For(i, 0, n - 1) ret += a[i].det(a[(i + 1) % n]);
    return ret / 2;
}
// 点 p 与多边形 a 的关系(射线法/奇偶规则):0 严格在外,1 在边界,2 严格在内;支持凹多边形。
int contain(int n, p2 *a, const p2 &p) {
    int ret = 0;
    For(i, 0, n - 1) {
        p2 u = a[i], v = a[(i + 1) % n];
        if(ons({u, v}, p)) return 1;
        if(cmp(u.y, v.y) <= 0) swap(u, v);
        if(cmp(p.y, u.y) > 0 || cmp(p.y, v.y) <= 0) continue;
        ret ^= crossop({p, u}, v) > 0;
    }
    return ret * 2;
}
// 求凸包,返回点数,k 个顶点按逆时针存在 b[0..k-1]。nos=0 去掉共线点,nos=1 保留。
// **前置条件:输出缓冲 b 至少要能放 2n + 2 个点**(不是 n 个!扫下凸壳时会先写满 n 个位置,
// 之后上凸壳还会再写)。
// 输入 a 会被原地排序;b 与 a 不能是同一块内存。
int convex_hull(int n, p2 *a, p2 *b, bool nos = 0) {
    if(n <= 1) return 1;
    sort(a, a + n);
    int k = 0;
    For(i, 0, n - 1) {
        while(k > 1 && crossop({b[k - 2], b[k - 1]}, a[i]) <= -nos) --k;
        b[k++] = a[i];
    }
    int t = k;
    rFor(i, n - 2, 0) {
        while(k > t && crossop({b[k - 2], b[k - 1]}, a[i]) <= -nos) --k;
        if(i) b[k++] = a[i];
    }
    return k;
}
// 凸多边形直径(最远点对)的距离平方;要求 a 逆时针、n >= 3(旋转卡壳)。
db convex_diameter(int n, p2 *a) {
    if(n <= 1) return 0;
    int is = 0, js = 0;
    For(k, 1, n - 1) {
        if(a[k] < a[is]) is = k;
        if(a[js] < a[k]) js = k;
    }
    int i = is, j = js;
    db ret = dis2(a[i] - a[j]);
    do {
        if((a[(i + 1) % n] - a[i]).det(a[(j + 1) % n] - a[j]) >= 0)
            (++j) %= n;
        else
            (++i) %= n;
        ret = max(ret, dis2(a[i] - a[j]));
    } while(i != is || j != js);
```

```
    return ret;
}
// 用直线 q 裁凸多边形,保留 q 左侧(cross(q, ·) >= 0);返回裁后多边形。
vector<p2> convex_cut(int n, p2 *a, const seg &q) {
    vector<p2> qs;
    For(i, 0, n - 1) {
        p2 u = a[i], v = a[(i + 1) % n];
        int d1 = crossop(q, u), d2 = crossop(q, v);
        if(d1 >= 0) qs.push_back(u);
        if(d1 * d2 < 0) qs.push_back(isll({u, v}, q));
    }
    return qs;
}
// 三点外接圆直径;三点共线返回 -1。
db circumcircle_diameter(p2 x, p2 y, p2 z) {
    if(!crossop({x, y}, z)) return -1;
    return dis(x - y) * dis(y - z) * dis(z - x) / abs((y - x).det(z - x));
}
// 三点外接圆圆心;要求三点不共线(否则除零)。
array<db, 2> circumcenter(p2 x, p2 y, p2 z) {
    auto [x1, y1] = x;
    auto [x2, y2] = y;
    auto [x3, y3] = z;
    auto A = dis2(x), B = dis2(y), C = dis2(z);
    auto u1 = x1 - x2, u2 = x1 - x3, v1 = y1 - y2, v2 = y1 - y3;
    return {db((C - A) * v1 - (B - A) * v2) / (2 * u1 * v2 - 2 * u2 * v1), db((C -
A) * u1 - (B - A) * u2) / (2 * v1 * u2 - 2 * v2 * u1)};
}

}  // namespace Geo
using namespace Geo;
```

7.2 Delaunay 三角剖分

```
// 依赖:geo.cpp 的 p2 / eps / sign / dis2 / convex_hull(用法是先 include geo.cpp 再
include 本文件)。
// 参数:a 是输入点集。允许含坐标完全相同的重复点(会被去重),允许全部共线,允许 n <= 2。
// 返回:拼平的三角形 —— 每连续 3 个点是同一个三角形的三个顶点,顶点按逆时针给出
//   (带号面积为正),坐标取自去重后的输入点(与输入逐位相同);一个非退化三角形都没有时
//   返回空 vect。调用后 Delaunay::pts 是去重后(按坐标升序)的点集,三角形顶点是它的下标。
// 前置条件:坐标在 db 精度内(long double 下 1e9 量级没问题)。不要求一般位置:三点共线、
//   四点共圆、点正好落在已有边(含凸包边)上都能处理 —— 共圆四边形的两条对角线都合法,
//   本实现按「严格在外接圆内才翻边」的判据任选其一(相对容差 1e-12)。
// 做法:先算一遍凸包,用凸包上的点做扇形剖分,再反复翻边把它变成 Delaunay;然后把内部点
//   按随机顺序逐个插入(定位用「从上一个三角形游走」,插入后继续翻边)。凸包边永不翻
//   (相当于把外面的无限远点当成一个虚顶点),所以不需要超级三角形,也就没有
//   「超级三角形顶点把凸包边的位置占了」这类退化问题。
// 复杂度:凸包 O(n log n);插入期望接近线性,最坏 O(n^2);翻边总数在最坏情况下 O(n^2),
//   三角形个数 <= 2n - 5,内存 O(n)。随机点实测 n=1e4 约 0.06s、n=1e5 约 2.6s(1e9 坐标);
//   插入顺序由固定种子的 mt19937 打乱,结果可复现。
namespace Delaunay {

vect<p2> pts;  // 去重后的点,按 (x, y) 精确升序
struct Tri { int v[3], f[3], ver; };  // v 逆时针;f[i] 是顶点 i 对边的邻接三角形(-1 表示
凸包边)
vector<Tri> tri;
vect<char> live;
vect<int> fre;
int verc;

// 分配一个新三角形(优先复用空闲槽),返回其下标。
int nw() {
    if(!fre.empty()) {
        int t = fre.back();
        fre.pop_back(), live[t] = 1;
        return t;
    }
    tri.push_back(Tri()), live += 1;
    return (int) tri.size() - 1;
}
// 释放三角形 t(标记死亡并回收下标)。
void rm(int t) { live[t] = 0, fre += t; }
// 按 (x, y) 精确字典序比较。
bool lt(p2 u, p2 v) { return u.x != v.x ? u.x < v.x : u.y < v.y; }
// 三点叉积符号(精确,不用 eps)。
int ori(int a, int b, int c) { return sign((pts[b] - pts[a]).det(pts[c] -
pts[a])); }
// 按相对容差判断 p 是否在外接圆内;要求 a,b,c 按逆时针排列。
int ill(int p, int a, int b, int c) {
    p2 A = pts[a] - pts[p], B = pts[b] - pts[p], C = pts[c] - pts[p];
    db t1 = dis2(A) * B.det(C), t2 = dis2(B) * C.det(A), t3 = dis2(C) * A.det(B);
    return t1 + t2 + t3 > 1e-12L * (abs(t1) + abs(t2) + abs(t3));
}
// 返回三角形 t 中有向边 (a,b) 的编号 [0,2],不存在返回 -1。
int finde(int t, int a, int b) {
    for(int i = 0; i < 3; ++i) if(tri[t].v[(i + 1) % 3] == a && tri[t].v[(i + 2) %
3] == b) return i;
    return -1;
}
// 将 t 的边 i 与三角形 g 双向相连;须共享反向边,0 <= i < 3。
// g=-1 时仅将该边标为凸包边。
void lnk(int t, int i, int g) {
    tri[t].f[i] = g;
    if(g >= 0) tri[g].f[finde(g, tri[t].v[(i + 2) % 3], tri[t].v[(i + 1) % 3])] = t;
}
// 创建逆时针三角形并返回编号;要求 a,b,c 非共线,必要时交换 b,c。
int mk(int a, int b, int c) {
    if(ori(a, b, c) < 0) swap(b, c);
    int t = nw();
    tri[t].v[0] = a, tri[t].v[1] = b, tri[t].v[2] = c;
    tri[t].f[0] = tri[t].f[1] = tri[t].f[2] = -1, tri[t].ver = ++verc;
    return t;
}
typedef array<int, 3> Tst;  // 待检查的边:(三角形, 该边对面顶点在三角形里的下标, 三角形版本
号)
```

```
// 消耗待检边栈并翻转非法边，更新邻接；凸包边不翻，过期版本跳过。
void flipall(vector<Tst> &st) {
    while(!st.empty()) {
        Tst e = st.back();
        st.pop_back();
        int t = e[0], i = e[1];
        if(!live[t] || tri[t].ver != e[2]) continue;  // 这条边早已被别的翻边改掉了
        int a = tri[t].v[(i + 1) % 3], b = tri[t].v[(i + 2) % 3], g = tri[t].f[i];
        if(g < 0) continue;  // 凸包边:合法
        int k = finde(g, b, a), q = tri[g].v[k];
        if(!ill(q, tri[t].v[0], tri[t].v[1], tri[t].v[2])) continue;  // 合法(含共圆):不翻
        int p = tri[t].v[i];
        int X = tri[t].f[(i + 1) % 3], Y = tri[t].f[(i + 2) % 3];
        int W = tri[g].f[(k + 1) % 3], Z = tri[g].f[(k + 2) % 3];
        tri[t].v[0] = p, tri[t].v[1] = a, tri[t].v[2] = q;  // (p,a,b)+(b,a,q) -> (p,a,q)+(p,q,b)
        tri[g].v[0] = p, tri[g].v[1] = q, tri[g].v[2] = b;
        lnk(t, 0, W);
        lnk(t, 2, Y);
        lnk(t, 1, g);
        lnk(g, 0, Z);
        lnk(g, 1, X);
        tri[t].ver = ++verc, tri[g].ver = ++verc;
        st.push_back({t, 0, tri[t].ver}), st.push_back({t, 2, tri[t].ver});
        st.push_back({g, 0, tri[g].ver}), st.push_back({g, 1, tri[g].ver});
    }
}

// 计算 Delaunay 三角剖分;返回拼平的逆时针三角形,顶点为去重后点集的下标。
vect<p2> delaunay(vect<p2> a) {
    pts.clear();
    tri.clear();
    live.clear();
    fre.clear();
    verc = 0;
    sort(a.begin(), a.end(), lt);
    for(int i = 0; i < (int) a.size(); ++i)
        if(pts.empty() || pts.back().x != a[i].x || pts.back().y != a[i].y) pts += a[i];
    int n = (int) pts.size();
    if(n < 3) return {};
    // 注意 convex_hull 会往 b 里写下凸壳 + 上凸壳,缓冲必须比 n 大(上界 2n-2),这里一律给 2n+2。
    vect<p2> q(pts.begin(), pts.end()), hb(2 * n + 2);
    int k = convex_hull(n, q.data(), hb.data());  // 凸包(逆时针,去掉共线点)
    if(k < 3) return {};                          // 全部共线:没有任何非退化三角形
    vect<int> hv(k), fs(k - 2);
    for(int j = 0; j < k; ++j) hv[j] = (int) (lower_bound(pts.begin(), pts.end(), hb[j], lt) - pts.begin());
    vector<Tst> st;
    For(i, 1, k - 2) fs[i - 1] = mk(hv[0], hv[i], hv[i + 1]);  // 先做扇形剖分
    For(i, 1, k - 3) lnk(fs[i - 1], 1, fs[i]);
    for(int t = 0; t < (int) tri.size(); ++t)
        for(int i = 0; i < 3; ++i) st.push_back({t, i, tri[t].ver});  // 把它翻成 Delaunay
    flipall(st);
    vect<int> in;  // 内部点(不在凸包上的点;可能正好落在凸包边上)
    vect<char> onh(n, 0);
    for(int j = 0; j < k; ++j) onh[hv[j]] = 1;
    for(int i = 0; i < n; ++i) if(!onh[i]) in += i;
    mt19937 rng(20240915);
    shuffle(in.begin(), in.end(), rng);
    int last = fs[0];
    for(int ii = 0; ii < (int) in.size(); ++ii) {
        int p = in[ii], cur = last, go;
        for(;;) {  // 游走定位:p 一定在凸包内(或凸包边上),所以走不出三角剖分
            go = -1;
            for(int j = 0; j < 3; ++j) if(ori(tri[cur].v[(j + 1) % 3], tri[cur].v[(j + 2) % 3], p) < 0) { go = j; break; }
            if(go < 0 || tri[cur].f[go] < 0) break;
            cur = tri[cur].f[go];
        }
        int e = -1;  // p 正好落在某条边上
        for(int j = 0; j < 3; ++j) if(!ori(tri[cur].v[(j + 1) % 3], tri[cur].v[(j + 2) % 3], p)) { e = j; break; }
        st.clear();
        if(e < 0) {  // 1. p 在三角形内:拆成 3 个
            int a = tri[cur].v[0], b = tri[cur].v[1], c = tri[cur].v[2];
            int f0 = tri[cur].f[0], f1 = tri[cur].f[1], f2 = tri[cur].f[2];
            rm(cur);
            int t1 = mk(a, b, p), t2 = mk(b, c, p), t3 = mk(c, a, p);
            lnk(t1, 2, f2);
            lnk(t2, 2, f0);
            lnk(t3, 2, f1);
            lnk(t1, 0, t2);
            lnk(t2, 0, t3);
            lnk(t3, 0, t1);
            st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 2, tri[t2].ver}), st.push_back({t3, 2, tri[t3].ver});
            last = t1;
        } else {  // 2. p 在边 (a,b) 上:两侧的三角形各拆成 2 个(凸包边只有一侧)
            int a = tri[cur].v[(e + 1) % 3], b = tri[cur].v[(e + 2) % 3], c = tri[cur].v[e];
            int g = tri[cur].f[e];
            int fc1 = tri[cur].f[(e + 1) % 3], fc2 = tri[cur].f[(e + 2) % 3];
            if(g < 0) {  // 凸包边:只有一个三角形,拆成 2 个
                rm(cur);
                int t1 = mk(c, a, p), t2 = mk(c, p, b);
                lnk(t1, 1, t2);
                lnk(t1, 2, fc2);
                lnk(t2, 1, fc1);
                st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 1, tri[t2].ver});
                last = t1;
            } else {
```

```
                int kk = finde(g, b, a), d = tri[g].v[kk];
                int fn1 = tri[g].f[(kk + 1) % 3], fn2 = tri[g].f[(kk + 2) % 3];
                rm(cur);
                rm(g);
                int t1 = mk(c, a, p), t2 = mk(c, p, b), u1 = mk(d, b, p), u2 = mk(d,
    p, a);
                lnk(t1, 0, u2);
                lnk(t1, 1, t2);
                lnk(t2, 0, u1);
                lnk(u1, 1, u2);
                lnk(t1, 2, fc2);
                lnk(t2, 1, fc1);
                lnk(u1, 2, fn2);
                lnk(u2, 1, fn1);
                st.push_back({t1, 2, tri[t1].ver}), st.push_back({t2, 1,
    tri[t2].ver});
                st.push_back({u1, 2, tri[u1].ver}), st.push_back({u2, 1,
    tri[u2].ver});
                last = t1;
            }
        }
        flipall(st);
    }
    vect<p2> res;
    for(int t = 0; t < (int) tri.size(); ++t) if(live[t]) {
        int a = tri[t].v[0], b = tri[t].v[1], c = tri[t].v[2];
        res += pts[a], res += pts[b], res += pts[c];
    }
    return res;
}

}  // namespace Delaunay
using namespace Delaunay;
```

7.3 Voronoi 图

```
// 依赖:geo.cpp 的 p2 / eps / sign / cmp / convex_cut(用法是先 include geo.cpp 再
  include 本文件)。
// 参数:a 是站点集,允许有坐标相同的重复站点(见下)。站点数 n 可以很小(0/1/2/3 都行)。
// 返回:第 i 个元素是站点 i 的 Voronoi 区域 —— 一个逆时针的凸多边形(顶点是 db 坐标,
//   可能含有重复相邻点),它是「到 a[i] 的距离严格最近」的点集与包围盒 Voronoi::box 的交;
//   区域被包围盒裁掉,所以这是有限 Voronoi 图。区域至少 3 个顶点(站点一定在自己的区域内部),
//   不会返回空多边形。调用后 Voronoi::box[0] / box[1] 是左下、右上角(点集包围盒四周各外扩
//   max(宽,高)+1),相邻区域沿中垂线共边,所有区域的面积和 == 包围盒面积(在 box 内构成划分)。
// 做法:区域 i 从 box 开始,对每个 j != i 用它和 a[j] 的中垂线去裁掉「离 a[j] 更近」那半边 ——
//   Voronoi::bisector(a[i], a[j]) 给出的有向直线把 a[i] 放在左侧,正好是 convex_cut 保留的
  一侧。
//   重合站点(用 p2 的 eps 相等判定)直接跳过,于是它们的区域相同 —— 也就是「并列最近」的约定,
//   如果要严格划分请先去重。
// 前置条件:坐标在 db 精度内即可;站点重合的约定见上;没有其它前置条件(不需要一般位置,
//   多个站点共线、四点共圆都行)。
// 复杂度:每个站点最多裁 n-1 次,每次裁剪 O(区域顶点数),区域顶点数 <= n+3,
//   所以总复杂度最坏 O(n^3)、常见 O(n^2 * 区域平均顶点数);随机站点实测 n=1000 约 0.2s、
//   n=3000 约 2s(区域顶点总数约 6n)。n 到几千都能用,n 很大(1e5)请换
//   「Delaunay 三角剖分的对偶」那种 O(n log n) 做法。
namespace Voronoi {

array<p2, 2> box;  // 裁剪用的包围盒:box[0] 左下、box[1] 右上

// 到 a 比到 b 近的半平面:以 ab 的中垂线为界,方向取 r90(b - a),这样 a 在它的左侧
// (convex_cut 保留 cross(q, ·) >= 0 的一侧,正好是 a 这一侧)。
seg bisector(p2 a, p2 b) {
    p2 m = (a + b) / 2;
    return {m, m + r90(b - a)};
}

// 求各站点的 Voronoi 区域(半平面交逐站点裁剪包围盒);返回第 i 个元素为站点 i 的逆时针区域。
vect<vect<p2>> voronoi(vect<p2> a) {
    int n = (int) a.size();
    vect<vect<p2>> res;
    if(!n) return res;
    db x1 = a[0].x, x2 = a[0].x, y1 = a[0].y, y2 = a[0].y;
    for(int i = 1; i < n; ++i) x1 = min(x1, a[i].x), x2 = max(x2, a[i].x), y1 =
  min(y1, a[i].y), y2 = max(y2, a[i].y);
    db r = max(x2 - x1, y2 - y1) + 1;
    box = {p2{x1 - r, y1 - r}, p2{x2 + r, y2 + r}};
    vect<p2> ini;
    ini += p2{box[0].x, box[0].y}, ini += p2{box[1].x, box[0].y};
    ini += p2{box[1].x, box[1].y}, ini += p2{box[0].x, box[1].y};  // 逆时针
    for(int i = 0; i < n; ++i) {
        vect<p2> cur = ini;
        for(int j = 0; j < n; ++j) {
            if(j == i || a[i] == a[j] || cur.size() < 3) continue;
            cur = convex_cut((int) cur.size(), cur.data(), bisector(a[i], a[j]));
        }
        res += cur;
    }
    return res;
}

}  // namespace Voronoi
using namespace Voronoi;
```

7.4 三维凸包

```
// 依赖:p3 / cross / det / dis / dis2 与 eps / sign / cmp(见本目录 三维向量.cpp、
  geo.cpp),
//      以及 三维平面.cpp 的 plane(面片用单位法向判内外侧)。
//
// face3{a, b, c}:一个三角面片,存的是**输入数组的下标**;三角面的法向按 (b-a)×(c-a) 定,
// convex3d 返回的每个面法向都朝凸包外侧(于是所有输入点都在每个面的内侧)。
//
// vector<face3> convex3d(const vector<p3> &a)
//   增量法:先挑一个「不共面」的初始四面体,再逐点插入 —— 每次把该点能「看见」的面
//   (点到面的带号距离 > eps)全部删掉,再沿地平线(可见区与非可见区的交界边)把新点
```

```
//    连成新面。返回面片(顺序不保证;输入数组不会被修改)。
//    点数 < 4、或全部重合/共线/共面时不存在三维凸包,返回空 vector。
//    重复点、落在已有面/棱上(距离 <= eps)的点不会被当成新顶点。
// 前置条件:eps 是**绝对**容差(1e-10),坐标量级很大时请自行放宽(否则点会被误当共面)。
// 复杂度:O(n * 当前面数) = 期望 O(n²),n <= 2000 量级够用。
//
// db hull_area(a, f)   f 是 convex3d 的返回值:凸包表面积
// db hull_volume(a, f) 凸包体积(面朝向一致,公式与原点位置无关)
//
// 已知约定(要么遵守要么自己处理):
//    - 共面/共线的退化输入返回空,调用方需要自己判(本函数不抛异常、不产生 NaN)。
//    - 全部点共面时不做「二维凸包」:那是另一份模板(二维 convex_hull)的活。
namespace Geo {

struct face3 {
    int a, b, c;
};

// 三维凸包(增量法);返回三角面片(法向朝外,下标为输入数组下标),退化输入返回空。
vector<face3> convex3d(const vector<p3> &a) {
    int n = a.size();
    vector<face3> ret;
    if(n < 4) return ret;
    // ---- 1. 找一个不共面的初始四面体 A,B,C,D ----
    int A = 0, B = -1, C = -1, D = -1;
    for(int i = 0; i < n; ++i) if(sign(dis(a[i] - a[A]))) { B = i; break; }  // 与 A 不重合
    if(B < 0) return ret;                                                    // 全部点重合
    p3 w0 = a[B] - a[A];
    for(int i = 0; i < n; ++i) if(sign(dis(cross(a[i] - a[A], w0)) / dis(w0))) { C = i; break; }  // 不共线
    if(C < 0) return ret;                                                    // 全部点共线
    plane p0(a[A], a[B], a[C]);
    for(int i = 0; i < n; ++i) if(sign(abs(p0.side(a[i])))) { D = i; break; }  // 离开 A,B,C 所在平面
    if(D < 0) return ret;                                                    // 全部点共面
    // ---- 2. 把四个种子点排到前面,后面按顺序插入 ----
    vector<int> id;  // 局部下标 -> 原下标
    id.push_back(A), id.push_back(B), id.push_back(C), id.push_back(D);
    for(int i = 0; i < n; ++i) if(i != A && i != B && i != C && i != D) id.push_back(i);
    vector<p3> b(n);
    for(int i = 0; i < n; ++i) b[i] = a[id[i]];
    struct F {
        int a, b, c;
        plane p;
    };
    vector<F> f;
    f.push_back(F{0, 1, 2, plane(b[0], b[1], b[2])});  // 两个朝向的初始三角形,
    f.push_back(F{0, 2, 1, plane(b[0], b[2], b[1])});  // 插入第 4 个点时朝向会自动定好
    unordered_map<ll, int> vis;                        // 有向边 (u,v) -> 最后一次被删的轮次
    For(i, 3, n - 1) {
        vector<F> keep, kill;
        for(int t = 0; t < f.size(); ++t) {
            if(f[t].p.side(b[i]) > eps) kill.push_back(f[t]);  // 严格在外侧 -> 可见,删掉重连
            else keep.push_back(f[t]);
        }
        if(kill.empty()) continue;  // 点在凸包内(或离面不超过 eps):不产生新顶点
        int e[3][2];
        for(int t = 0; t < kill.size(); ++t) {  // 先把被删面的三条有向边打上本轮标记
            e[0][0] = kill[t].a, e[0][1] = kill[t].b;
            e[1][0] = kill[t].b, e[1][1] = kill[t].c;
            e[2][0] = kill[t].c, e[2][1] = kill[t].a;
            for(int k = 0; k < 3; ++k) vis[(ll) e[k][0] * n + e[k][1]] = i;
        }
        for(int t = 0; t < kill.size(); ++t) {  // 反向边没被标记的边就是地平线,与新点连成新面
            e[0][0] = kill[t].a, e[0][1] = kill[t].b;
            e[1][0] = kill[t].b, e[1][1] = kill[t].c;
            e[2][0] = kill[t].c, e[2][1] = kill[t].a;
            for(int k = 0; k < 3; ++k)
                if(vis[(ll) e[k][0] * n + e[k][1]] == i && vis[(ll) e[k][1] * n + e[k][0]] != i)
                    keep.push_back(F{e[k][0], e[k][1], i, plane(b[e[k][0]], b[e[k][1]], b[i])});
        }
        f.swap(keep);
    }
    for(int t = 0; t < f.size(); ++t) ret.push_back(face3{id[f[t].a], id[f[t].b], id[f[t].c]});
    return ret;
}

// 凸包表面积(f 的面法向朝外,这里只用三角形面积,与朝向无关)
db hull_area(const vector<p3> &a, const vector<face3> &f) {
    db s = 0;
    for(int t = 0; t < f.size(); ++t) s += area2(a[f[t].a], a[f[t].b], a[f[t].c]);
    return s / 2;
}
// 凸包体积:每个面与原点围成的四面体带号体积求和(朝向一致时符号相同,取绝对值)
db hull_volume(const vector<p3> &a, const vector<face3> &f) {
    db s = 0;
    for(int t = 0; t < f.size(); ++t) s += det(a[f[t].a], a[f[t].b], a[f[t].c]);
    return abs(s) / 6;
}

}  // namespace Geo
using namespace Geo;
```

7.5 三维向量

```
// 依赖:Geo 里的 eps / sign / cmp / db(见本目录 geo.cpp),以及 house 的 db(long double)。
// p3 的接口与 geo.cpp 的 p2 平行(都是命名空间 Geo 里的自由函数):
```

```
//   a + b, a - b, -a, a * k, k * a, a / k      基础运算
//   a * b                                     点积(p2 的 * 也是点积)
//   cross(a, b)                               三维叉积(右手法则);|cross| = 平行四边形面积
//   det(a, b, c) = a · (b × c)                混合积 = 平行六面体带号体积
//   area2(a, b, c)   = |(b-a) × (c-a)|        三角形面积的 2 倍
//   volume6(a, b, c, d) = det(b-a, c-a, d-a)  四面体带号体积的 6 倍
//   单位化 / 判等 / 字典序 / 垂直向量          unit / == / != / < / perp
//   夹角:cosang(a, b) 余弦、sinang(a, b) 正弦(>= 0)、angle(a, b) = atan2(sin, cos) ∈ [0, pi]
//   退化判定:colinear(a, b, c) 三点共线、coplanar(a, b, c, d) 四点共面
//
// 前置条件:unit(a) 要求 a 非零(零向量会得到 inf/nan)。
// eps 是绝对容差:colinear / coplanar 直接将叉积模长/混合积与 eps 比较。
// 坐标缩放会改变判定尺度,应按数据尺度调整容差或另用相对判据。
// 复杂度:全部 O(1)。
namespace Geo {

struct p3 {
    db x, y, z;
    // 按 eps 容差判等。
    bool operator==(const p3 &b) const {
        return !cmp(x, b.x) && !cmp(y, b.y) && !cmp(z, b.z);
    }
    // 按 eps 容差判不等。
    bool operator!=(const p3 &b) const { return !(*this == b); }
    // 字典序(x,y,z),用于排序/二分。
    bool operator<(const p3 &b) const {
        int c = cmp(x, b.x);
        if(c) return c < 0;
        c = cmp(y, b.y);
        if(c) return c < 0;
        return cmp(z, b.z) < 0;
    }
};
// 向量相加。
p3 operator+(p3 a, p3 b) { return {a.x + b.x, a.y + b.y, a.z + b.z}; }
// 向量相减。
p3 operator-(p3 a, p3 b) { return {a.x - b.x, a.y - b.y, a.z - b.z}; }
// 取负。
p3 operator-(p3 a) { return {-a.x, -a.y, -a.z}; }
// 数乘(标量在右)。
p3 operator*(p3 a, db k) { return {a.x * k, a.y * k, a.z * k}; }
// 数乘(标量在左)。
p3 operator*(db k, p3 a) { return {a.x * k, a.y * k, a.z * k}; }
// 除以标量;要求 k 非零。
p3 operator/(p3 a, db k) { return {a.x / k, a.y / k, a.z / k}; }
// 向量自加,返回新值。
p3 operator+=(p3 &a, p3 b) { return a = a + b; }
// 向量自减,返回新值。
p3 operator-=(p3 &a, p3 b) { return a = a - b; }
// 向量自乘标量,返回新值。
p3 operator*=(p3 &a, db k) { return a = a * k; }
// 向量自除标量,返回新值;要求 k 非零。
p3 operator/=(p3 &a, db k) { return a = a / k; }
// 返回两向量的点积。
db operator*(p3 a, p3 b) { return a.x * b.x + a.y * b.y + a.z * b.z; }
// 三维叉积(右手法则),模长为两向量张成平行四边形面积。
p3 cross(p3 a, p3 b) {
    return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
}
// 返回混合积 a·(b×c),即平行六面体带号体积。
db det(p3 a, p3 b, p3 c) { return a * cross(b, c); }
// 模长平方。
db dis2(p3 a) { return a.x * a.x + a.y * a.y + a.z * a.z; }
// 模长。
db dis(p3 a) { return sqrt(dis2(a)); }
// 单位化;要求 a 非零(零向量得 inf/nan)。
p3 unit(p3 a) { return a / dis(a); }
// 返回夹角余弦;要求 a、b 均非零。
db cosang(p3 a, p3 b) { return (a * b) / (dis(a) * dis(b)); }
// 返回夹角的非负正弦;要求 a、b 均非零。
db sinang(p3 a, p3 b) { return dis(cross(a, b)) / (dis(a) * dis(b)); }
// 返回 [0,pi] 内的夹角(弧度);要求 a、b 均非零。
db angle(p3 a, p3 b) { return atan2(sinang(a, b), cosang(a, b)); }
// 返回三角形 abc 面积的 2 倍。
db area2(p3 a, p3 b, p3 c) { return dis(cross(b - a, c - a)); }
// 返回四面体 abcd 带号体积的 6 倍。
db volume6(p3 a, p3 b, p3 c, p3 d) { return det(b - a, c - a, d - a); }
// 返回与 a 垂直的向量;a 非零时结果也非零。
p3 perp(p3 a) {
    return abs(a.x) > abs(a.z) ? p3{a.y, -a.x, 0} : p3{0, -a.z, a.y};
}
// 三点是否共线。
bool colinear(p3 a, p3 b, p3 c) { return cmp(dis(cross(b - a, c - a)), 0) == 0; }
// 四点是否共面。
bool coplanar(p3 a, p3 b, p3 c, p3 d) { return sign(abs(det(b - a, c - a, d - a))) == 0; }

}  // namespace Geo
using namespace Geo;
```

7.6 三维平面

```
// 依赖:p3 / cross / det / dis / dis2 / unit / cosang / sinang 与 eps / sign / cmp
//      (见本目录 三维向量.cpp 与 geo.cpp),以及 三维直线.cpp 的 line3。
// 平面用点法式 n·x = d 表示,n 是**单位**法向量(构造时已归一化),于是:
//   side(p) = n·p - d 就是 p 到平面的**带号距离**(法向一侧为正),dis(p) = |side(p)|。
//
// 接口:
//   plane(a, b, c)        过三点的平面,法向 = (b-a)×(c-a)(右手法则);三点共线时退化成 n = 0
//   plane::from(o, nor)   过点 o、法向 nor(nor 非零)
//   side / dis / proj / reflect / ons(p)(点是否在平面上)
//   ispara / isperp / angle(a, b)(两平面夹角,取 [0, pi/2] 的锐角)
//   ispl(a, b)            两平面交线(平行时方向为零向量,结果无意义)
```

```
//   islp(p, l)            直线与平面的交点(要求两者不平行:n·l.d != 0)
//   perpthrough(p, o)     过点 o 作平面的垂线
//   ons(p, l)             直线是否整个落在平面内
//
// 前置条件:from() 的 nor 非零;islp() 要求 n·l.d != 0;ispl() 要求两平面不平行。
// eps 是绝对容差(1e-10),坐标量级很大时请自行放宽。
// 复杂度:全部 O(1)。
namespace Geo {

struct plane {
    p3 n;  // 单位法向量
    db d;  // n · x = d
    // 默认构造:成员未初始化,使用前须赋值。
    plane() {}
    // 过三点的平面,法向 (b-a)×(c-a);三点共线时退化为 n=0。
    plane(p3 a, p3 b, p3 c) {
        p3 t = cross(b - a, c - a);
        db l = sqrt(t * t);  // 注意:这里不能写 dis(t) —— 会被解析成成员 dis(p3)(遮蔽自由函数)
        if(!sign(l)) {
            n = {0, 0, 0}, d = 0;  // 三点共线:退化(整个平面无意义)
        } else {
            n = t / l, d = n * a;
        }
    }
    // 过点 o、法向 nor 的平面(nor 非零,内部单位化)。
    static plane from(p3 o, p3 nor) {
        plane r;
        r.n = unit(nor), r.d = r.n * o;
        return r;
    }
    // 返回点 p 到有效平面的带号距离,单位法向一侧为正。
    db side(p3 p) const { return n * p - d; }
    // 返回点 p 到有效平面的距离。
    db dis(p3 p) const { return abs(side(p)); }
    // 返回点 p 在有效平面上的正投影。
    p3 proj(p3 p) const { return p - n * side(p); }
    // 关于平面的对称点。
    p3 reflect(p3 p) const { return p - n * (2 * side(p)); }
    // 点是否在平面上。
    bool ons(p3 p) const { return !sign(side(p)); }
};
// 两平面是否平行(含重合)。
bool ispara(const plane &a, const plane &b) { return !sign(sinang(a.n, b.n)); }
// 两平面是否垂直。
bool isperp(const plane &a, const plane &b) { return !sign(cosang(a.n, b.n)); }
// 两平面夹角:法向量夹角的锐角部分,取值 [0, pi/2]
db angle(const plane &a, const plane &b) { return acos(max((db) -1, min((db) 1, abs(a.n * b.n)))); }
// 两平面的交线:方向 n1×n2,过点 (d1·(n2×w) + d2·(w×n1)) / |w|²(w = n1×n2)
// 平行(含重合)时没有唯一的交线:返回方向为零向量的退化直线,调用方用 dis(l.d) == 0 判退化
line3 ispl(const plane &a, const plane &b) {
    p3 w = cross(a.n, b.n);
    if(!sign(sinang(a.n, b.n))) return line3(a.proj(b.n * b.d), a.proj(b.n * b.d));
    p3 o = (cross(b.n, w) * a.d + cross(w, a.n) * b.d) / dis2(w);
    return line3(o, o + w);  // 传两点(o, o+w),d 就是 w
}
// 直线与平面的交点:解 n·(o + t·d) = d 得 t = (d - n·o) / (n·d)
p3 islp(const plane &p, const line3 &l) { return l.o + l.d * ((p.d - p.n * l.o) / (p.n * l.d)); }
// 过点 o 作平面的垂线
line3 perpthrough(const plane &p, p3 o) { return line3(o, o + p.n); }
// 直线是否整个在平面内
bool ons(const plane &p, const line3 &l) { return !sign(p.side(l.o)) && !sign(p.n * l.d); }
// 点到平面距离(转发成员版)。
db dis(const plane &p, p3 x) { return p.dis(x); }
// 点在平面上的投影(转发成员版)。
p3 proj(const plane &p, p3 x) { return p.proj(x); }

}  // namespace Geo
using namespace Geo;
```

7.7 三维旋转

```
// 依赖:p3 / cross / dis / dis2 / unit / perp / det 与 eps / sign / cmp(见本目录 三维向量.cpp、geo.cpp)。
//
// p3 rot(v, axis, ang)            把 v 绕「过原点、方向 axis」的轴转 ang 弧度(罗德里格斯公式)
// p3 rot(v, axis, ang, o)         绕「过点 o、方向 axis」的轴转 ang
// mat3 rotmat(axis, ang)          旋转矩阵(R = I·cos + sin·[k]× + (1-cos)·k kᵀ)
// m * v / x * y                   矩阵作用到向量 / 矩阵相乘
// mat3 transpose(m)
// void basis(p3 n, p3 &u, p3 &v, p3 &w)   由法向 n 造右手正交基(w = n 的单位化)
// p3 tolocal(x, o, u, v, w)       世界坐标 -> 以 o 为原点、u/v/w 为基的局部坐标
// p3 toworld(l, o, u, v, w)       局部坐标 -> 世界坐标(tolocal 的逆)
//
// 约定:轴方向必须非零(内部会单位化);零轴会得到 NaN。
// 旋转保长度、保夹角;rot 与 rotmat 是同一件事的两种写法(可互相校验)。
// 复杂度:全部 O(1)。
namespace Geo {

struct mat3 {
    db a[3][3];
    // 矩阵作用于向量。
    p3 operator*(p3 v) const {
        return {a[0][0] * v.x + a[0][1] * v.y + a[0][2] * v.z,
                a[1][0] * v.x + a[1][1] * v.y + a[1][2] * v.z,
                a[2][0] * v.x + a[2][1] * v.y + a[2][2] * v.z};
    }
};
// 矩阵相乘。
mat3 operator*(const mat3 &x, const mat3 &y) {
    mat3 r{};
```

```cpp
    for(int i = 0; i < 3; ++i)
        for(int j = 0; j < 3; ++j) {
        db s = 0;
        for(int k = 0; k < 3; ++k) s += x.a[i][k] * y.a[k][j];
        r.a[i][j] = s;
    }
    return r;
}
// 转置。
mat3 transpose(const mat3 &m) {
    mat3 r{};
    for(int i = 0; i < 3; ++i)
        for(int j = 0; j < 3; ++j) r.a[i][j] = m.a[j][i];
    return r;
}
// 罗德里格斯:v' = v·cos + (k×v)·sin + k·(k·v)·(1-cos)
p3 rot(p3 v, p3 axis, db ang) {
    p3 k = unit(axis), c = cross(k, v), d = k * (k * v);
    return v * cos(ang) + c * sin(ang) + d * (1 - cos(ang));
}
// 绕「过点 o、方向 axis」的轴转 ang 弧度;要求 axis 非零。
p3 rot(p3 v, p3 axis, db ang, p3 o) { return o + rot(v - o, axis, ang); }
// 旋转矩阵;要求 axis 非零。
mat3 rotmat(p3 axis, db ang) {
    p3 k = unit(axis);
    db c = cos(ang), s = sin(ang), t = 1 - c;
    mat3 m{};
    m.a[0][0] = t * k.x * k.x + c, m.a[0][1] = t * k.x * k.y - s * k.z, m.a[0][2] =
t * k.x * k.z + s * k.y;
    m.a[1][0] = t * k.x * k.y + s * k.z, m.a[1][1] = t * k.y * k.y + c, m.a[1][2] =
t * k.y * k.z - s * k.x;
    m.a[2][0] = t * k.x * k.z - s * k.y, m.a[2][1] = t * k.y * k.z + s * k.x, m.a[2]
[2] = t * k.z * k.z + c;
    return m;
}
// 以 n 为第三个基向量造右手正交基(u, v, w 两两垂直、都是单位向量、det(u,v,w) = 1)
void basis(p3 n, p3 &u, p3 &v, p3 &w) {
    w = unit(n), u = unit(perp(w)), v = cross(w, u);
}
// 世界坐标转局部坐标(以 o 为原点、u/v/w 为基)。
p3 tolocal(p3 x, p3 o, p3 u, p3 v, p3 w) {
    p3 d = x - o;
    return {u * d, v * d, w * d};
}
// 局部坐标转世界坐标(tolocal 的逆)。
p3 toworld(p3 l, p3 o, p3 u, p3 v, p3 w) { return o + u * l.x + v * l.y + w * l.z; }

}  // namespace Geo
using namespace Geo;
```

7.8 三维直线与线段


```cpp
// 依赖:p3 / cross / det / dis / dis2 / unit / cosang / sinang 与 eps / sign / cmp
//      (见本目录 三维向量.cpp 与 geo.cpp)。
// 两条线段分别用 seg3{x, y} 表示;直线用参数式 line3{o, d}(o + t·d),d 非零即可、不必单位。
//
// 点与直线:proj / reflect / dis / ons(点是否在直线上,按距离 <= eps 判)
// 两直线:  dis(含平行、相交、异面)/ closest_on(a, b)(a 上离 b 最近的点)/ ispara / isperp
// 点与线段:proj(最近点,参数夹到 [0,1])/ dis / ons(点是否在线段上)/ ons_s(严格在内部)
// 两线段:  dis(a, b, pa, pb)(最近点对)/ dis(a, b) / chkss(相交,含端点相接与共线重叠)
//          chkss_s(严格相交:交点同时在两段内部,共线不算)/ isss(交于唯一一点时输出交点)
//
// 前置条件与约定:
//   - 零长线段(退化成点)被当作点处理:proj/dis/ons/两线段距离都成立(不会出 NaN)。
//   - 平行/共线的两线段最近点对不唯一,只保证「距离」正确。
//   - 判定门槛都是绝对 eps(1e-10):坐标量级很大(>1e6)时请自行放宽容差。
//   - 两线段接近平行时,最近点对的 s/t 由显式公式解出后夹取,已避免除以 0(平行分支另走)。
// 复杂度:全部 O(1)。
namespace Geo {

struct line3 {
    p3 o, d;  // 参数式 o + t·d
    // 默认构造:成员未初始化。
    line3() {}
    // 由两点构造直线(o=a,d=b-a)。
    line3(p3 a, p3 b) : o(a), d(b - a) {}
    // 参数 t 对应的点。
    p3 at(db t) const { return o + d * t; }
};
struct seg3 {
    p3 x, y;
    // 方向向量 y - x。
    p3 dir() const { return y - x; }
};

// ---------- 点与直线 ----------
// 点在直线上的投影。
p3 proj(const line3 &l, p3 p) { return l.o + l.d * ((l.d * (p - l.o)) /
dis2(l.d)); }
// 点关于直线的对称点。
p3 reflect(const line3 &l, p3 p) { return proj(l, p) * 2 - p; }
// 点到直线的距离。
db dis(const line3 &l, p3 p) { return dis(p - proj(l, p)); }
// 点是否在直线上。
bool ons(const line3 &l, p3 p) { return !sign(dis(l, p)); }
// 两直线是否平行。
bool ispara(const line3 &a, const line3 &b) { return !sign(sinang(a.d, b.d)); }
// 判断线段方向是否平行;按 eps 判为零长的线段约定与任意线段平行。
bool ispara(const seg3 &a, const seg3 &b) {
    return !sign(dis(a.dir())) || !sign(dis(b.dir())) || !sign(sinang(a.dir(),
b.dir()));
}
// 两直线是否垂直。
```

```
bool isperp(const line3 &a, const line3 &b) { return !sign(cosang(a.d, b.d)); }

// ---------- 两直线:最近点对与距离 ----------
// pa 在 a 上、pb 在 b 上,dis(pa, pb) 即两直线距离;平行时取 pa = a.o。
db dis(const line3 &a, const line3 &b, p3 &pa, p3 &pb) {
    if(ispara(a, b)) {
        pa = a.o, pb = proj(b, pa);
        return dis(pa - pb);
    }
    db A = dis2(a.d), B = a.d * b.d, C = dis2(b.d), den = A * C - B * B;
    p3 w = a.o - b.o;
    db D = a.d * w, E = b.d * w;
    pa = a.o + a.d * ((B * E - C * D) / den);
    pb = b.o + b.d * ((A * E - B * D) / den);
    return dis(pa - pb);
}
// 两直线距离(忽略最近点)。
db dis(const line3 &a, const line3 &b) { p3 pa, pb; return dis(a, b, pa, pb); }
// a 上离 b 最近的点。
p3 closest_on(const line3 &a, const line3 &b) { p3 pa, pb; return dis(a, b, pa, pb), pa; }

// ---------- 点与线段 ----------
// 点在线段上的最近点(参数夹到 [0,1]);线段可退化为点。
p3 proj(const seg3 &s, p3 p) {
    p3 d = s.dir();
    if(!sign(dis2(d))) return s.x;  // 退化成点
    db t = (d * (p - s.x)) / dis2(d);
    return s.x + d * max((db) 0, min((db) 1, t));
}
// 点到线段的距离。
db dis(const seg3 &s, p3 p) { return dis(proj(s, p) - p); }
// 点是否在线段上(含端点)。
bool ons(const seg3 &s, p3 p) { return !sign(dis(s, p)); }
// 点是否严格在线段内部(不含端点)。
bool ons_s(const seg3 &s, p3 p) { return ons(s, p) && sign((p - s.x) * (p - s.y)) < 0; }

// ---------- 两线段:最近点对与距离 ----------
// pa 在 a 上、pb 在 b 上,返回 |pa - pb|;退化(零长)线段也能用。
db dis(const seg3 &a, const seg3 &b, p3 &pa, p3 &pb) {
    p3 u = a.dir(), v = b.dir();
    db la = dis(u), lb = dis(v);
    if(!sign(la) || !sign(lb)) {  // 至少一条退化成点
        if(!sign(la) && !sign(lb)) return pa = a.x, pb = b.x, dis(pa - pb);
        if(!sign(la)) return pa = a.x, pb = proj(b, pa), dis(pa - pb);
        return pb = b.x, pa = proj(a, pb), dis(pa - pb);
    }
    p3 e = u / la, f = v / lb, w = a.x - b.x;
    db c = e * f, k = 1 - c * c, p = e * w, q = f * w, s, t;
    if(ispara(a, b)) {  // 平行:最近点对不唯一,s = 0 起步,后面照样投影夹取
        s = 0, t = q / lb;
    } else {
        s = (c * q - p) / (la * k), t = (q - c * p) / (lb * k);
    }
    s = max((db) 0, min((db) 1, s)), t = max((db) 0, min((db) 1, t));
    // |w + s·u - t·v|² 是 (s, t) 上的凸二次函数:盒约束下逐坐标做「一维精确最优 + 夹取」,
    // 每步都让距离不增,几轮就收敛(平行时的平坦方向也在其中)
    For(_, 1, 12) {
        t = max((db) 0, min((db) 1, (q + s * la * c) / lb));
        s = max((db) 0, min((db) 1, (t * lb * c - p) / la));
    }
    pa = a.x + u * s, pb = b.x + v * t;
    return dis(pa - pb);
}
// 两线段间距离(忽略最近点)。
db dis(const seg3 &a, const seg3 &b) { p3 pa, pb; return dis(a, b, pa, pb); }

// 两线段是否相交(公共点存在即真,含端点相接、共线重叠)。
bool chkss(const seg3 &a, const seg3 &b) { return !sign(dis(a, b)); }
// 严格相交:交点严格在两段内部(共线/平行/端点相接都不算)。
bool chkss_s(const seg3 &a, const seg3 &b) {
    if(ispara(a, b) || !sign(dis(a.dir())) || !sign(dis(b.dir()))) return false;
    p3 pa, pb;
    return !sign(dis(a, b, pa, pb)) && ons_s(a, pa) && ons_s(b, pb);
}
// 交于唯一一点时输出该点(共线重叠/不相交返回 false)。交点取 a 上的最近点。
bool isss(const seg3 &a, const seg3 &b, p3 &p) {
    if(ispara(a, b) || !sign(dis(a.dir())) || !sign(dis(b.dir()))) return false;
    p3 pa, pb;
    return !sign(dis(a, b, pa, pb)) ? (p = pa, true) : false;
}

}  // namespace Geo
using namespace Geo;
```

7.9 上凸壳

```
//
// int upper_hull(int n, p2 *a, p2 *b, bool nos = 0): 求点集 a[0..n-1] 的**上凸壳**(上半条凸链),
//   结果按 x 递增写进 b,返回点数。上凸壳 = 「从最左上角的点到最右上角的点」那段边界,链上相邻
//   三点全是右转(叉积 < 0)。
//   nos = 0:严格上凸壳(去掉上边界上的共线点);nos = 1:保留上边界上的共线点。
//   每个不同的 x 只保留 y 最大的那个点(竖直边上只留最高点),这正是「上凸壳」的通常定义。
//   n <= 0 返回 0;n == 1 返回 1 并把唯一点写进 b;会原地排序 a(调用方注意)。
//   复杂度 O(n log n)(排序 + 单调栈),空间 O(1) 额外。
//   依赖:geo.cpp 的 p2 / eps / cmp / crossop。
//   用途:斜率优化的「上凸壳」判定、凸包直径/切线的双指针、只在凸包上方取极值的一类问题;
//   要完整凸包请用 geo.cpp 的 convex_hull(n, a, b, nos)。

int upper_hull(int n, p2 *a, p2 *b, bool nos = 0) {
    if(n <= 0) return 0;
    sort(a, a + n, [](const p2 &u, const p2 &v) { return u.x != v.x ? u.x < v.x : u.y > v.y; });
```

```
    int k = 0;
    for(int i = 0; i < n; ++i) {
        if(k && b[k - 1].x == a[i].x) continue;  // 同一个 x(精确相等)只留最高的那个
        while(k > 1 && crossop({b[k - 2], b[k - 1]}, a[i]) >= (int) nos) --k;  // nos=0:只留右转;nos=1:共线也留
        b[k++] = a[i];
    }
    return k;
}
```

7.10 凸包内点判定

```
//
// int in_convex(int n, p2 *a, const p2 &p): 判定点 p 与逆时针凸多边形 a[0..n-1] 的关系。
//   返回 0:严格在外;1:在边界上(边或顶点);2:严格在内。
//   前置条件:n >= 3;a 逆时针、**严格凸**(没有三点共线的多余顶点 —— convex_hull(nos=0) 的输出满足这条)。带共线多余顶点时「内部对角线上的点」会被误判成边界。
//   复杂度 O(log n)(以 a[0] 为扇心二分定位所在的扇形三角形,再判最后一条外边)。
//   依赖:geo.cpp 的 p2 / eps / crossop / ons。
//   说明:有共线多余顶点时先用 convex_hull(nos=0) 去一下;或者改用 O(n) 的 contain(n, a, p)(geo.cpp 自带),它的 0/1/2 语义与这里的 eps 容差一致。

int in_convex(int n, p2 *a, const p2 &p) {
    if(n < 3) return 0;
    if(crossop({a[0], a[1]}, p) < 0 || crossop({a[0], a[n - 1]}, p) > 0) return 0;  // 落在以 a[0] 为心的扇形外
    if(ons({a[0], a[1]}, p) || ons({a[n - 1], a[0]}, p)) return 1;                  // 与 a[0] 相邻的两条边
    int l = 1, r = n - 1;
    while(l + 1 < r) {  // 找最大的 l,使 p 不在 a[0]->a[l] 的右侧
        int m = (l + r) >> 1;
        if(crossop({a[0], a[m]}, p) >= 0)
            l = m;
        else
            r = m;
    }
    int c = crossop({a[l], a[l + 1]}, p);  // p 只可能在扇形三角形 (a[0],a[l],a[l+1]) 里
    if(c == 0) return 1;                   // 落在外边 a[l]a[l+1] 上(含端点 a[l])
    if(c < 0) return 0;                    // 落在外边之外(含「对角线超出 a[l]」的情形)
    if(crossop({a[0], a[l]}, p) == 0) return ons({a[0], a[l]}, p) ? 2 : 0;  // 严格凸时对角线在内部
    return 2;
}
```

7.11 半平面交

```
//
// hpi(vector<seg> vs): 交所有「有向直线 vs[i] 的**左侧**半平面」(左侧 = cross(vs[i], p) >= 0)。
//   返回交集的边界多边形顶点,**逆时针**(沿每条直线的前进方向走时内部在左)。
//   交集为空、或退化成点/线段(带号面积 <= eps)时返回空 vector。**交集无界时结果没有意义**
//   (真实区域没法用有限顶点表示,队列会给出一个既违反输入直线、面积又为负的假多边形),
//   所以调用方一定要保证有界:常规做法是把 4 条 ±B 的边界直线(B 取足够大,如 1e9)一起传进来,把无界区域切成有界多边形。
//   角度相差 < eps 的直线视为同向平行,只保留限制更强的那条(左侧更靠左的一条)。
//   前置条件:vs 非空;各条直线的方向两两不「几乎平行」—— 注意这里的「平行」指**无向**的平行,即两条直线夹角接近 0 或接近 pi 都不行(方向差 pi 时交点在 1/夹角 量级上,会失真)。
//   这是浮点半平面交的固有限制,不是本实现的特性。
//   复杂度:O(n log n)(按极角排序 + 单调双端队列),空间 O(n)。
//   依赖:geo.cpp 的 p2 / seg / eps / cmp / sign / crossop / isll / pi(用了 dir().alpha())。
//   提示:直线用两点给出方向,点的先后顺序决定哪一侧是内侧;要写「ax + by + c <= 0」时把方向取成 (-b, a) 即可。

vector<p2> hpi(vector<seg> vs) {
    if(vs.empty()) return {};
    sort(vs.begin(), vs.end(), [](const seg &a, const seg &b) {
        db x = a.dir().alpha(), y = b.dir().alpha();  // 按精确极角排序,保证严格弱序
        if(x != y) return x < y;
        return crossop(a, b.x) < 0;  // 同向时 b 在 a 右侧 -> a 的左侧半平面更小,排前面
    });
    vector<seg> ls;  // 同向平行(角度差 < eps)只留更强的一条
    for(int i = 0; i < (int) vs.size(); ++i) {
        if(!ls.empty() && !cmp(ls.back().dir().alpha(), vs[i].dir().alpha())) {
            // 注意:近乎同向的两条直线极角可能只差 1 ulp,谁在前是随机的 —— 必须显式比「强弱」:
            // crossop(ls.back(), vs[i].x) >= 0 表示 vs[i] 的限制更紧(或相同),用它替换
            if(crossop(ls.back(), vs[i].x) >= 0) ls.back() = vs[i];
            continue;
        }
        ls.push_back(vs[i]);
    }
    if(ls.size() > 1 && !cmp(ls.front().dir().alpha() - ls.back().dir().alpha() + 2 * pi, 2 * pi)) {
        if(crossop(ls.front(), ls.back().x) >= 0)
            ls.erase(ls.begin());
        else
            ls.pop_back();
    }
    int n = ls.size();
    if(n < 3) return {};
    vector<int> q(n + 1);
    int h = 0, t = 2;  // 队列里是 q[h..t-1],先放前两条
    q[0] = 0, q[1] = 1;
    for(int i = 2; i < n; ++i) {
        while(t - h >= 2 && crossop(ls[i], isll(ls[q[t - 2]], ls[q[t - 1]])) < 0) --t;
        while(t - h >= 2 && crossop(ls[i], isll(ls[q[h]], ls[q[h + 1]])) < 0) ++h;
        q[t++] = i;
    }
    while(t - h >= 2 && crossop(ls[q[h]], isll(ls[q[t - 2]], ls[q[t - 1]])) < 0) --t;
    while(t - h >= 2 && crossop(ls[q[t - 1]], isll(ls[q[h]], ls[q[h + 1]])) < 0) ++h;
    if(t - h < 3) return {};
    vector<p2> ret;  // 相邻两条直线的交点,按环形顺序:逆时针
```

```
    for(int i = h; i < t; ++i) ret.push_back(isll(ls[q[i]], ls[q[i + 1 < t ? i + 1 : h]]));
    vector<p2> qs;
    for(int i = 0; i < (int) ret.size(); ++i) if(qs.empty() || !(qs.back() == ret[i])) qs.push_back(ret[i]);  // 合并重合点
    while(qs.size() > 1 && qs.front() == qs.back()) qs.pop_back();
    if(qs.size() < 3) return {};
    db s = 0;
    for(int i = 0; i < (int) qs.size(); ++i) s += qs[i].det(qs[(i + 1) % qs.size()]);
    // 交集为空时,双端队列会退化成一组「自相矛盾」的直线,交出来的多边形必然是**顺时针**的
    // (真实的交集多边形一定逆时针:沿每条直线前进时内部在左)。所以面积 <= 0 一律按空处理 ——
    // 这一条是「空交集」的唯一判别手段:不判的话空交集会返回一个既违反输入直线、面积又为负的
    // 假多边形。这也是教科书式哨兵写法的共同弱点,这里用 O(k) 的面积符号兜住。
    if(sign(s) <= 0) return {};
    return qs;
}
```

7.12 圆相关求交与面积交

```
// 圆 与 直线/线段/圆 的交点,以及 圆与圆的交面积、圆与简单多边形的交面积。
// 依赖 geo.cpp 的 p2 / seg / eps(1e-10) / pi / sign / cmp / det / dis / dis2 / alpha / crossop /
//   isll / ons / area / contain;不定义 main,不 include。
// 本文件自带 struct circle { p2 o; db r; }(geo.cpp 里没有 circle);若与「最小圆覆盖.cpp」
//   同时 include,删掉其中一份 circle 定义(两份字段相同,兼容)。
//
// 接口(全部按 vector<p2> 返回;db = long double):
//   cir_line(c, l)        圆与直线 l 的交点。0 个(相离:判别式 < -eps)/ 1 个(相切)/
//                         2 个(按参数 t = dir*(p - x) 递增排序,即从 l.x 指向 l.y 的方向)。
//                         圆心在直线上时两交点关于垂足对称。要求 l.x != l.y(否则返回空)。
//   cir_seg(c, l)         圆与线段 l 的交点(先算直线交点,再用 ons 过滤)。端点都在圆内
//                         -> 0 个;恰好穿过 -> 2 个;一个端点在圆上 -> 1 个。同样按 t 递增。
//   cir_cir(c1, c2)       两圆交点:0 / 1(相切)/ 2 个,按相对 c1.o 的极角 alpha() 排序。
//                         内含、内离、同心一律返回空;r = 0 的点圆除非该点恰在另一圆上,否则空。
//   cir_cir_area(c1, c2) 两圆「交」面积(解析法:S = r1^2*a1 + r2^2*a2 - d*h/2,扇形 - 三角形)。
//                         内含/内切/重合返回 pi * min(r1,r2)^2(小圆整个被盖住);外离/外切返回 0。
//                         两个形参没有半径大小要求,内部会自己按 r 大小归一。
//   cir_poly_area(n, a, c) 逆时针**简单**多边形 a[0..n-1](凹也正确)与圆的交面积,恒 >= 0。
//                         做法:逐边求「三角形(圆心, a[i], a[i+1]) 与圆盘」的有向交面积再累加
//                         (圆心到各边的扇形剖分,有向面积自动抵消凹口);扇形一律用**有向**角
//                         arg2(u, v) = atan2(叉积, 点积) ∈ (-pi, pi],不能归一到 [0, 2pi) ——
//                         圆心在多边形外时各边有向角之和本来是 0,归一化后会变成 2pi,
//                         于是「圆完全在多边形外」会错算出整整一块圆面积。
//   side_of(a, b, p)      有向直线 ab(a->b) 的左侧(>0)/右侧(<0)/线上(0),内部复用。
//
// 复杂度:c 个交点类的接口 O(1);cir_cir_area O(1);cir_poly_area O(n)(n 为多边形顶点数)。
// 退化约定:圆与直线无交返回空;相切返回 1 个点(判别式在 eps 内按 0 处理);两圆重合/内含
//   在 cir_cir 里返回空,而 cir_cir_area 里返回 pi*r^2;r = 0 的圆视为点,面积恒 0。

struct circle {
    p2 o;
    db r;
};

// 有向直线 ab(a -> b) 与 p:> 0 表示 p 在左侧,< 0 右侧,0 共线(eps 容差)
int side_of(const p2 &a, const p2 &b, const p2 &p) { return sign((b - a).det(p - a)); }

// 圆与直线的交点。n = 0/1/2 个,按 t = dir*(p - l.x) 递增排序;圆心在直线上时关于垂足对称。
// 前置:l.x != l.y(退化返回空)。O(1)
int cir_line2(const circle &c, const seg &l, p2 *o) {
    p2 d = l.y - l.x;
    db L2 = dis2(d);
    if(!sign(L2)) return 0;                                  // 退化直线
    p2 f = proj(l, c.o);                                     // 圆心在直线上的投影(垂足)
    db x = c.r * c.r - dis2(c.o - f);                        // = (r - |of|)(r + |of|)
    // 相切判据必须按半径缩放:x ≈ 2r(r - |of|),除以 r 之后才和 geo 的**绝对** eps = 1e-10 可比。
    // 不缩放的话半径 1e-6 的圆(判别式 ~1e-12)会被判成相切、只返回 1 个交点。
    db sc = cmp(c.r, 0) > 0 ? x / c.r : x;
    if(sign(sc) < 0) return 0;                               // 相离
    db h = sign(sc) > 0 ? sqrtl(x) : 0;                      // 半弦长,相切时只有垂足一个点
    p2 u = d / sqrtl(L2);                                    // 直线方向的单位向量
    int n = 0;
    if(sign(h) > 0) { // 两个交点:垂足沿直线方向偏移 ±h
        o[n++] = f + u * h;
        o[n++] = f - u * h;
    } else o[n++] = f; // 相切:1 个点
    return n;
}
// 圆与直线交点的向量返回版,按参数 t = dir*(p - l.x) 递增;退化直线返回空。
vector<p2> cir_line(const circle &c, const seg &l) {
    p2 t[2];
    int n = cir_line2(c, l, t);
    vector<p2> r(t, t + n);
    if(n == 2 && t[0] * (l.dir()) > t[1] * (l.dir())) swap(r[0], r[1]);  // 按参数(取 l.x 为原点)递增
    return r;
}

// 圆与线段的交点(直线交点再用 ons 过滤)。按 t 递增。O(1)
vector<p2> cir_seg(const circle &c, const seg &l) {
    vector<p2> r;
    for(p2 x : cir_line(c, l))
        if(ons(l, x)) r.push_back(x);
    return r;
}

// 两圆的交点:0(外离/内含/内离/同心/重合)/ 1(内切或外切)/ 2 个,按相对 c1.o 的极角排序。
```

```cpp
// 做法:沿 o1->o2 方向量 d,再沿垂线取 +-h,并逐个验证到两圆心的距离确实都是 r。O(1)
vector<p2> cir_cir(const circle &c1, const circle &c2) {
    p2 dv = c2.o - c1.o;
    db d = dis(dv);
    if(!sign(d)) return {};                                  // 同心/重合:无孤立交点
    p2 u = dv / d;                                           // o1 -> o2 的单位方向
    db x = (d * d + c1.r * c1.r - c2.r * c2.r) / (2 * d);     // 交点连线到 o1 的距离
    db h2 = c1.r * c1.r - x * x;
    // 同 cir_line2:相切判据按半径缩放(否则半径 1e-6 的两圆会被判成相切/相离)
    db sc2 = cmp(c1.r, 0) > 0 ? h2 / c1.r : h2;
    if(sign(sc2) < 0) return {};                             // 两圆相离(含内含)
    vector<p2> r;
    p2 base = c1.o + u * x;
    if(sign(sc2) > 0) {
        db h = sqrtl(h2);
        p2 w = r90(u) * h;
        r.push_back(base + w);
        r.push_back(base - w);
    } else
        r.push_back(base);                                   // 相切:两处解重合,只留 1 个
    auto good = [&](const p2 &p) {
        db t1 = dis(p - c1.o) - c1.r, t2 = dis(p - c2.o) - c2.r;
        db e = eps * (1 + max(c1.r, c2.r));
        return abs(t1) <= e && abs(t2) <= e;                 // 防止 r = 0 / 极端半径比下的伪交点
    };
    vector<p2> q;
    for(p2 p : r)
        if(good(p)) q.push_back(p);
    sort(q.begin(), q.end(), [&](const p2 &a, const p2 &b) { return (a - c1.o).alpha() < (b - c1.o).alpha(); });
    return q;
}

// 两圆「交」面积(解析法:两个弓形之和)。设 d 为圆心距(0 < |r1 - r2| < d < r1 + r2),
// x = (d^2 + r1^2 - r2^2) / (2d) 是公共弦到圆 1 圆心的距离,h = sqrt(r1^2 - x^2) 是公共弦的一半,
// a_i = 2 * acos(...) 是交点在圆 i 里的圆心角;则 S = (r1^2 a1 / 2 - d h / 2) + (r2^2 a2 / 2 - d h / 2),
// 即两个「扇形 - 三角形」弓形之和。内含/内切/重合返回 pi * min(r1,r2)^2(小圆整个被盖住),
// 外离/外切返回 0(容差 eps)。形参无半径大小要求,内部按 r 归一。O(1)
db cir_cir_area(const circle &c1, const circle &c2) {
    db r1 = c1.r, r2 = c2.r;
    if(r1 < r2) swap(r1, r2);                                // 保证 r1 >= r2
    db d = dis(c1.o - c2.o);
    if(d >= r1 + r2 - eps) return 0;                         // 外离 / 外切:交面积 0
    if(d <= r1 - r2 + eps) return pi * r2 * r2;              // 内含 / 内切:小圆整个被盖住
    db a1 = 2 * acosl((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d));   // 交点在圆 1 里的圆心角
    db a2 = 2 * acosl((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d));   // 交点在圆 2 里的圆心角
    db x = (d * d + r1 * r1 - r2 * r2) / (2 * d);            // 公共弦到圆 1 圆心的距离
    db h = sqrtl(max((db) 0, r1 * r1 - x * x));              // 公共弦的一半
    return max((db) 0, (r1 * r1 * a1 / 2 - d * h / 2) + (r2 * r2 * a2 / 2 - d * h / 2));
}

// —— cir_poly_area 的零件:圆盘与「三角形 (圆心, a, b)」的交(有向面积 * 2) ——
// 从 u 到 v 的**有向**夹角,取值 (-pi, pi]:atan2(叉积, 点积)。
// 这里必须用有向角,不能把角差归一到 [0, 2pi) —— 圆心落在多边形外时各边扇形角的有向和是 0,
// 归一化之后会变成 2pi,于是「圆完全在多边形外」会算出整整一块圆面积。
db arg2(const p2 &u, const p2 &v) { return atan2l(u.det(v), u * v); }
// a、b 先平移到「以圆心为原点」;va/vb = a、b 是否在圆盘内(含边界)。三种情形:
//   · a、b 都在圆内 -> 整块三角形都在圆内:a.det(b)
//   · 线段 ab 与圆盘无交 -> 只贡献一个扇形:arg2(a, b) * r^2
//   · 否则 -> 沿 a -> b 把「弧 + 弦上那段直线段」依次拼起来(扇形 + 三角形)
db tri_disk2(const circle &c, p2 a, p2 b) {
    a = a - c.o, b = b - c.o;
    db r2 = c.r * c.r;
    // 判「点是否在圆盘内」比的是**距离**而不是平方距离:平方距离在半径 1e-6 时只有 1e-12,
    // 会被 geo 的绝对 eps = 1e-10 全部判成「在圆内」
    int va = cmp(dis(a), c.r) <= 0, vb = cmp(dis(b), c.r) <= 0;
    if(va && vb) return a.det(b);
    vector<p2> v = cir_seg(circle{{0, 0}, c.r}, {a, b});  // 圆(圆心在原点)与线段 ab 的交点,按 a->b 递增
    if(v.empty()) return arg2(a, b) * r2;                 // 整条边在圆盘外:纯扇形(有向角,不用归一)
    db s = 0;
    s += va ? a.det(v[0]) : arg2(a, v[0]) * r2;           // 起点那一侧的扇形/三角形
    s += vb ? v.back().det(b) : arg2(v.back(), b) * r2;   // 终点那一侧
    if(v.size() > 1) s += v[0].det(v[1]);                 // 圆内那段弦与圆心围成的三角形
    return s;
}

// 逆时针**简单**多边形 a[0..n-1](凹也对)与圆的交面积,恒 >= 0。O(n)
// 做法:对每条边 (a[i], a[i+1]) 求「三角形(圆心, a[i], a[i+1]) 与圆盘」的有向交面积并累加 ——
// 这些三角形是圆心的「扇形剖分」,有向面积自动抵消外部与凹口,所以凹多边形同样正确。
// 退化:r 在 eps 内按 0 处理;多边形面积 0 时结果 0。顺时针多边形不在契约内(会返回 0)。
db cir_poly_area(int n, p2 *a, const circle &c) {
    if(n < 3 || !sign(c.r)) return 0;
    db s = 0;
    for(int i = 0; i < n; ++i) s += tri_disk2(c, a[i], a[(i + 1) % n]);
    return s > 0 ? s / 2 : 0;  // 逆时针 -> 有向和为正;数值噪声造成的极小负值归零
}
```

7.13 多边形包含

```cpp
//
// int wn_contain(int n, p2 *a, const p2 &p): 用**缠绕数**(nonzero winding rule)判定 p 与
//   多边形 a[0..n-1] 的关系,返回 0:外;1:在边界上(边或顶点);2:缠绕数非零(内部)。
//   与 geo.cpp 的 contain(射线法,**奇偶**规则)的区别:自交多边形上两者会给出不同答案 —
```

```
//   例如蝴蝶结 (0,0),(4,4),(4,0),(0,4) 的两个「翅膀」缠绕数分别为 ±1(非零 -> 判内),
//   而奇偶规则在其中一个翅膀里会判成外部。要处理「带孔多边形/自交多边形」时用本函数。
//   前置条件:n >= 3;相邻顶点可以重复(退化边会被跳过);顶点顺序无所谓(逆/顺时针都行)。
//   复杂度 O(n),不用 atan2,只有跨 y 符号与叉积符号的判定。
//   依赖:geo.cpp 的 p2 / eps / sign / cmp / crossop / ons。

int wn_contain(int n, p2 *a, const p2 &p) {
    if(n < 3) return 0;
    int wn = 0;
    for(int i = 0; i < n; ++i) {
        p2 u = a[i], v = a[(i + 1) % n];
        if(u == v) continue;  // 退化边(相邻重复点):跳过,否则 ons 会认为任意点都在它上面
        if(ons({u, v}, p)) return 1;
        int su = sign(u.y - p.y), sv = sign(v.y - p.y);
        if(su <= 0 && sv > 0 && crossop({u, v}, p) > 0) ++wn;       // 向上穿过 p 所在水平线且 p 在左
        else if(su > 0 && sv <= 0 && crossop({u, v}, p) < 0) --wn;  // 向下穿过且 p 在右
    }
    return wn ? 2 : 0;
}
```

7.14 最小圆覆盖

```
//
// struct circle { p2 o; db r; };            // 本文件自带(geo.cpp 里没有 circle;若与 图形交.cpp 同时
//                                           // include,删掉其中一份定义即可,两份完全同构)
// circle mcc(vector<p2> a): 返回覆盖 a 中**所有点**的最小圆的圆心 o 与半径 r。
//   期望复杂度 O(n)(随机增量法,内部先 shuffle;最坏 O(n^3)),空间 O(n)。
//   前置条件:n >= 1;点可以重复。空输入返回 { (0,0), -1 } 作为「无解」标记(调用方自己判 r < 0)。
//   退化处理:三点共线时不做外接圆(会除零),改用「最远两点为直径」。这一步是**防御性**的:
//   正常路径很少走到,留着是为了输入里出现极端浮点噪声时不炸。
//   依赖:geo.cpp 的 p2 / eps / dis / dis2 / crossop / cmp / circumcenter,以及 std 的
//         mt19937_64 / shuffle / chrono(自带随机源,不依赖外部 rng)。

struct circle {
    p2 o;
    db r;
};
// 返回以 ab 为直径的圆;允许 a=b,此时半径为 0。
circle cir(p2 a, p2 b) { return {(a + b) / 2, dis(a - b) / 2}; }
// 点 x 是否在圆 c 内(含边界,eps 容差)。
bool in_circle(const circle &c, const p2 &x) { return cmp(dis(c.o - x), c.r) <= 0; }
// 最小圆覆盖(随机增量法);返回圆心与半径,空输入返回 { (0,0), -1 } 作无解标记。
circle mcc(vector<p2> a) {
    if(a.empty()) return {{0, 0}, -1};
    static mt19937_64 mcc_rng(chrono::steady_clock::now().time_since_epoch().count());
    shuffle(a.begin(), a.end(), mcc_rng);
    circle c = {a[0], 0};
    for(int i = 1; i < (int) a.size(); ++i) {
        if(in_circle(c, a[i])) continue;
        c = cir(a[0], a[i]);
        for(int j = 0; j < i; ++j) {
            if(in_circle(c, a[j])) continue;
            c = cir(a[i], a[j]);
            for(int k = 0; k < j; ++k) {
                if(in_circle(c, a[k])) continue;
                if(!crossop({a[i], a[j]}, a[k])) {  // 三点共线:最小圆取最远两点的直径
                    db dij = dis2(a[i] - a[j]), dik = dis2(a[i] - a[k]), djk = dis2(a[j] - a[k]);
                    if(dij >= dik && dij >= djk)
                        c = cir(a[i], a[j]);
                    else if(dik >= djk)
                        c = cir(a[i], a[k]);
                    else
                        c = cir(a[j], a[k]);
                } else {
                    auto o = circumcenter(a[i], a[j], a[k]);
                    p2 oc = {o[0], o[1]};
                    c = {oc, dis(oc - a[k])};
                }
            }
        }
    }
    return c;
}
```

7.15 最近点对

```
//
// dis_closest(vector<p2> a): 返回 a 中最近两个点的**欧氏距离**(不是平方);两点重合时返回 0。
//   n < 2 时返回 1e100 作为「无解」标记(调用方自己判)。
//   分治 + 按 y 归并:每个递归段先把左右两半按 y 归并(段内保持 y 有序),再只检查横坐标距
//   分界线 < ans 的窄条,窄条内每个点只需回看 y 坐标差 < ans 的点 —— 经典结论下只有 O(1) 个。
//   复杂度 O(n log n) 时间、O(n) 额外空间(归并缓冲一次分配);要求 n >= 2。
//   依赖:geo.cpp 的 p2 / dis / eps,以及 std 的 sort / inplace_merge 语义(手写归并)。

db dis_closest(vector<p2> a) {
    int n = a.size();
    if(n < 2) return 1e100L;
    sort(a.begin(), a.end(), [](const p2 &u, const p2 &v) { return u.x != v.x ? u.x < v.x : u.y < v.y; });
    vector<p2> b(n);  // 归并缓冲 + 窄条缓冲,只分配一次
    db ans = 1e100L;
    auto rec = [&](auto &&self, int l, int r) -> void {  // 处理 [l, r),结束后 a[l..r) 按 y 有序
        if(r - l <= 1) return;
        int mid = (l + r) >> 1;
        db mx = a[mid].x;
        self(self, l, mid), self(self, mid, r);
        int i = l, j = mid, k = l;  // 归并两半(都按 y 有序)
        while(i < mid && j < r) b[k++] = a[i].y <= a[j].y ? a[i++] : a[j++];
```

```
        while(i < mid) b[k++] = a[i++];
        while(j < r) b[k++] = a[j++];
        for(int t = l; t < r; ++t) a[t] = b[t];
        int m = 0;  // 只留下横坐标距分界线 < ans 的点(按 y 递增)
        for(int t = l; t < r; ++t)
            if(abs(a[t].x - mx) < ans) b[m++] = a[t];
        for(int t = 0; t < m; ++t) for(int u = t - 1; u >= 0 && b[t].y - b[u].y < ans; --u) ans = min(ans, dis(b[t] - b[u]));
    };
    rec(rec, 0, n);
    return ans;
}
```

7.16 简单多边形三角剖分

```
//
// 参数: n 顶点数, a 顶点数组(逆时针顺序, 只读, 剖分中不改写)
// 返回: vector<array<int, 3>>, 每项是一个三角形的顶点下标三元组 (i, j, k), 按逆时针排列。
//       无三点共线时恰好 n - 2 个三角形、每个下标都出现, 每个三角形面积都严格为正;
//       有多余共线点时(边上插了点)先把共线点从环上摘掉, 于是只有「真顶点」参与剖分:
//       三角形数 = 非共线点数 - 2, 表面积和 == 多边形面积, 每个三角形面积仍严格为正。
// 前置: 简单多边形(边不自交)、顶点逆时针、无重复点; 无三点共线时行为最干净。
//       多边形退化成面积 0(n < 3 或去掉共线点后不足 3 个)时返回空 vector。
// 做法: 经典耳切。反复找一个「耳」——顶点 prev, cur, next 处左转(凸), 且三角形
//       (prev, cur, next) 内不含环上任何其它顶点——把它切下来, 把 cur 从环里删掉;
//       对逆时针简单多边形, 双耳定理保证至少存在两个耳, 于是总能切到只剩一个三角形。
//       先把 det == 0 的共线点摘掉(它们不影响多边形形状, 只让耳切卡住: 共线点两侧的角
//       可能被切成 180 度, 之后两个方向的角都不是凸的, 环就再也切不动了)。
// 实现: prv/nxt 下标数组维护双向链表, 每轮沿环扫一遍找耳, 切掉一个就从头重扫(共 O(n) 轮)。
//       候选必须是环上的活点(判据 prv[nxt[x]] == x && nxt[prv[x]] == x): 被删掉的点
//       prv/nxt 仍指着旧邻居, 只看凸性的话会被当成耳再切一次, 输出重复三角形。
// 判定: 凸性看 (cur - prev) × (next - cur) > 0, 与 crossop > 0 同语义; 顶点落在耳内要求
//       三角形的三条边 det 全部 > 0(落在耳边上的顶点不算「在内部」, 于是共线点也不会卡死)。
//       这里刻意写成精确比较而不是 sign(x) > 0: geo.cpp 的 eps = 1e-10 是绝对容差, 几何尺度
//       小于 1e-5 时(例如边长 1e-9 的正方形)所有 det 都会被当成 0, 一个耳都找不到 —— 见尾注。
// 复杂度: O(n^2) 时间, O(n) 空间。
// 依赖: 需先有 geo.cpp 的 p2 / db / det(本文件不再自带 p2、eps、sign)。

vector<array<int, 3>> ear_clip(int n, p2 *a) {
    vector<array<int, 3>> ret;
    if(n < 3) return ret;
    // 「p 严格落在逆时针三角形 (x, y, z) 内部」: 三条边的左半平面都在左侧
    auto inside = [&](const p2 &p, const p2 &x, const p2 &y, const p2 &z) {
        return (y - x).det(p - x) > 0 && (z - y).det(p - y) > 0 && (x - z).det(p - z) > 0;
    };
    // 环上的前驱、后继: 用下标数组维护双向链表, 不真的删点, 避免下标错位
    // cnt 是环上剩下的点数, 不能拿 n 当计数器 —— n 还是数组的大小
    vector<int> prv(n), nxt(n);
    for(int i = 0; i < n; ++i) prv[i] = (i + n - 1) % n, nxt[i] = (i + 1) % n; // for(int i = l; i < r; ++i) 即 i = l..r-1
    int cnt = n;
    auto live = [&](int x) { return prv[nxt[x]] == x && nxt[prv[x]] == x; };  // 还在环上
    auto drop = [&](int x) {
        int p = prv[x], q = nxt[x];
        nxt[p] = q, prv[q] = p;
        --cnt;
    };
    // 预处理: 摘掉三点共线的中间点。判据用「局部相对」容差: 三个点的坐标本来就带舍入,
    // 插在边上的点常常只是 det ≈ 1e-14 而非精确 0, 若只用 == 0 会漏掉它们, 之后就会切出
    // 一个面积 ~1e-14 的退化三角形。容差取 1e-13 * |cur - prev| * |next - cur|(叉积的
    // 浮点误差上界就是这个量级), 这样薄三角形(底 2e9 高 1、det = 2e9)不会被误判成共线。
    // 只有这一步用容差, 耳切本身仍然用精确比较。
    auto colin = [&](int x) {
        p2 u = a[x] - a[prv[x]], v = a[nxt[x]] - a[x];
        return abs(u.det(v)) <= 1e-13L * dis(u) * dis(v);
    };
    // 摘掉一个点会暴露新的共线点, 所以反复扫, 直到环上不再有共线点
    for(bool ch = 1; ch && cnt >= 3;) {
        ch = 0;
        for(int x = 0; x < n; ++x)
            if(live(x) && cnt > 3 && colin(x)) {
                drop(x);
                ch = 1;
            }
    }
    if(cnt < 3) return ret;  // 全是共线点: 面积 0, 没有合法三角形
    for(; cnt > 2;) {
        int ear = -1;
        for(int cur = 0; cur < n; ++cur) {
            if(!live(cur)) continue;
            int p = prv[cur], q = nxt[cur];
            if(p == cur || q == cur || p == q) continue;
            if((a[cur] - a[p]).det(a[q] - a[cur]) <= 0) continue;  // 该顶点不是严格凸的
            bool ok = 1;
            for(int i = nxt[q]; i != p && ok; i = nxt[i])
                if(inside(a[i], a[p], a[cur], a[q])) ok = 0;
            if(ok) {
                ear = cur;
                break;
            }
        }
        if(ear < 0) break;  // 一个耳都没有(输入不满足前置条件): 剩下的切不动, 直接停
        int p = prv[ear], q = nxt[ear];
        ret.push_back({p, ear, q});  // (prev, cur, next) 左转, 按逆时针输出
        drop(ear);
    }
    // 只剩 3 个点时上面那个循环不再进入: 若这三点共线(整个多边形面积 0), 没有合法三角形
    if(cnt == 3 && (a[nxt[0]] - a[0]).det(a[prv[0]] - a[0]) == 0) ret.clear();
    return ret;
}
// 注: 若确实想用 geo.cpp 的 sign 做容差判定, 请先把 det 归一化 —— sign 用的是绝对 eps,
//     大坐标下几乎不生效(1e18 的 det 永远非 0), 小尺度下又会把整个多边形判成退化。
```

7.17 多边形重心

```cpp
//
// p2 centroid(int n, p2 *a): 简单多边形 a[0..n-1] 的**面积重心**(形心)。
//   公式:x = Σ(x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i) / (6A),y 同理,A 是带号面积;
//   顶点顺序任意(逆/顺时针都行,带号面积会自动带出符号,结果一样)。
//   前置条件:n >= 3 的**简单多边形**(自交时按带号面积的正负部分相抵,结果没有几何意义)。
//   退化(A == 0,例如全共线/自交抵消)时返回顶点算术平均(调用方需自己判断是否可信)。
//   复杂度 O(n),空间 O(1)。n == 0 返回 (0,0)。
//   依赖:geo.cpp 的 p2 / eps / sign / dis(带号面积也可以直接复用 area(n, a))。
//   配套:db perimeter(int n, p2 *a) 返回闭合周长,同样 O(n)。

// 多边形闭合周长(顶点顺序任意)。
db perimeter(int n, p2 *a) {
    db s = 0;
    for(int i = 0; i < n; ++i) s += dis(a[(i + 1) % n] - a[i]);
    return s;
}
// 面积重心(形心);要求简单多边形,退化(A == 0)返回顶点算术平均。
p2 centroid(int n, p2 *a) {
    if(n == 0) return {0, 0};
    p2 ret = {0, 0};
    db s = 0;
    for(int i = 0; i < n; ++i) {
        p2 u = a[i], v = a[(i + 1) % n];
        db c = u.det(v);
        s += c;
        ret = ret + (u + v) * c;
    }
    if(!sign(s)) {  // 退化:退回顶点平均
        p2 av = {0, 0};
        for(int i = 0; i < n; ++i) av = av + a[i];
        return av / (db) n;
    }
    return ret / (3 * s);
}
```

8 通用

8.1 大质数表

每个数量级取 10^k 以上最小的 10 个素数横着排,表里只写偏移 x(读作 10^k+x)。

10^k	1	2	3	4	5	6	7	8	9	10
10^2	+1	+3	+7	+9	+13	+27	+31	+37	+39	+49
10^3	+9	+13	+19	+21	+31	+33	+39	+49	+51	+61
10^4	+7	+9	+37	+39	+61	+67	+69	+79	+91	+93
10^5	+3	+19	+43	+49	+57	+69	+103	+109	+129	+151
10^6	+3	+33	+37	+39	+81	+99	+117	+121	+133	+151
10^7	+19	+79	+103	+121	+139	+141	+169	+189	+223	+229
10^8	+7	+37	+39	+49	+73	+81	+123	+127	+193	+213
10^9	+7	+9	+21	+33	+87	+93	+97	+103	+123	+181
10^{10}	+19	+33	+61	+69	+97	+103	+121	+141	+147	+207
10^{11}	+3	+19	+57	+63	+69	+73	+91	+103	+129	+171
10^{12}	+39	+61	+63	+91	+121	+163	+169	+177	+189	+193
10^{13}	+37	+51	+99	+129	+183	+259	+267	+273	+279	+283
10^{14}	+31	+67	+97	+99	+133	+139	+169	+183	+261	+357
10^{15}	+37	+91	+159	+187	+223	+241	+249	+259	+273	+279
10^{16}	+61	+69	+79	+99	+453	+481	+597	+613	+639	+669
10^{17}	+3	+13	+19	+21	+49	+81	+99	+141	+181	+337
10^{18}	+3	+9	+31	+79	+177	+183	+201	+283	+381	+387

8.2 常数速查表

n	1	2	3	4	5	6	7	8	9	10
log10 n	0	0.30103	0.477121	0.60206	0.69897	0.778151	0.845098	0.90309	0.954243	1
n!	1	2	6	24	120	720	5040	40320	362880	3628800
C(n, n/2)	1	2	3	6	10	20	35	70	126	252
lcm(1..n)	1	2	6	12	60	60	420	840	2520	2520
P_n	1	2	3	5	7	11	15	22	30	42
n	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**
log10 n	1.041393	1.079181	1.113943	1.146128	1.176091	1.20412	1.230449	1.255273	1.278754	1.30103
n!	39916800	479001600	6.23e9	8.72e10	1.31e12	2.09e13	3.56e14	6.40e15	1.22e17	2.43e18
C(n, n/2)	462	924	1716	3432	6435	12870	24310	48620	92378	184756
lcm(1..n)	27720	27720	360360	360360	360360	720720	12252240	12252240	232792560	232792560
P_n	56	77	101	135	176	231	297	385	490	627
n	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**
log10 n	1.322219	1.342423	1.361728	1.380211	1.39794	1.414973	1.431364	1.447158	1.462398	1.477121
n!	5.11e19	1.12e21	2.59e22	6.20e23	1.55e25	4.03e26	1.09e28	3.05e29	8.84e30	2.65e32
C(n, n/2)	352716	705432	1352078	2704156	5200300	10400600	20058300	40116600	77558760	155117520
lcm(1..n)	232792560	232792560	5.35e9	5.35e9	2.68e10	2.68e10	8.03e10	8.03e10	2.33e12	2.33e12
P_n	792	1002	1255	1575	1958	2436	3010	3718	4565	5604

P_n 超过 10^9 的最小 $n=115$;$P_{40}=37338$、$P_{50}=204226$、$P_{60}=966467$、$P_{100}=190569292$。

卡特兰数 $C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}$,递推 $C_n=C_{n-1}\frac{4n-2}{n+1}$;$C_{35}<2^{63}(n\le 35$ 都在 64 位整数内),表里 $n\ge 19$ 起改写成 3 位有效数字的科学记数法。

n	0	1	2	3	4	5	6	7	8	9
C_n	1	1	2	5	14	42	132	429	1430	4862
n	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**
C_n	16796	58786	208012	742900	2674440	9694845	35357670	129644790	477638700	1.77e9
n	**20**	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**
C_n	6.56e9	2.45e10	9.15e10	3.43e11	1.29e12	4.86e12	1.84e13	6.95e13	2.64e14	1.00e15
n	**30**	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**
C_n	3.81e15	1.45e16	5.55e16	2.12e17	8.13e17	3.12e18	1.20e19	4.60e19	1.77e20	6.80e20

10^k 以内的增长上界:ω 是不同质因子个数,d 是因子个数,π(n) 是不超过 n 的素数个数。

$n\le$	max ω(n)	max d(n)	π(n)	$n\le$	max ω(n)	max d(n)	π(n)
10^1	2	4	4	10^{10}	10	2304	4.55e8
10^2	3	12	25	10^{11}	10	4032	4.12e9
10^3	4	32	168	10^{12}	11	6720	3.7e10
10^4	5	64	1229	10^{13}	12	10752	—
10^5	6	128	9592	10^{14}	12	17280	—
10^6	7	240	78498	10^{15}	13	26880	—
10^7	8	448	664579	10^{16}	13	41472	—
10^8	8	768	5761455	10^{17}	14	64512	—
10^9	9	1344	5.08e7	10^{18}	15	103680	—

素数定理:$\pi(x)\approx\frac{x}{\ln x}$;— 表示未列出值。

高合成数:从小到大,因子个数刷新纪录的 n;左右两组依次往下读。

n	分解	d(n)	n	分解	d(n)
2	2	2	6983776800	$2^5 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	2304
4	2^2	3	13967553600	$2^6 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	2688
6	$2 \cdot 3$	4	27935107200	$2^7 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	3072
12	$2^2 \cdot 3$	6	64250746560	$2^6 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	3584
24	$2^3 \cdot 3$	8	97772875200	$2^6 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	4032
60	$2^2 \cdot 3 \cdot 5$	12	128501493120	$2^7 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	4096
120	$2^3 \cdot 3 \cdot 5$	16	240940299600	$2^4 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	4800
240	$2^4 \cdot 3 \cdot 5$	20	481880599200	$2^5 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	5760
360	$2^3 \cdot 3^2 \cdot 5$	24	963761198400	$2^6 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	6720
840	$2^3 \cdot 3 \cdot 5 \cdot 7$	32	1927522396800	$2^7 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	7680
1680	$2^4 \cdot 3 \cdot 5 \cdot 7$	40	3373164194400	$2^5 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	8640
2520	$2^3 \cdot 3^2 \cdot 5 \cdot 7$	48	6746328388800	$2^6 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	10080
7560	$2^3 \cdot 3^3 \cdot 5 \cdot 7$	64	9316358251200	$2^6 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29$	10752
15120	$2^4 \cdot 3^3 \cdot 5 \cdot 7$	80	13492656777600	$2^7 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23$	11520
27720	$2^3 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11$	96	32607253879200	$2^5 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29$	13824
55440	$2^4 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11$	120	65214507758400	$2^6 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29$	16128
83160	$2^3 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11$	128	97821761637600	$2^5 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29$	17280
110880	$2^5 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11$	144	130429015516800	$2^7 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29$	18432
221760	$2^6 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11$	168	260858031033600	$2^8 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29$	20736
498960	$2^4 \cdot 3^4 \cdot 5 \cdot 7 \cdot 11$	200	391287046550400	$2^7 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29$	23040
720720	$2^4 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11 \cdot 13$	240	866421317361600	$2^6 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	26880
1441440	$2^5 \cdot 3^2 \cdot 5 \cdot 7 \cdot 11 \cdot 13$	288	1010824870255200	$2^5 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	27648
3603600	$2^4 \cdot 3^2 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13$	360	2021649740510400	$2^6 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	32256
7207200	$2^5 \cdot 3^2 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13$	432	4043299481020800	$2^7 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	36864
8648640	$2^6 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11 \cdot 13$	448	8086598962041600	$2^8 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	41472
14414400	$2^6 \cdot 3^2 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13$	504	12129898443062400	$2^7 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	46080
32432400	$2^4 \cdot 3^4 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13$	600	30324746107656000	$2^6 \cdot 3^4 \cdot 5^3 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	53760
61261200	$2^4 \cdot 3^2 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17$	720	60649492215312000	$2^7 \cdot 3^4 \cdot 5^3 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31$	61440
73513440	$2^5 \cdot 3^3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17$	768	74801040398884800	$2^6 \cdot 3^3 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$	64512
122522400	$2^5 \cdot 3^2 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17$	864	112201560598327200	$2^5 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$	69120
245044800	$2^6 \cdot 3^2 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17$	1008	224403121196654400	$2^6 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$	80640
367567200	$2^5 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17$	1152	448806242393308800	$2^7 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$	92160
735134400	$2^6 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17$	1344	897612484786617600	$2^8 \cdot 3^4 \cdot 5^2 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$	103680
2095133040	$2^4 \cdot 3^4 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	1600	1122015605983272000	$2^6 \cdot 3^4 \cdot 5^3 \cdot 7^2 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 \cdot 29 \cdot 31 \cdot 37$	107520
3491888400	$2^4 \cdot 3^3 \cdot 5^2 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19$	1920			